GRIDWISE | PROJECT DOCUMENTATION

GridWise

Smart Building Energy Peak-Shaving Controller

Smarter demand. Safer peak management.

Prepared by

C Lokesh Chowdary – EDGE IT ANALYST - DTCC

GitHub Repository: https://github.com/Loki-0110/smart-building-energy-controller

---

PROJECT OVERVIEW

GridWise is a simulation-backed building energy application that monitors electricity demand, detects threshold-based peak events, evaluates equipment eligibility and presents explainable load-reduction recommendations for operator review. It combines an operational web application with a separate analytics path for historical reporting.

KEY CAPABILITIES

- Building, equipment, occupancy and comfort-rule management.
- Synthetic meter ingestion, validation and duplicate handling.
- Threshold-based peak detection and protected-equipment eligibility checks.
- Advisory load schedules with operator approval, rejection and override workflow.
- MySQL operational storage, Snowflake analytics and Power BI reporting.
- Docker-based deployment on AWS EC2 with GitHub Actions CI/CD.

PROJECT BOUNDARY | Simulation and decision support only — no connection to BMS/SCADA and no physical equipment actuation. Expected reduction values are estimates, not measured savings.

GRIDWISE APPLICATION DASHBOARD

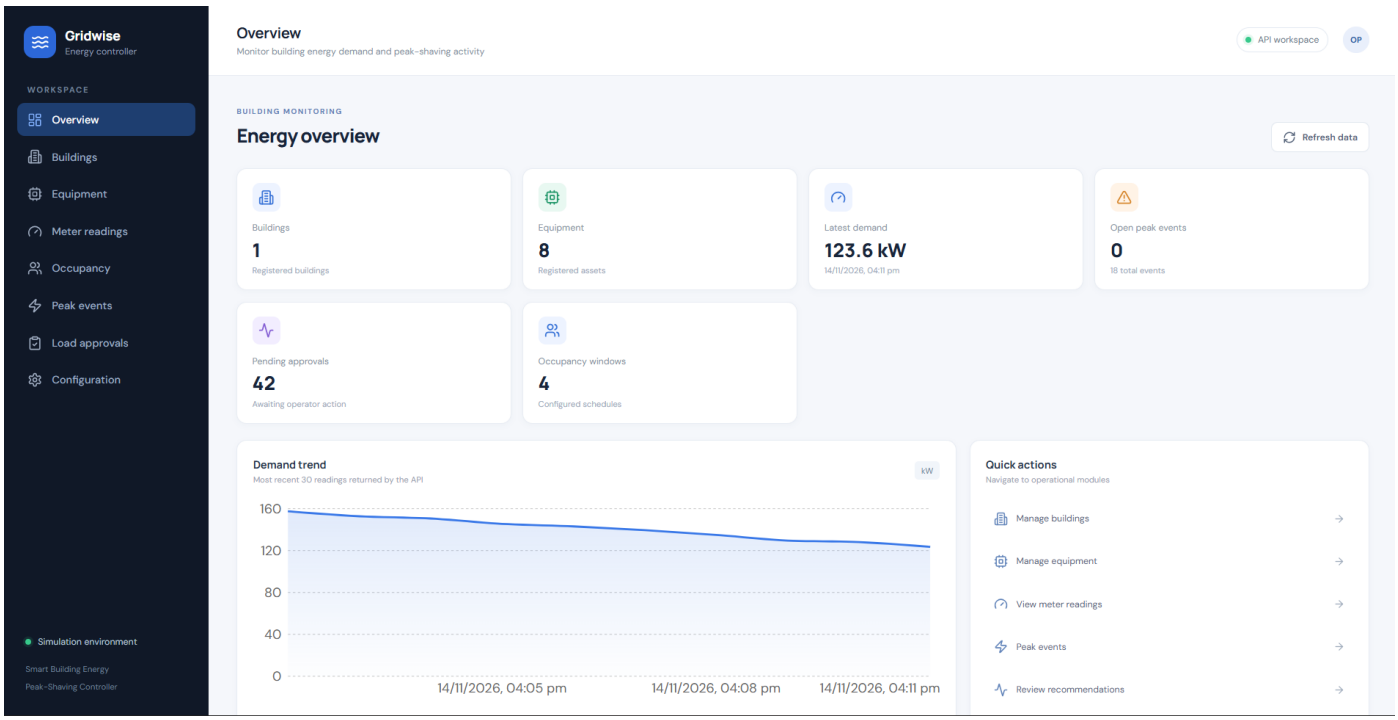


Figure 1. Operator dashboard showing the energy overview, building monitoring navigation, KPI cards and demand trend.

Project at a Glance

The Smart Building Energy Peak-Shaving Controller is a simulation-backed application that monitors building electricity demand, detects threshold-based peak events, evaluates equipment eligibility and presents explainable load-reduction recommendations for operator review. The solution combines an operational web application with a separate analytics path for historical reporting.

The system is intentionally advisory. It does not connect to building-management hardware, issue commands to electrical equipment, or prove real-world energy savings. Values such as EXPECTED_REDUCTION_KW are estimates generated by the application and must be labelled as simulated.

Area	Implemented responsibility
Operator experience	React/Vite web application for building, equipment, occupancy, peak configuration, events and schedule approvals.
Application services	Spring Boot REST API, validation, persistence, peak detection, recommendation and operator workflow.
Operational data	MySQL 8 database named smart_building_db.
Meter generation	Python simulator creates synthetic readings, validates them and submits accepted readings through the backend API.
Analytics	Python ETL integrates operational data with Snowflake; Power BI consumes Snowflake data for reporting.
Cloud runtime	Docker containers hosted on AWS EC2; GitHub Actions deploys updated images through a self-hosted runner.
Backup	MySQL dump files are stored in a private S3 bucket. The documented backup operation is manual.

How to Use This Document

A new developer should read Sections 1–7 for the product, architecture and decision logic; Sections 8–13 for the database and application components; and Sections 14–22 for containerization, deployment, setup, verification and operations. The appendices contain the glossary, development notes and troubleshooting quick reference. Commands are labelled by execution location; do not run an EC2 command in Windows PowerShell or a Snowflake statement in a Windows terminal.

Implementation Status and Evidence Rules

- “Implemented” means supported by the supplied code, configuration, screenshots or deployment notes reviewed for this document.
- “Operationally observed” means a working state was shown during the project, but can change over time (for example, public IP, container IDs and record counts).
- “Not verified” means the available evidence does not prove the capability. Do not mark it complete without a new test.
- Do not copy passwords, tokens, private keys or connection strings containing credentials into this document or screenshots.

1. Project Overview

1.1 Business Context

Commercial buildings contain loads with different operational importance. During periods of high demand, operators may wish to reduce or defer selected non-critical loads. A safe decision requires more than comparing total demand with a threshold: the system must also consider equipment criticality, controllability, occupancy and comfort rules.

1.2 Project Objectives

- Generate synthetic meter readings and validate their structure and freshness.
- Persist operational readings and configuration in MySQL.
- Detect demand peaks against an active building threshold.
- Evaluate equipment using explicit protection and eligibility rules.
- Create explainable recommendations and persist their status.
- Allow an operator workflow to approve, reject or override a pending recommendation.
- Expose operational information through a React interface and REST APIs.
- Prepare historical data for Snowflake and Power BI analysis.
- Deploy the application as Docker containers on AWS EC2 and automate image deployment through GitHub Actions.

1.3 Scope Boundary

Included	Excluded
Synthetic meter ingestion; threshold-based peak detection; recommendation records; operator approval/rejection/override workflow; dashboards; analytics and cloud deployment.	Direct equipment actuation; connection to real BMS/SCADA; commercial smart-meter integration; guaranteed savings; production-grade identity and authorization unless separately implemented and verified.

2. Functional Modules

Building management

Create, list, retrieve, update and delete building records. Building records anchor equipment, readings, occupancy and peak configuration.

Equipment management

Maintain equipment type, priority, criticality, controllability and rated power. Critical and non-controllable equipment are excluded from reduction recommendations.

Occupancy management

Maintain area-level occupancy windows. Unknown occupancy is treated conservatively and blocks reduction where required.

Peak configuration

Maintain building-specific demand threshold and active configuration.

Meter readings

Accept, list, retrieve, update and delete readings. Ingestion is idempotent on building_id plus reading_timestamp.

Peak detection

Compare the latest valid demand with the active threshold and create or identify peak events.

Load schedule approvals

List recommendations and pending items; approve, reject or override a recommendation. Only PENDING recommendations can be approved/rejected.

Python ingestion

Generate synthetic readings, validate them, submit accepted readings to the backend and save rejected/skipped records for review.

Analytics and reporting

Move operational data into Snowflake and present report pages through Power BI.

3. Technology Stack and Responsibilities

Technology	Responsibility	Where it runs
Java 21 / Spring Boot	REST APIs, domain services, validation, scheduler and persistence.	Local Windows development and AWS backend container.
Maven 3.9+	Backend dependency management and build lifecycle.	Developer machine / CI runner.
React + Vite / Node.js 20+	Browser-based operator interface and API client.	Local dev server and AWS frontend container.
Python 3.12	Synthetic meter simulator, validation, API submission and ETL support.	Local virtual environment or scheduled/cloud execution path as configured.
Pandas / NumPy	Tabular validation, transformation and numerical processing.	Python processing environment.
MySQL 8	Transactional operational database.	Local Docker for development; MySQL container on EC2 for deployed app.
Snowflake	Analytical warehouse for reporting datasets.	Snowflake account.
Power BI	Interactive reporting and KPI visualization.	Power BI Desktop / connected reporting environment.
Docker / Compose	Package and run frontend, backend and database services.	Developer machine and EC2.
AWS EC2 / S3	Application host and private backup object storage.	AWS eu-north-1.
GitHub Actions / Docker Hub	Build workflow, image publishing and EC2 deployment orchestration.	GitHub-hosted workflow plus EC2 self-hosted runner; Docker Hub registry.

4. System Architecture

The solution has two logical paths. The operational path serves the browser and persists application records in MySQL. The analytics path extracts operational records, transforms and loads them into Snowflake for Power BI. S3 is used for backup object storage; do not confuse the SQL backup path with the analytics staging path unless the ETL implementation explicitly uses the same bucket/prefix.

4.1 Architecture Overview

Architecture overview

The application is divided into three separate concerns: (1) the operational application, which ingests readings, detects peaks and records recommendations; (2) the analytics path, which reads historical operational data and prepares reporting datasets; and (3) the delivery and backup paths, which deploy containers and preserve database snapshots. These paths share infrastructure but have different responsibilities and failure boundaries.

How to read the architecture

Follow the solid arrows for normal application and data movement. The browser opens the React frontend on the EC2 web entry point. React calls Spring Boot REST APIs. Spring Boot reads and writes MySQL and coordinates the scheduler, peak detection and recommendation rules. Python ingestion submits validated readings through the backend API. A separate Python ETL reads operational records and loads Snowflake; Power BI reads the analytical model after refresh. Dashed arrows show deployment and manual backup operations, not live business transactions.

Key component responsibilities

- React frontend — operator-facing screens for buildings, equipment, meter readings, peak events and schedule decisions. It does not own the safety rules.
- Spring Boot backend — authoritative REST API, validation, persistence, scheduler coordination, peak detection and scheduling business rules.
- MySQL 8 — operational source of truth for buildings, equipment, readings, occupancy, comfort rules, peak events, schedules and override records.
- Python ingestion — generates or processes meter readings, validates candidates and submits accepted readings to the backend. It does not make final load-scheduling decisions.
- Snowflake and Power BI — downstream analytical storage and reporting. Power BI Import mode must be refreshed before it reflects newly loaded Snowflake data.
- AWS S3 — private storage for manual SQL backup files. It is not a live replica of MySQL and is not shown as a mandatory ETL staging layer.
- GitHub Actions, Docker Hub and the EC2 runner — build, check, publish and deploy application container images.

4.2 Local Development Topology

```
Windows 10/11
  ├─ React/Vite       http://localhost:5173
  ├─ Spring Boot API  http://localhost:8082/api
  ├─ MySQL 8          localhost:3306
  └─ Python 3.12      data-processing virtual environment
```

The local backend uses DB_URL, DB_USERNAME and DB_PASSWORD environment variables. Its default URL is jdbc:mysql://localhost:3306/smart_building_db. The frontend default origin is http://localhost:5173. The backend schema mode is ddl-auto=validate: Hibernate checks mappings against the existing schema; it does not create or reset tables.

4.3 AWS Runtime Topology

The documented EC2 deployment runs three Compose services: frontend (host port 80 to container port 80), backend (container port 8082, not published to the public host), and MySQL (container port 3306, not published to the public host). The frontend is the public entry point. Keep database and backend ports private unless a deliberate, secured operational requirement exists.

The diagram brings the operational application, downstream analytics, cloud backup and deployment pipeline into one view. Use it as the reference map when tracing a request from the browser, following meter data into MySQL, or understanding how reporting and releases happen without bypassing the backend rules.

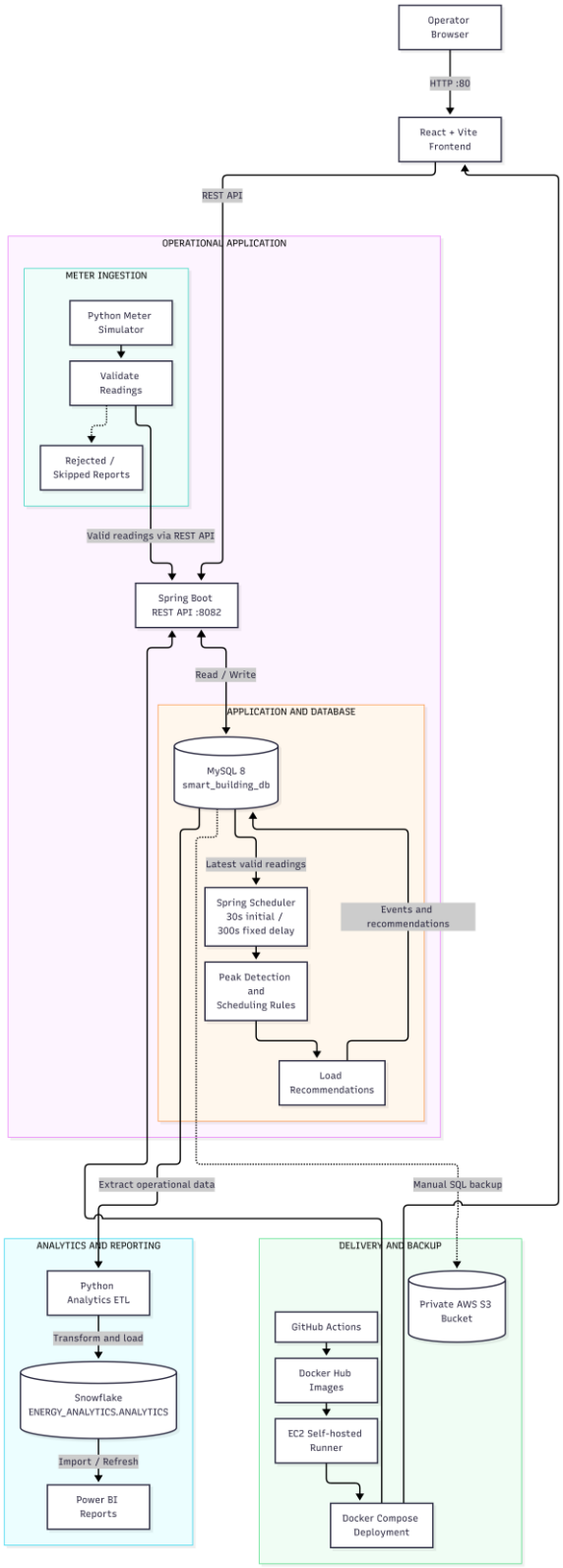


Figure 2. System architecture diagram showing the operational application, Python ingestion, MySQL, Snowflake/Power BI, S3 backup and CI/CD deployment paths.

Timing note: the Spring scheduler is configured with a 30-second initial delay and a 300,000-millisecond (five-minute) fixed delay. Fixed delay is measured after the previous execution completes. The analytics ETL cadence has been described in project notes as approximately 20 minutes; verify its scheduler/configuration in the deployed repository before treating that interval as a production guarantee.

5. End-to-End Runtime Workflow

1. An operator creates a building and registers equipment.
2. The operator configures occupancy windows and an active peak threshold.
3. The Python simulator retrieves registered buildings and existing meter readings from the backend.
4. The simulator generates readings after the latest stored timestamp for each building.
5. The validator checks timestamps and values, filters invalid or duplicate candidates, and records rejected/skipped rows.
6. Accepted readings are submitted through the Spring Boot meter-reading API and stored in MySQL.
7. The Spring scheduler starts after its configured initial delay and periodically checks for peak conditions.
8. When demand exceeds the active threshold, peak detection evaluates the event and asks the scheduling logic to assess equipment eligibility.
9. Critical and non-controllable equipment are protected. Occupancy and comfort constraints are evaluated before a recommendation is created.
10. Recommendations are persisted and displayed in the React operator interface.
11. An operator can approve, reject or override an eligible pending recommendation; the action is a workflow record, not a physical command.
12. The analytics process extracts operational data, transforms/reconciles it, loads Snowflake, and Power BI presents historical trends and KPIs.

5.1 Workflow Diagram

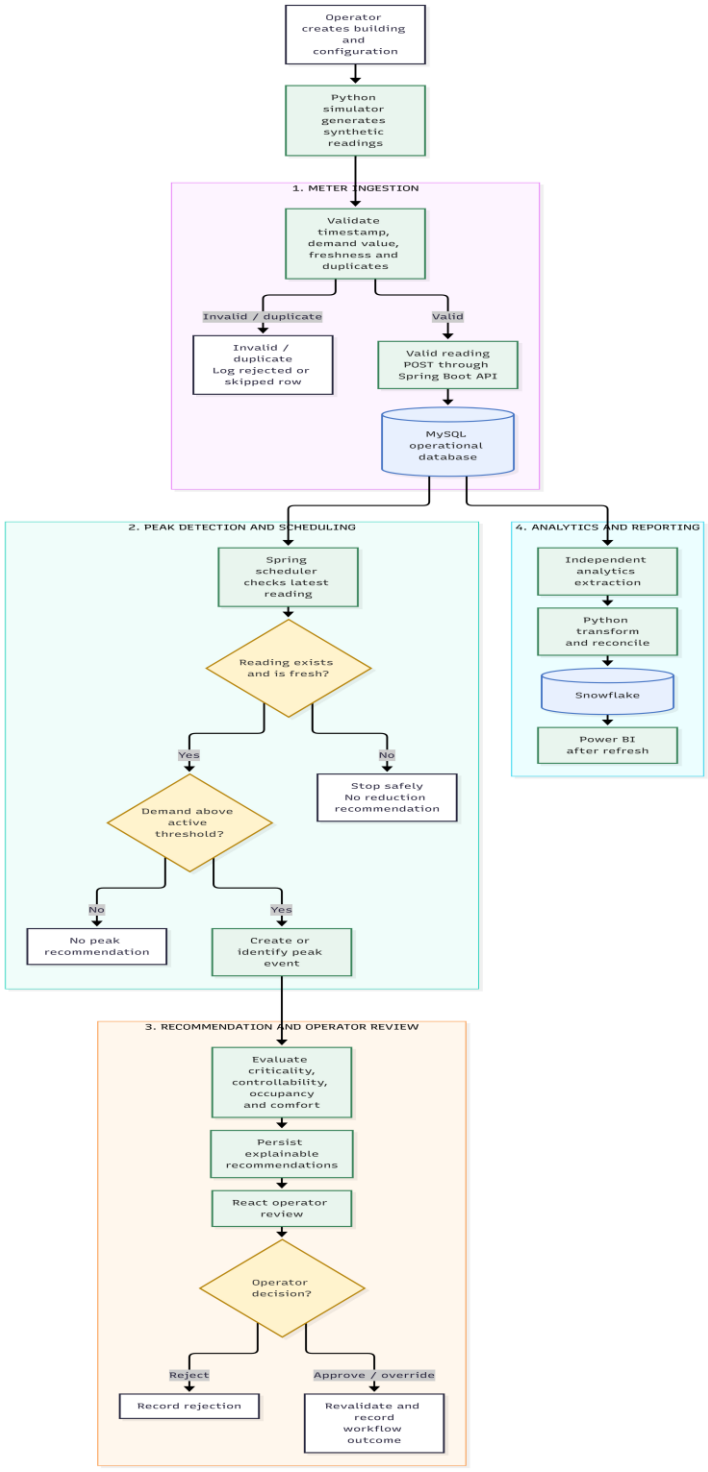
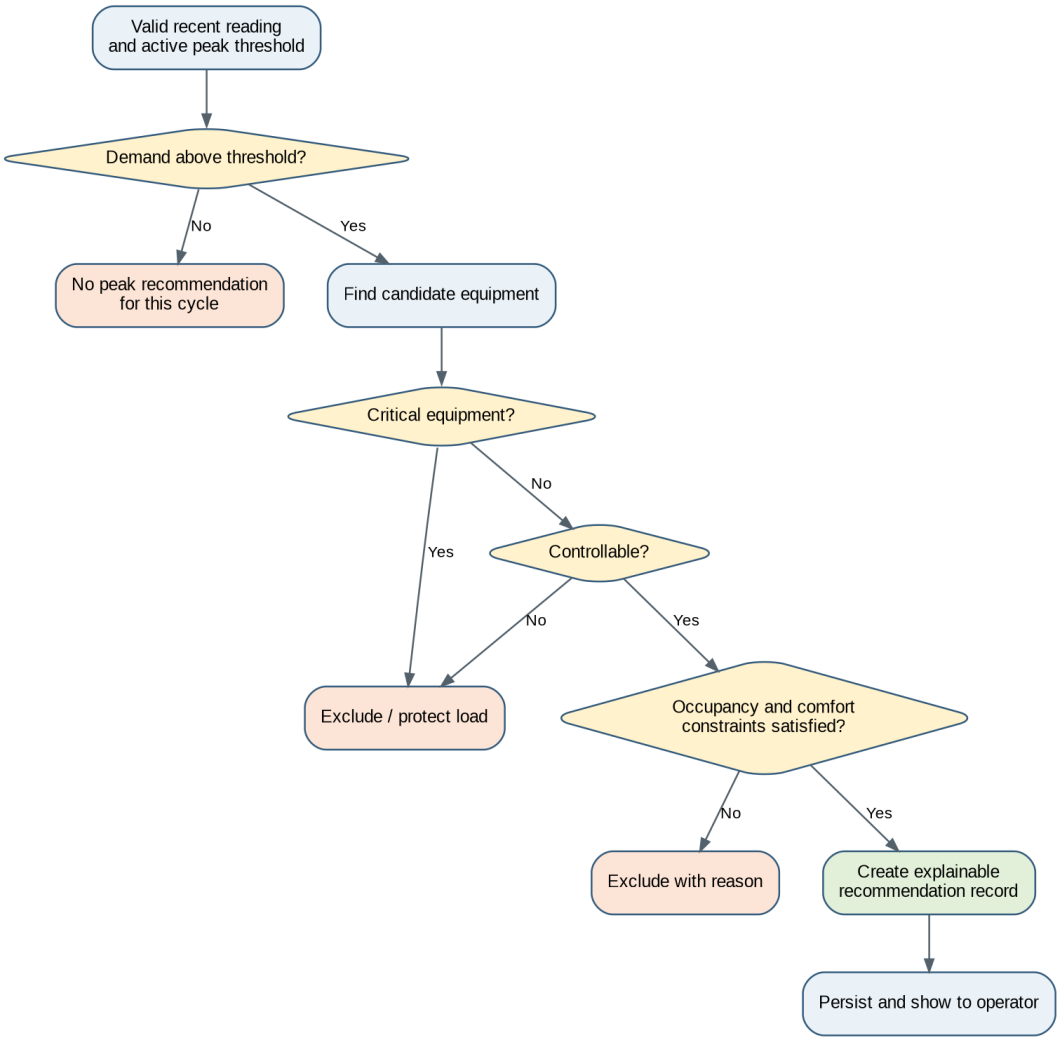


Figure 3. End-to-end workflow diagram showing operator setup, synthetic meter ingestion, validation, peak evaluation, safety checks and operator review.

6. Business Rules and Safety Controls

The decision path below makes the protection order explicit. A candidate must pass every gate before a recommendation is created; a failed gate excludes the candidate and should retain an understandable reason.



Process diagram – equipment eligibility and safety decision path. This is an advisory workflow; it does not issue equipment commands.

Rule	Expected system behaviour	Documentation note
Critical equipment	Never recommend a reduction for critical equipment.	Show protected status/reason in UI where available.
Controllability	Exclude equipment not marked controllable.	Rated power is not automatically equal to reducible demand.
Occupancy	Unknown occupancy is conservative and blocks reduction.	Do not infer an area is empty from missing data.
Comfort	Reject HVAC-related recommendations when configured constraints are not satisfied.	Document exact fields/rules from the deployed schema before claiming complete comfort enforcement.
Demand threshold	Compare latest valid reading to active building threshold.	A peak is threshold-based, not a forecast.
Recommendation state	Approval/rejection only for PENDING records.	Invalid state returns conflict response.

Override	Requires an active ADMIN/OPERATOR user ID in APP_USERS in the supplied project notes.	This check is not authentication or secure authorization.
Physical control	No command is sent to equipment.	All actions are simulated workflow records.

7. Scheduler and Peak Detection

The supplied application.properties configures the scheduler with energy.scheduler.enabled=true, energy.scheduler.initial-delay=30000 and energy.scheduler.fixed-delay=300000. These values mean the scheduled process begins after 30 seconds and subsequent executions are separated by a five-minute fixed delay after the preceding invocation completes. The scheduler delegates work to services; it should not be treated as the only place where business rules live.

```
energy.scheduler.enabled=true
energy.scheduler.initial-delay=30000
energy.scheduler.fixed-delay=300000
```

7.1 Scheduler Decision Sequence

13. Acquire the in-process run guard / avoid overlapping work in the current application instance.
14. Read the latest valid meter reading for the building.
15. Stop safely if no reading exists or the reading is stale.
16. Load the active peak configuration.
17. Compare demand with the threshold.
18. If demand is above threshold, find or create the relevant peak event.
19. Invoke the equipment eligibility and scheduling services.
20. Persist the event and recommendation result.
21. Log failures and release the run guard.

7.2 Scheduler Diagram

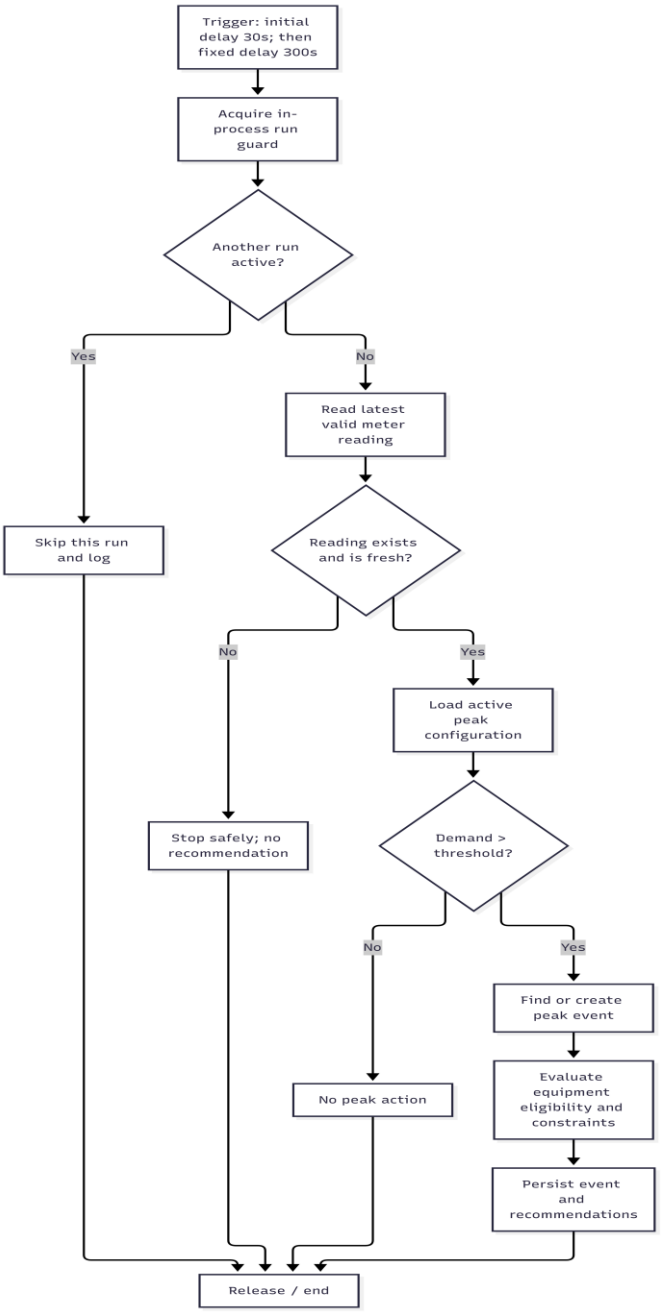


Figure 4. Scheduler decision diagram showing the run guard, reading freshness check, threshold comparison, event handling and recommendation flow.

8. MySQL Database Design

The operational database is smart_building_db. The supplied source includes entities for Building, LoadEquipment, MeterReading, OccupancySchedule, ComfortRule, PeakConfig, PeakEvent, LoadSchedule, AppUser, OverrideLog and PeakProcessingState. The deployed schema must match these entity mappings because ddl-auto is validate. Do not execute a schema reset or a sample migration against a populated database without reviewing its target IDs and current schema.

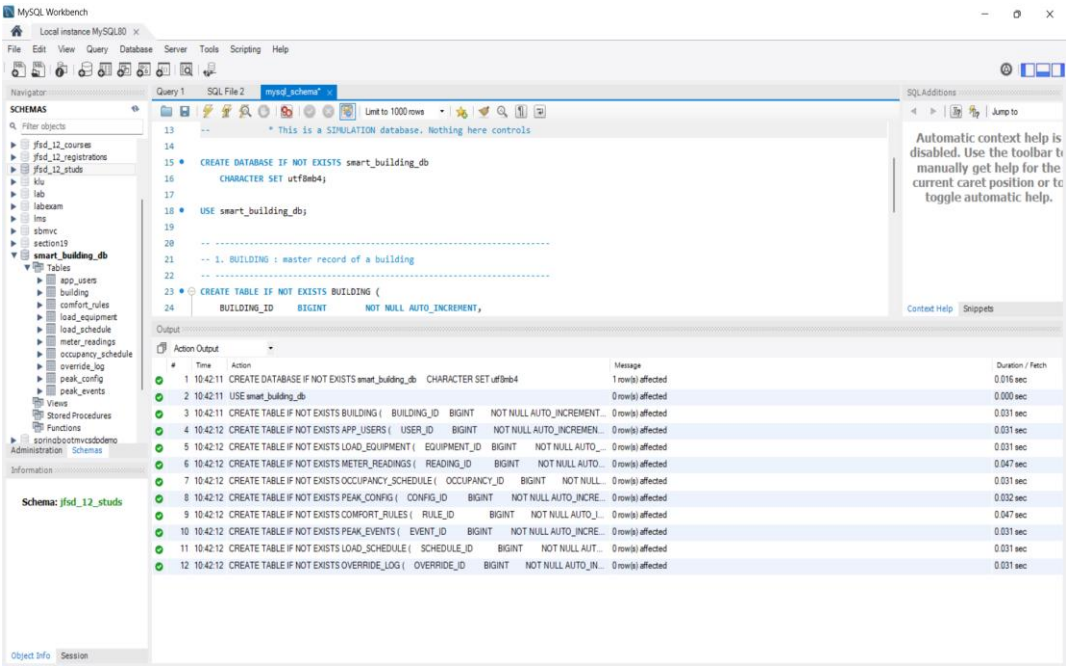


Figure 5. MySQL Workbench view of the project schema and SQL workspace used to inspect the operational database.

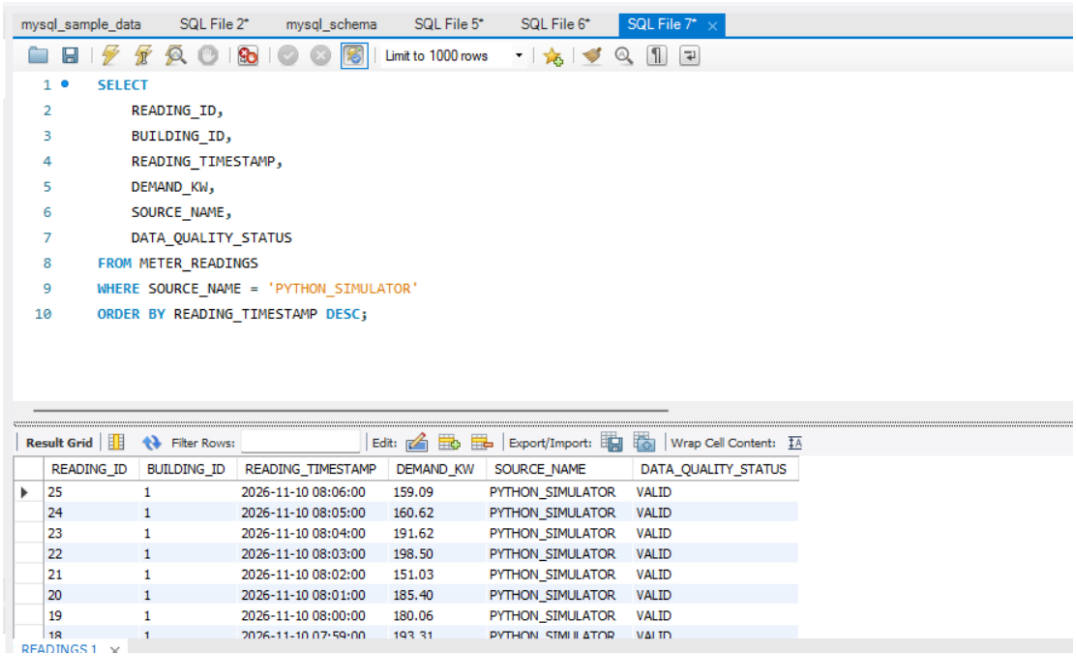


Figure 6. SQL query and result grid showing recent meter readings with source and data-quality fields.

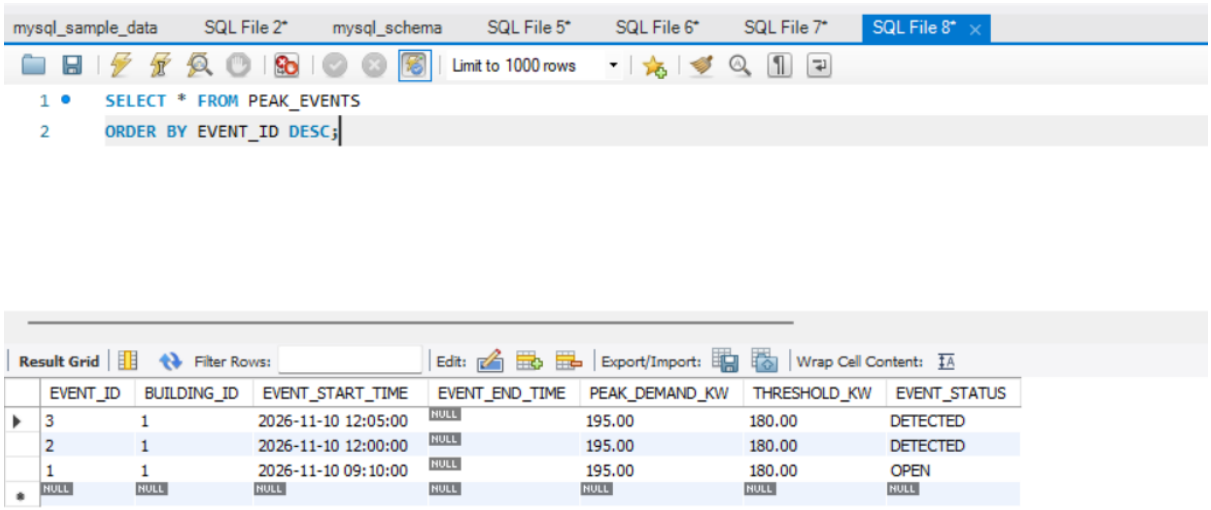


Figure 7. SQL query and result grid showing peak events ordered by event ID.

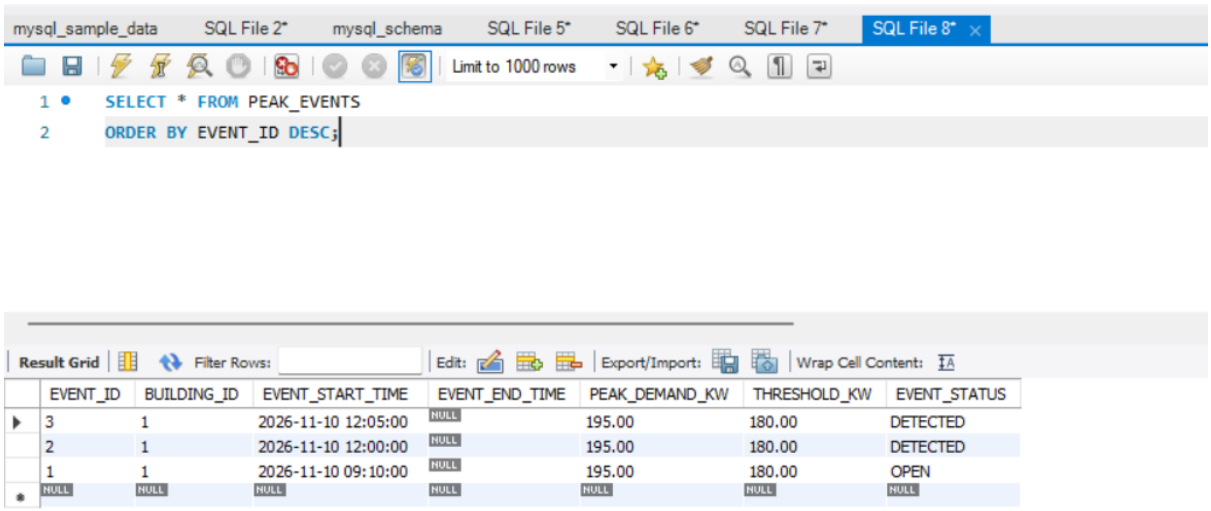


Figure 8. SQL query and result grid showing peak events ordered by event ID.

8.1 Entity Inventory

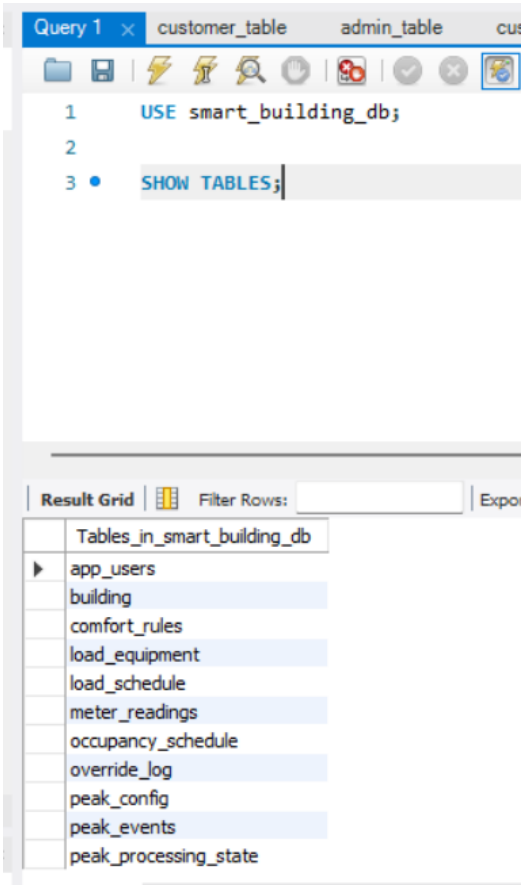


Figure 9. MySQL command output listing the tables in smart_building_db.

Entity / table concept	Purpose	Key relationship
BUILDING	Building master record.	Parent of equipment, meter readings, occupancy, peak config and peak events.
LOAD_EQUIPMENT	Equipment metadata and protection flags.	Belongs to a building; referenced by load schedules.
METER_READINGS	Timestamped demand observations.	Belongs to a building; unique on building + reading timestamp.
OCCUPANCY_SCHEDULE	Area occupancy windows.	Belongs to a building.
COMFORT_RULES	Comfort constraints by building/area.	Belongs to a building.
PEAK_CONFIG	Active demand threshold/configuration.	Belongs to a building.
PEAK_EVENTS	Detected peak event lifecycle.	Belongs to a building; parent of load schedules.
LOAD_SCHEDULE	Recommendation and approval state.	References peak event and equipment.
APP_USERS	Operator identity/status/role record.	Referenced by override records.
OVERRIDE_LOG	Override audit history.	References schedule and user.
PEAK_PROCESSING_STATE	Processing state used by peak workflow.	Used to track detection/processing state; verify exact constraints from live DDL.

8.2 Relationship Diagram

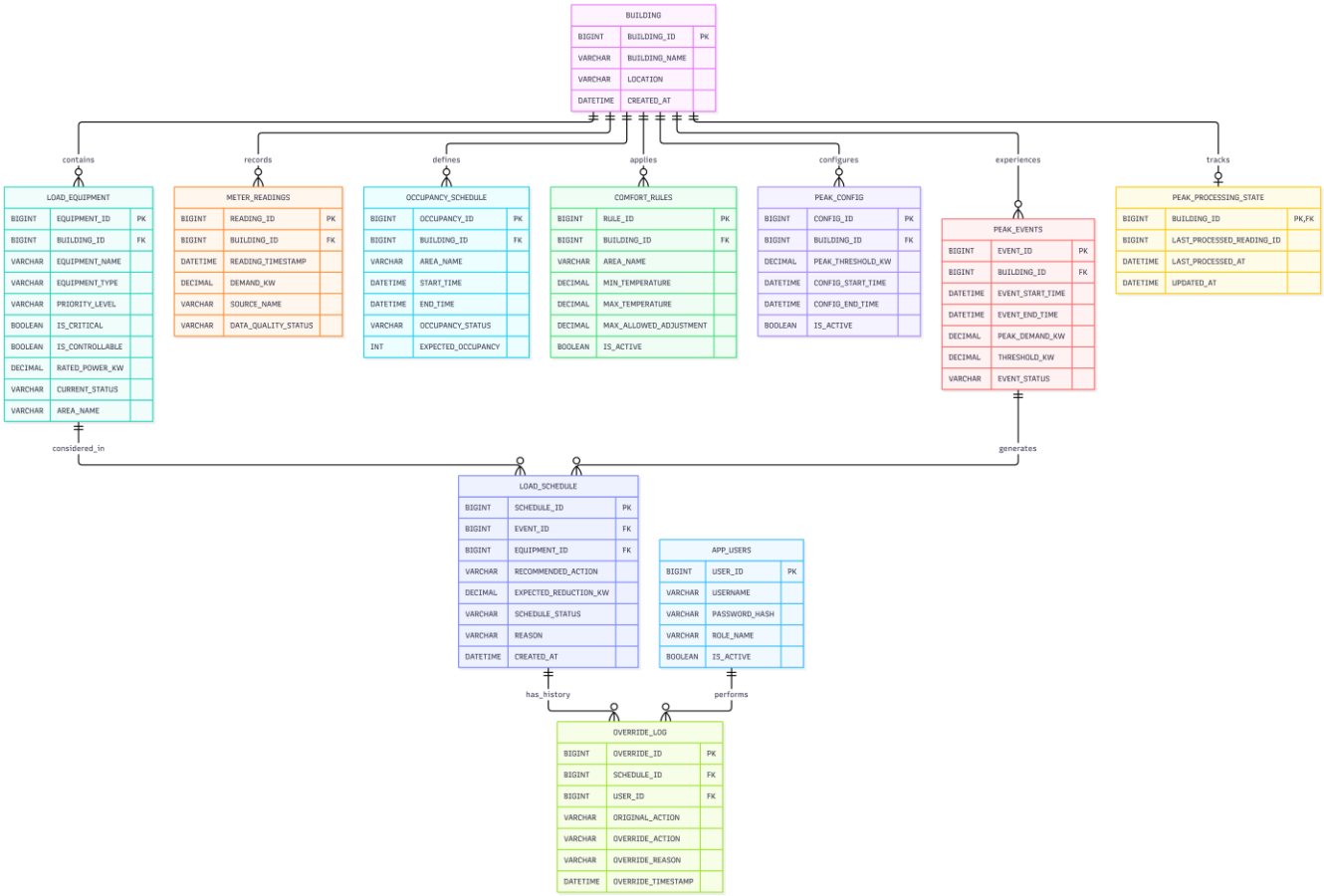


Figure 10. Entity-relationship diagram showing the building table and its related equipment, readings, occupancy, configuration, event and schedule tables.

8.3 Database Safety and Migration

The application is configured with Hibernate schema validation (ddl-auto=validate). This means the database must already contain the tables and columns expected by the entity mappings before the backend starts. The application does not automatically create or repair a missing schema.

- Use a separate development database when testing entity or schema changes.
- Compare the entity mappings with the current MySQL DDL before changing column names, data types, nullability or foreign keys.
- Take a fresh mysqldump backup before applying a migration to a database that contains project data.
- Apply reviewed migration scripts in a controlled order, then verify table definitions, row counts and backend startup.
- Keep the unique key on (building_id, reading_timestamp) so repeated ingestion does not create duplicate meter readings.

8.4 SQL Commands for Inspection and Verification

Run these read-only statements in MySQL Workbench or a MySQL client after connecting to the project database. They are intended for schema inspection and operational verification. The examples use the table and column names shown in the project database screenshots; confirm the live schema before adapting them.

Select the operational database

```
USE smart_building_db;
```

List the available tables

```
SHOW TABLES;
```

Inspect a table definition

```
DESCRIBE meter_readings;
```

Review recent meter readings submitted by the simulator

```
SELECT reading_id, building_id, reading_timestamp, demand_kw, source_name, data_quality_status
FROM meter_readings
WHERE source_name = 'PYTHON_SIMULATOR'
ORDER BY reading_timestamp DESC
LIMIT 20;
```

Review recent peak events

```
SELECT event_id, building_id, event_start_time, event_end_time,
       peak_demand_kw, threshold_kw, event_status
FROM peak_events
ORDER BY event_id DESC
LIMIT 20;
```

Count readings by quality status

```
SELECT data_quality_status, COUNT(*) AS reading_count
FROM meter_readings
GROUP BY data_quality_status;
```

Count peak events by status

```
SELECT event_status, COUNT(*) AS event_count
FROM peak_events
GROUP BY event_status;
```

Check duplicate building/timestamp pairs

```
SELECT building_id, reading_timestamp, COUNT(*) AS duplicate_count
FROM meter_readings
GROUP BY building_id, reading_timestamp
HAVING COUNT(*) > 1;
```

Safety: SELECT, SHOW and DESCRIBE are used here for inspection. Do not run DROP, TRUNCATE, DELETE, ALTER or schema-reset statements on the deployed database as part of routine verification. Back up the database and review the migration file and target IDs before any structural or destructive change.

- Take a verified backup before schema changes.
- Review database/PEAK_LIFECYCLE_MIGRATION.sql and confirm referenced IDs before executing it.
- Use a migration tool and versioned migration scripts for future schema changes.
- Keep unique key (building_id, reading_timestamp) to support idempotent ingestion.
- Do not expose MySQL port 3306 publicly on EC2.
- Do not store credentials in GitHub or screenshots.

9. Spring Boot Backend

The backend package structure is layered: controllers expose HTTP endpoints; services coordinate business rules; repositories provide database access; entities map to MySQL tables; configuration controls CORS and application properties; the scheduler initiates periodic peak checks; and the global exception handler converts expected errors into HTTP responses.

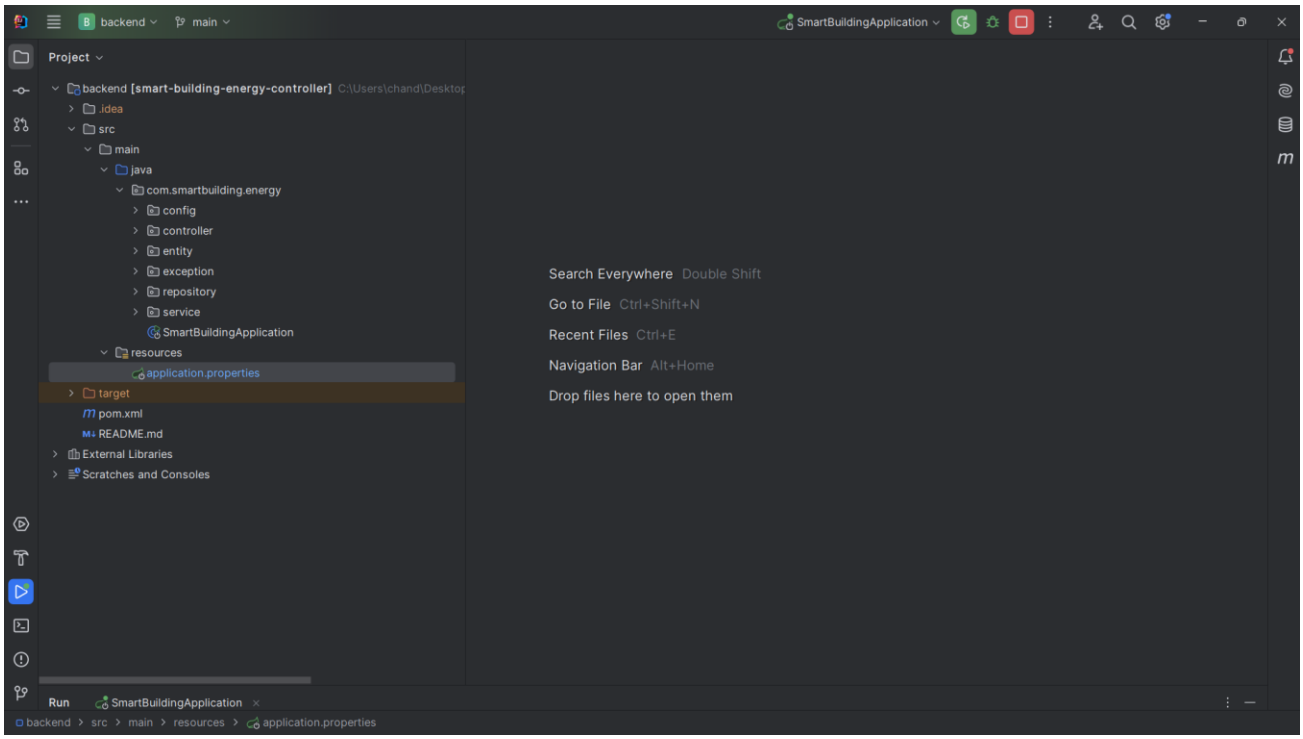


Figure 11. IDE project explorer showing the Spring Boot backend package structure and application entry point.

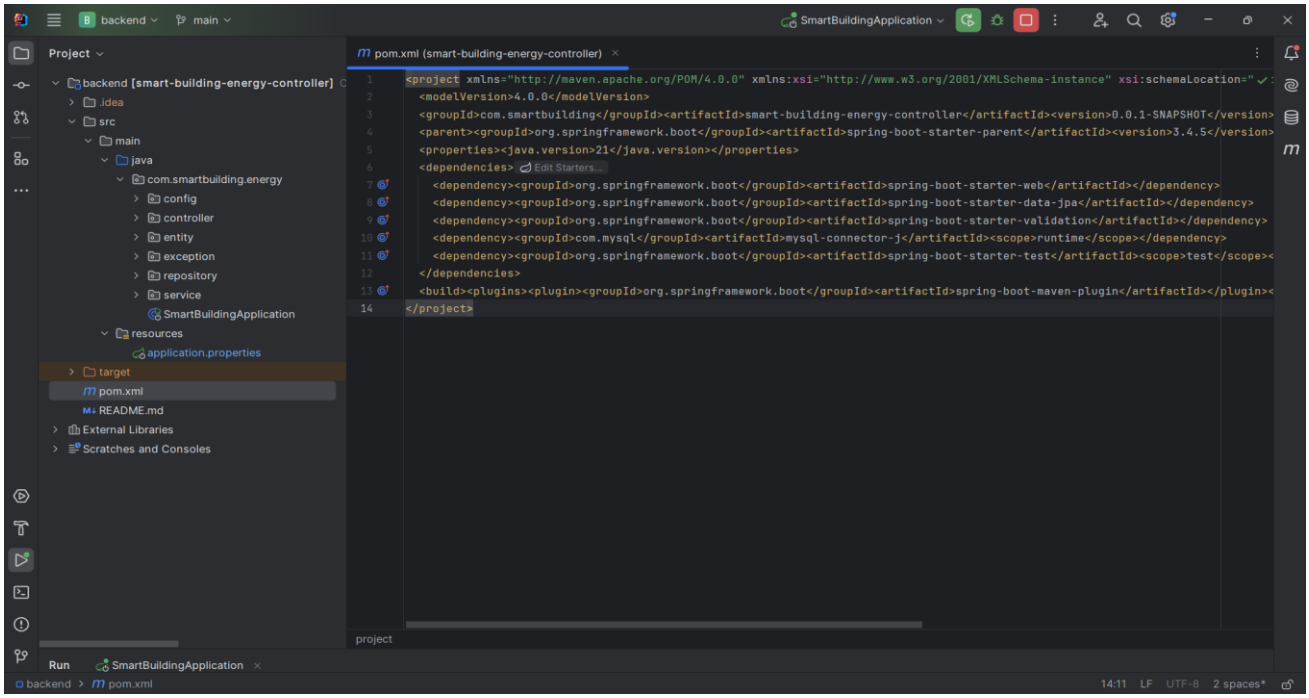


Figure 12. Maven pom.xml showing the backend project dependencies and build configuration.

application.properties

```
spring.application.name=smart-building-energy-controller
server.port=${SERVER_PORT:8082}
spring.datasource.url=${DB_URL:jdbc:mysql://localhost:3306/smart_building_db?useSSL=false&allowPublicKeyRetrieval=true&serverTimezo
spring.datasource.username=${DB_USERNAME:root}
spring.datasource.password=${DB_PASSWORD}
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
spring.jpa.open-in-view=false
app.cors.allowed-origin=${FRONTEND_ORIGIN:http://localhost:5173}


# Peak Detection Scheduler Configuration
energy.scheduler.enabled=true

# Peak Detection Scheduler
energy.scheduler.fixed-delay=600000
energy.scheduler.initial-delay=30000
# Python integration
energy.python.executable=.venv/Scripts/python.exe
energy.python.script=main.py
energy.python.working-directory=../data-processing
spring.jpa.properties.hibernate.jdbc.time_zone=UTC
# Maximum time allowed for Python ingestion
energy.python.timeout-minutes=3
# Snowflake ETL Scheduler
energy.etl.scheduler.enabled=true
energy.etl.scheduler.fixed-delay=1200000
energy.etl.scheduler.initial-delay=120000
energy.etl.script=run_etl.py
energy.etl.timeout-minutes=15
```

Figure 13. application.properties showing the backend port, MySQL connection settings, CORS origin and scheduler/ETL configuration.

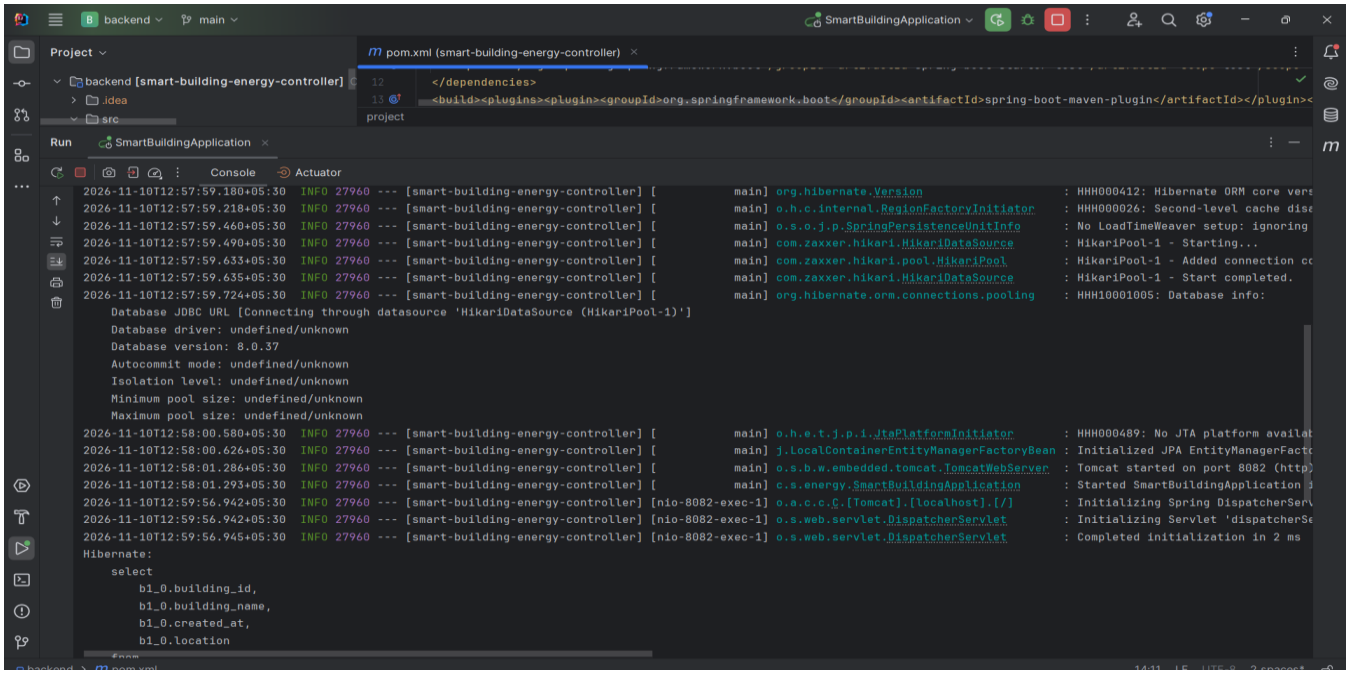


Figure 14. Backend Maven build configuration and plugin declarations in pom.xml.

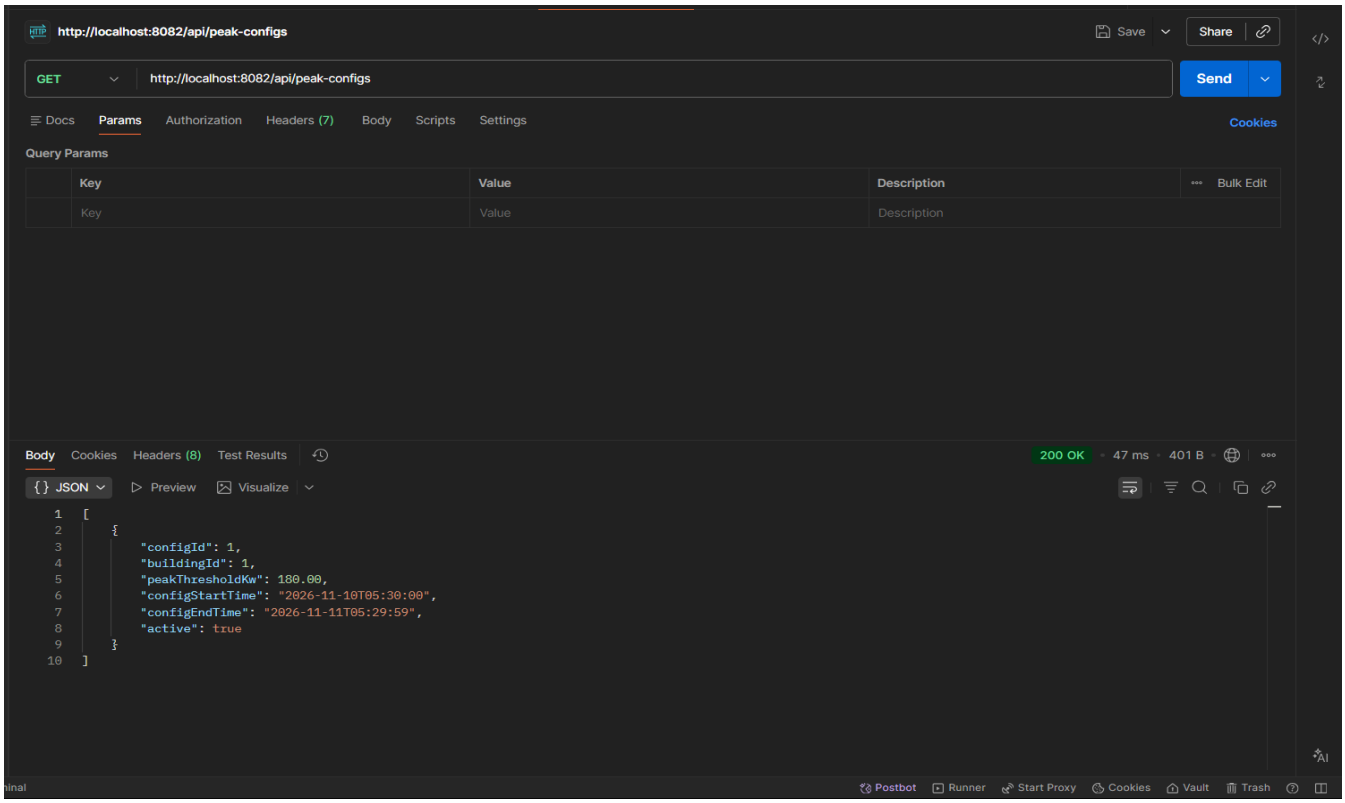


Figure 15. Spring Boot application startup logs showing the backend starting and initializing its services.

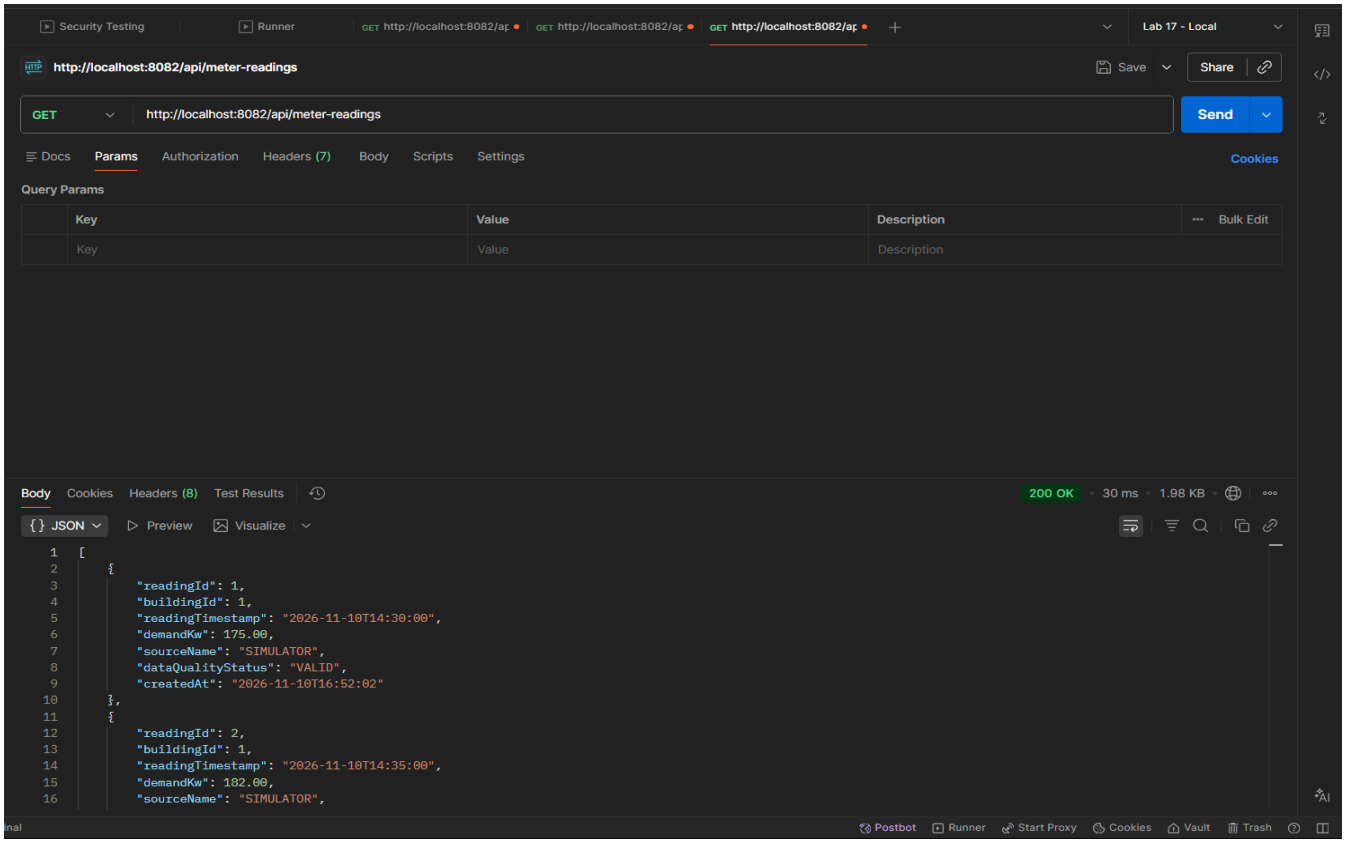


Figure 16. Postman response for the peak-configuration API, showing a successful backend request.

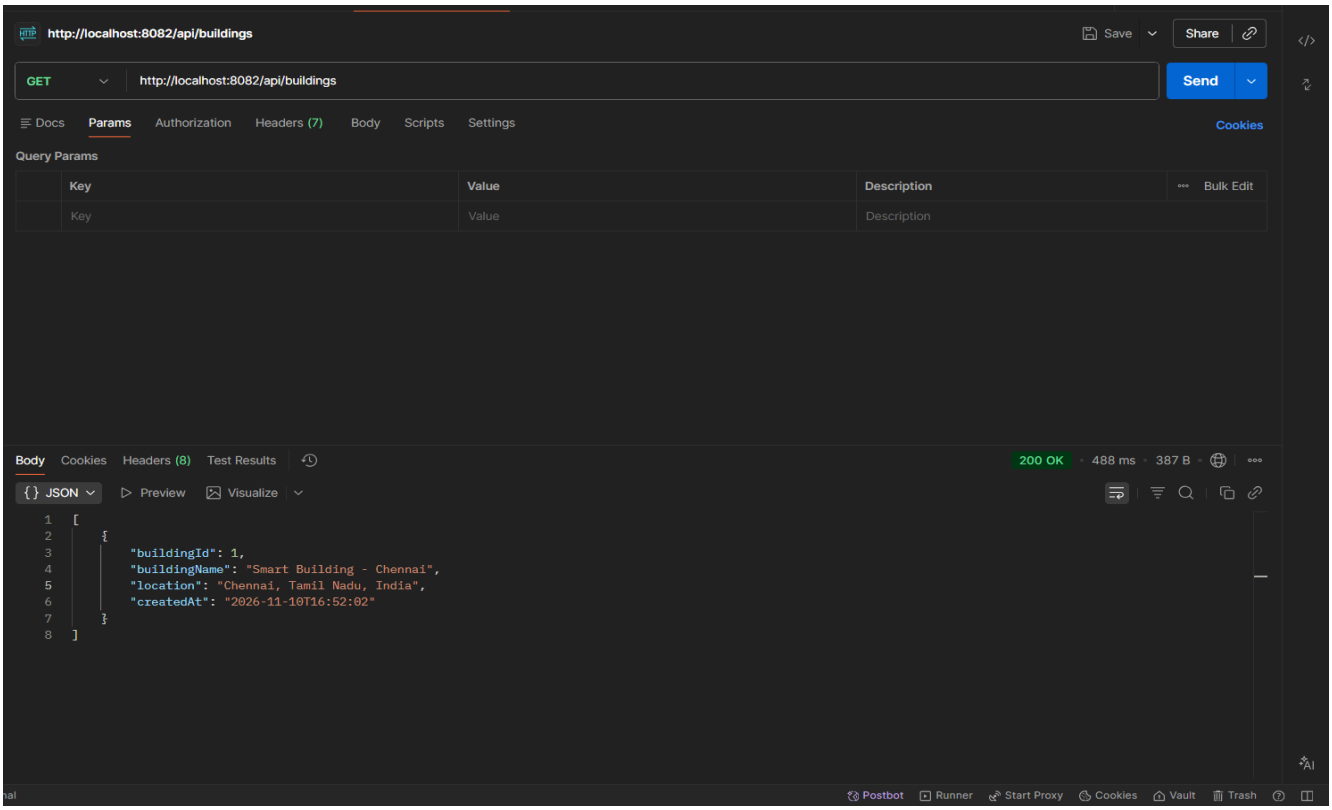


Figure 17. Postman response for the meter-reading API, showing accepted reading data returned by the backend.

Layer	Representative classes	Responsibility
Controller	BuildingController, LoadEquipmentController, MeterReadingController, OccupancyScheduleController, PeakConfigController, PeakDetectionController, LoadScheduleController	HTTP routing, request binding and response status.
Service	BuildingService, LoadEquipmentService, MeterReadingService, OccupancyScheduleService, PeakConfigService, PeakDetectionService, LoadScheduleService, EquipmentEligibilityService, PythonIngestionService	Business rules, validation orchestration and transactional operations.
Repository	JPA repositories for each entity	Database queries and persistence.
Entity	Building, LoadEquipment, MeterReading, PeakEvent, LoadSchedule, AppUser, OverrideLog, etc.	ORM mappings.
Scheduler	PeakDetectionScheduler	Timed invocation of peak detection.
Exception	GlobalExceptionHandler, ResourceNotFoundException	Consistent error handling.

9.1 Configuration

9.1.1 Configure and start the backend
PowerShell — from the project root:

```
cd backend
$env:SERVER_PORT="8082"
$env:DB_URL="jdbc:mysql://localhost:3306/smart_building_db"
$env:DB_USERNAME="root"
$env:DB_PASSWORD="<your-local-password>"
$env:FRONTEND_ORIGIN="http://localhost:5173"
mvn clean package
mvn spring-boot:run
```

Verify:
• Open http://localhost:8082/swagger-ui/index.html (if Swagger UI is enabled).
• Confirm the console shows successful datasource connection and application startup.
• Use Postman/Swagger to call GET /api/buildings and confirm the API responds.
• If startup fails, check DB_URL, credentials, MySQL availability and schema validation.

```
server.port=${SERVER_PORT:8082}
spring.datasource.url=${DB_URL:jdbc:mysql://localhost:3306/smart_building_db?useSSL=false&allowPublicKeyRetriev
al=true&serverTimezone=UTC}
spring.datasource.username=${DB_USERNAME:root}
spring.datasource.password=${DB_PASSWORD:}
spring.jpa.hibernate.ddl-auto=validate
app.cors.allowed-origin=${FRONTEND_ORIGIN:http://localhost:5173}
```

Never commit a real password. The password property should be injected from a protected environment variable or secret manager. Remove any hard-coded password fallback from application configuration.

9.2 API Endpoint Inventory

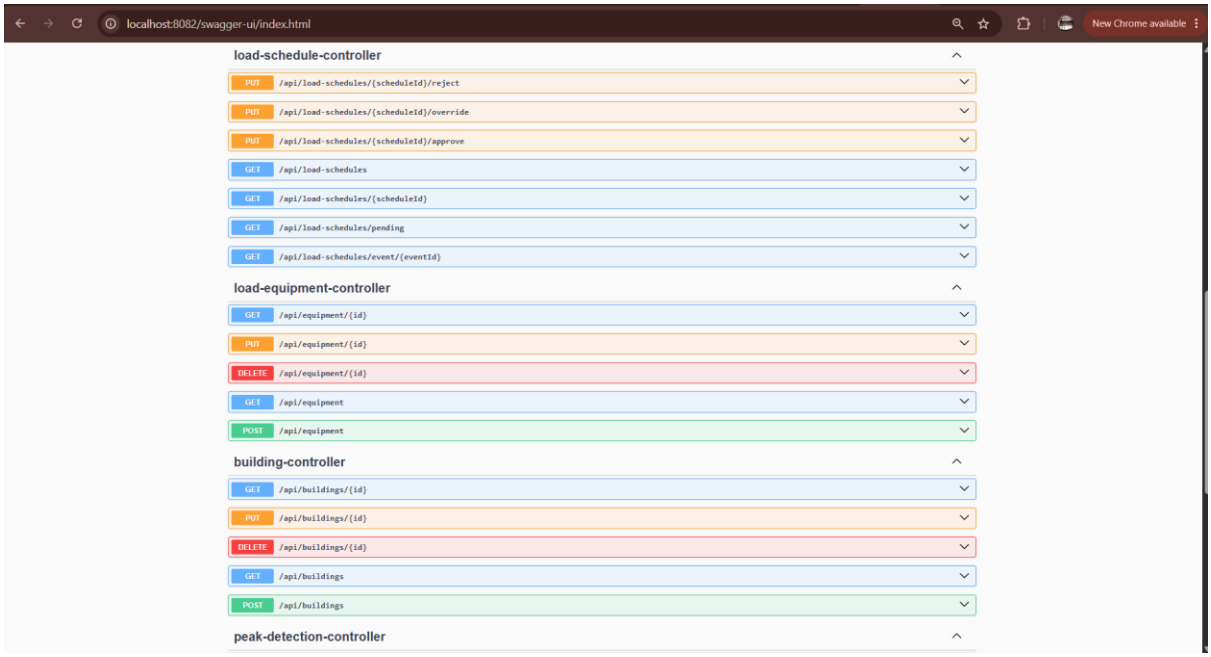


Figure 18. Swagger UI listing the backend REST API controller groups and available endpoints.

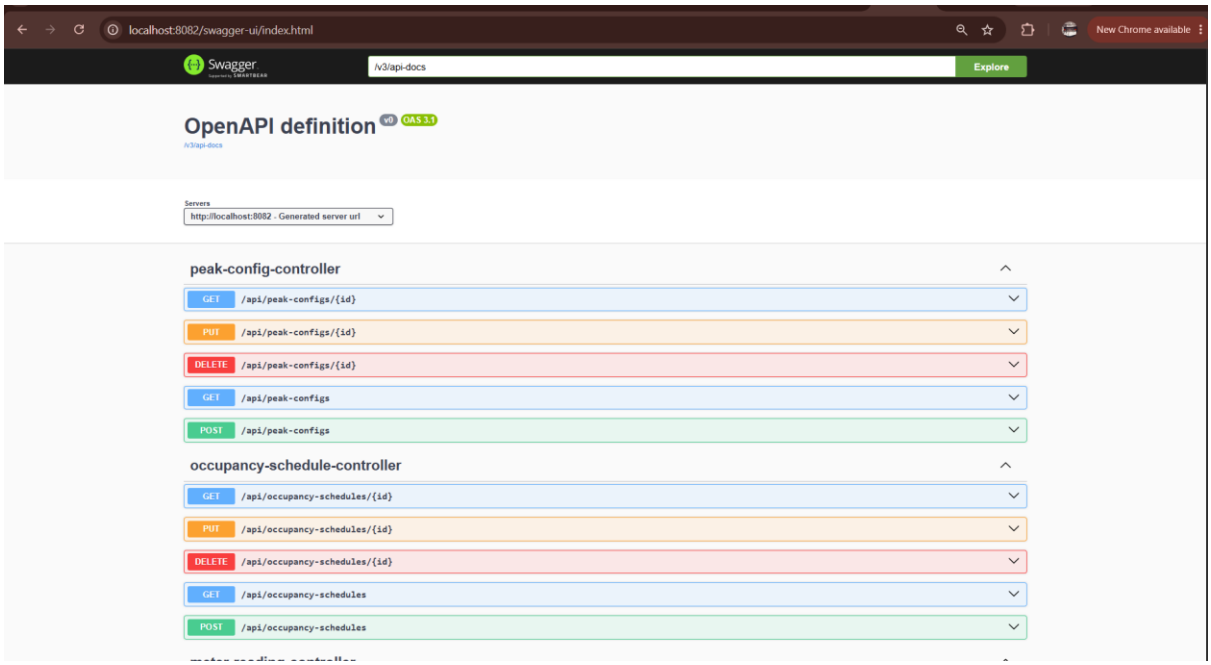


Figure 19. Swagger UI showing the API endpoint groups for peak configuration and occupancy schedule operations.

Resource	HTTP method and route	Purpose
Buildings	GET /api/buildings	List buildings.
Buildings	GET /api/buildings/{id}	Retrieve one building.
Buildings	POST /api/buildings	Create a building.
Buildings	PUT /api/buildings/{id}	Update a building.
Buildings	DELETE /api/buildings/{id}	Delete a building.
Equipment	GET/POST /api/equipment	List or create equipment.
Equipment	GET/PUT/DELETE /api/equipment/{id}	Retrieve, update or delete equipment.
Meter readings	GET/POST /api/meter-readings	List or ingest readings.
Meter readings	GET/PUT/DELETE /api/meter-readings/{id}	Retrieve, update or delete reading.
Occupancy	GET/POST /api/occupancy-schedules	List or create occupancy windows.
Occupancy	GET/PUT/DELETE /api/occupancy-schedules/{id}	Retrieve, update or delete occupancy record.
Peak config	GET/POST /api/peak-configs	List or create peak configuration.
Peak config	GET/PUT/DELETE /api/peak-configs/{id}	Retrieve, update or delete configuration.
Peak events	GET /api/peak-events	List peak events.
Peak detection	POST /api/peak-detection/{buildingId}	Trigger peak detection for a building.
Load schedules	GET /api/load-schedules	List schedules.

Load schedules	GET /api/load-schedules/event/{eventId}	List schedules for an event.
Load schedules	GET /api/load-schedules/pending	List pending schedules.
Load schedules	GET /api/load-schedules/{scheduleId}	Retrieve schedule details.
Load schedules	PUT /api/load-schedules/{scheduleId}/approve	Approve a pending recommendation.
Load schedules	PUT /api/load-schedules/{scheduleId}/reject	Reject a pending recommendation.
Load schedules	PUT /api/load-schedules/{scheduleId}/override	Record an override.

This inventory is derived from the supplied controller annotations. Confirm exact request DTOs, query parameters and response examples from the current Swagger/OpenAPI output before treating this table as a complete API contract.

9.3 HTTP Error Behaviour

The API should return a response that distinguishes a successful request from a validation, lookup or state-transition failure. The exact status and response body must be confirmed against the current GlobalExceptionHandler and controller implementation; the following is a verification guide, not a substitute for the live API contract.

- 200 OK / 201 Created — successful retrieval or creation, according to the endpoint.
- 400 Bad Request — malformed input or failed request validation.
- 404 Not Found — requested building, equipment, reading, event or schedule does not exist.
- 409 Conflict — invalid recommendation state transition or a conflicting operation, where implemented.
- 5xx response — unexpected server-side failure; inspect backend logs and preserve the request context without exposing secrets.

When testing an error path, record the request, expected result, actual HTTP status and response body. Do not assume every controller uses identical error handling until it has been checked in Swagger/Postman.

Backend build check — from the backend directory:
mvn clean test
mvn package

Run tests before packaging. Do not treat a successful package as proof that every API workflow has been functionally tested.

HTTP status	Meaning in this application
201	Resource created.
200	Successful read/update/action.
204	Successful delete with no response body.
400	Validation or malformed request.
404	Requested resource not found.
409	Invalid workflow state or database conflict.
500	Unexpected server error; inspect backend logs.

10. Python Meter Simulator and Ingestion

The supplied data-processing/main.py orchestrates the simulator → validator → Spring Boot API path. It first retrieves buildings and existing meter readings. For each building with a valid buildingId it generates new readings, validates them, saves a rejected-reading report, and submits accepted readings one at a time through MeterApiClient. A submission failure is collected and causes the process to exit with an error after processing attempts.



Figure 20. Command Prompt showing creation and activation of the Python virtual environment for the data-processing module.

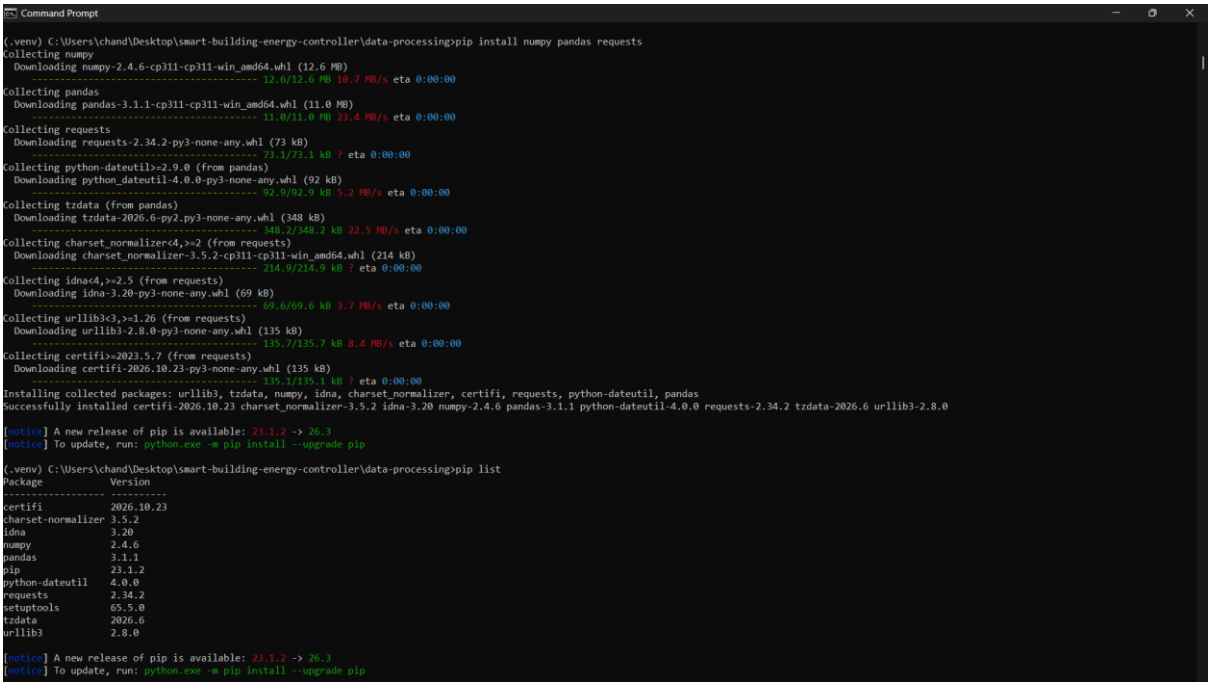


Figure 21. Terminal output showing installation of the Python dependencies, including NumPy, Pandas and Requests.

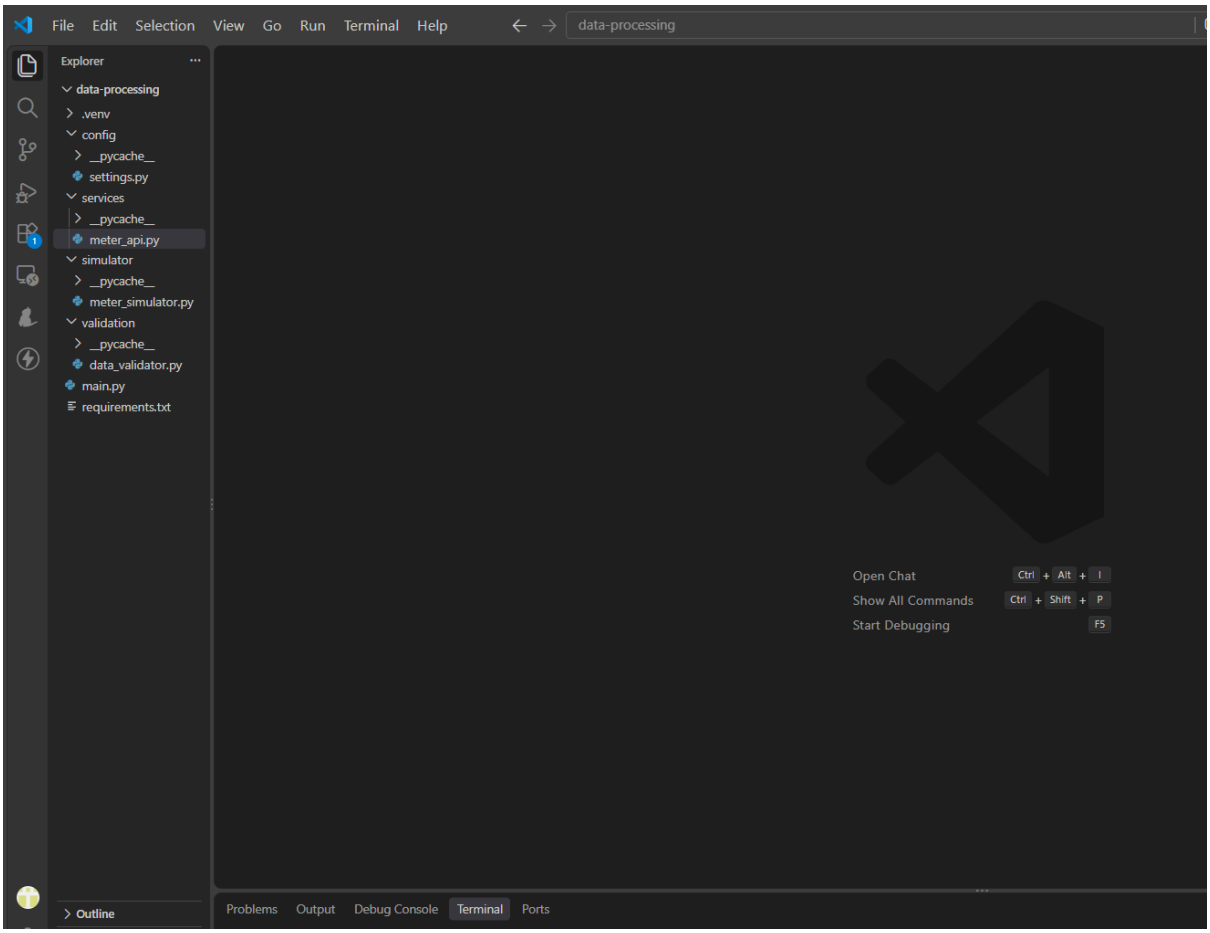


Figure 22. VS Code project explorer showing the Python simulator, validation and service modules.

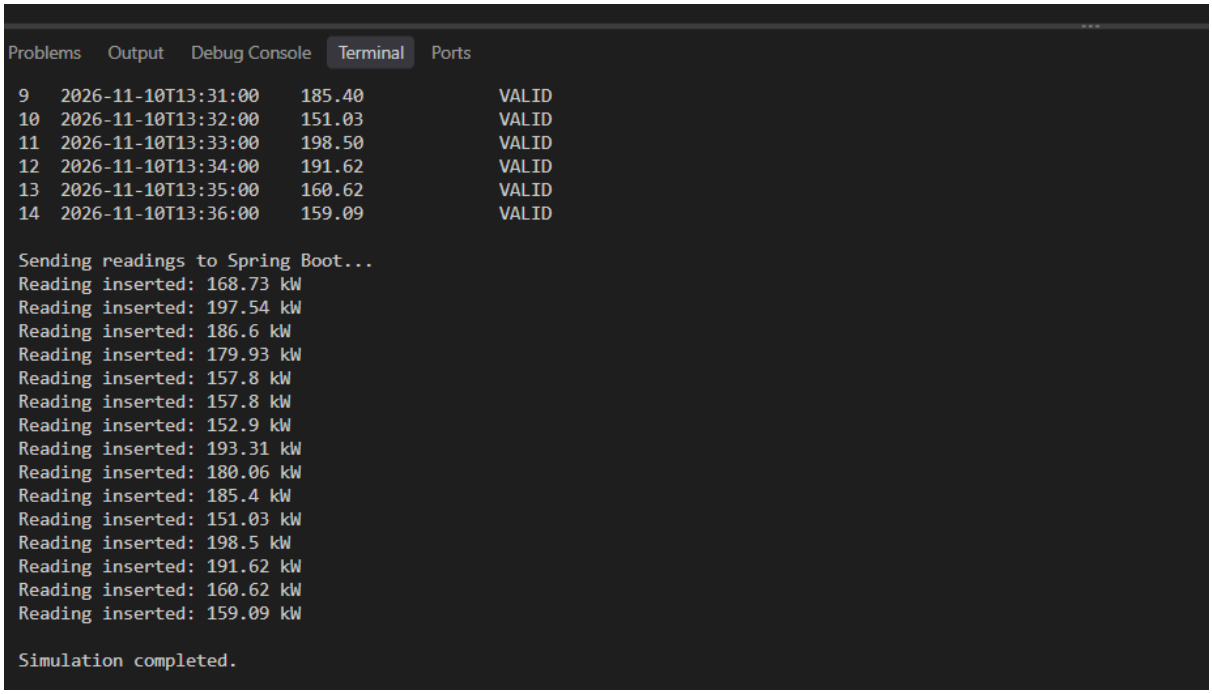


Figure 23. Python terminal output showing generated meter readings with timestamps, demand values and VALID quality status.

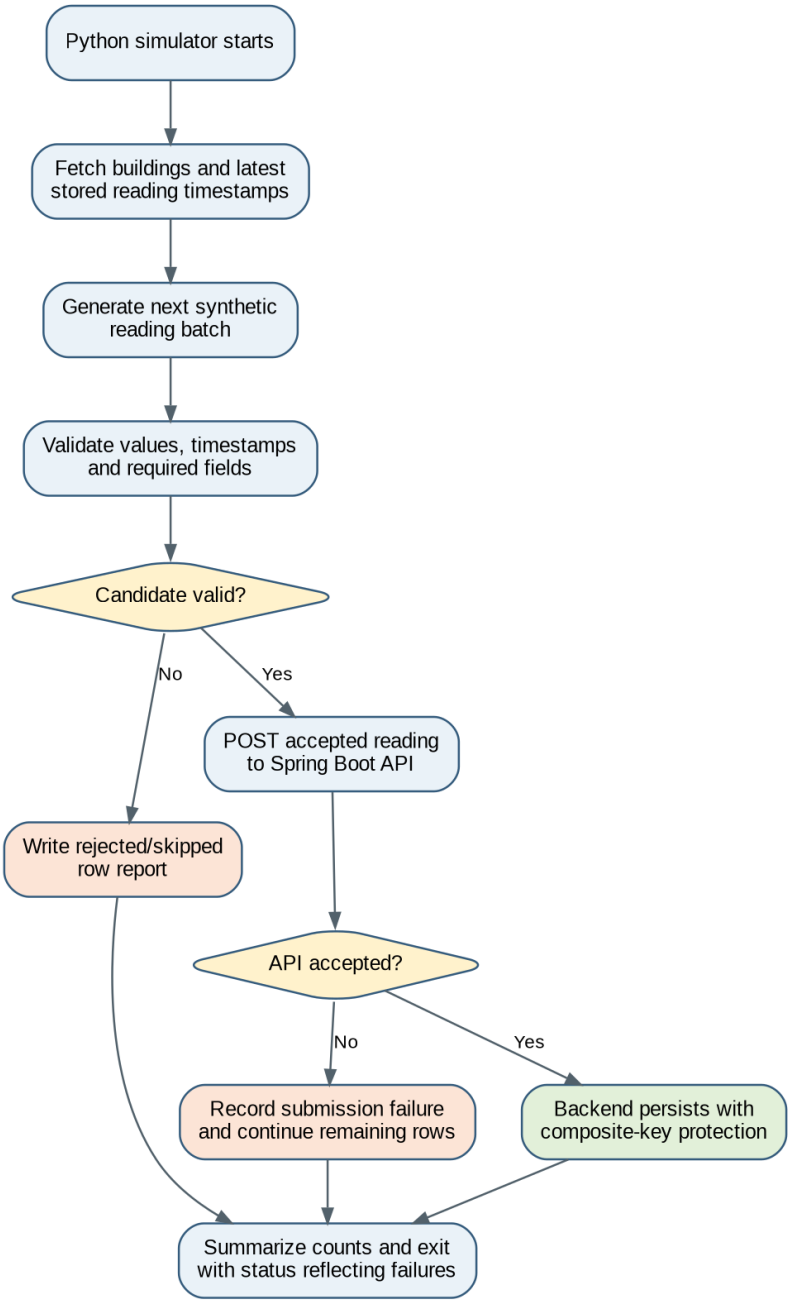
10.1 Processing Steps

- Read the backend base URL and request the list of registered buildings.
- For each valid building identifier, retrieve the latest stored reading timestamp.
- Generate the next synthetic readings using timestamps later than the stored watermark.
- Validate required fields, numeric demand values, timestamp format and chronological freshness.
- Separate accepted candidates from rejected or skipped rows and write the rejection report for review.
- Submit accepted readings through MeterApiClient to the Spring Boot API; the backend remains responsible for persistence and duplicate protection.
- Collect per-reading submission failures and continue processing the remaining candidates where the client flow permits.
- At the end, report generated, accepted, rejected/skipped and submitted/failed counts. Return a failure exit status when submission errors occurred.

Operational note: a rejected-row report is useful evidence, but it does not automatically repair the source data. Review the reason, correct the generator or input, and rerun only after checking whether the corresponding timestamp has already been accepted.

Step	What happens	Evidence/output
1. Discover	GET registered buildings and existing readings from backend.	API response data.
2. Generate	Create synthetic timestamps/demand values after latest stored timestamp per building.	Generated candidate readings.
3. Validate	Check required fields, timestamp/value rules and duplicates against existing data.	Valid list and rejected/skipped list.
4. Report	Write rejected/skipped rows to a report file.	Report under data-processing/reports.
5. Submit	POST each accepted reading through backend API.	Persisted MySQL meter records.
6. Summarize	Print generated, valid, rejected/skipped, submitted and failed counts.	Console execution log.

The diagram summarizes the simulator's validation and API-submission path, including rejected rows and failed submissions.



Process diagram – Python ingestion, validation, API submission and failure reporting.

10.2 Idempotency

The backend protects meter ingestion using the composite identity (building_id, reading_timestamp). Re-sending an already stored reading returns the stored record rather than causing the complete batch to fail with MySQL duplicate-key error 1062. The simulator also calculates future timestamps after the latest stored reading to reduce overlapping rolling batches.

10.3 Local Python Setup

10.3.1 Create the environment and install packages

PowerShell — from the project root:

```
cd data-processing
py -3.12 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
```

10.3.2 Configure and run the simulator
Set the backend API base URL using the configuration key read by the current simulator code. Do not invent a new key; check the config/service module and use the exact name already implemented.
python main.py

10.3.3 Check the output

• Confirm the building list was retrieved.
• Check generated, valid, rejected/skipped and submitted/failed counts.
• Open rejected_readings.csv and review the rejection reason before rerunning.
• Confirm accepted readings appear through the backend API/MySQL.
• Do not rerun the same timestamp batch without checking the backend's idempotency behaviour.

If PowerShell activation is blocked, use Command Prompt:

.venv\Scripts\activate.bat

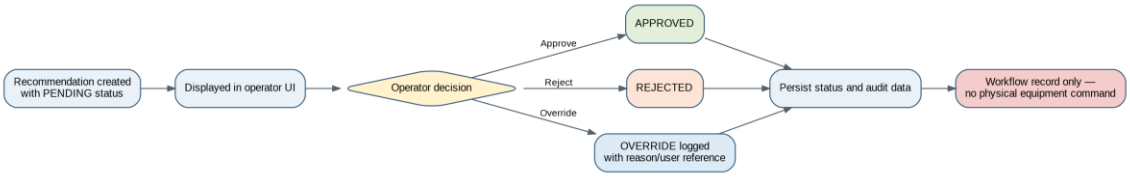
```
cd data-processing
py -3.12 -m venv .venv

Windows (PowerShell)   .\.venv\Scripts\Activate.ps1
Windows (Command Prompt)   .venv\Scripts\activate.bat

python -m pip install -r requirements.txt
python main.py
```

If PowerShell blocks script activation, use the approved local execution-policy approach or activate with the batch script from Command Prompt. Do not weaken machine-wide policy without approval.

The recommendation lifecycle shown below clarifies the operator's role. Approval, rejection and override update application records only; they do not control building equipment.



Process diagram – recommendation review and operator decision lifecycle.

11. React Frontend

The React/Vite frontend is the operator-facing layer. It calls the backend REST API and presents building information, meter readings, peak events, equipment and schedule approvals. It should not implement a second independent copy of the safety rules; the backend remains the source of truth for validation and recommendation state.

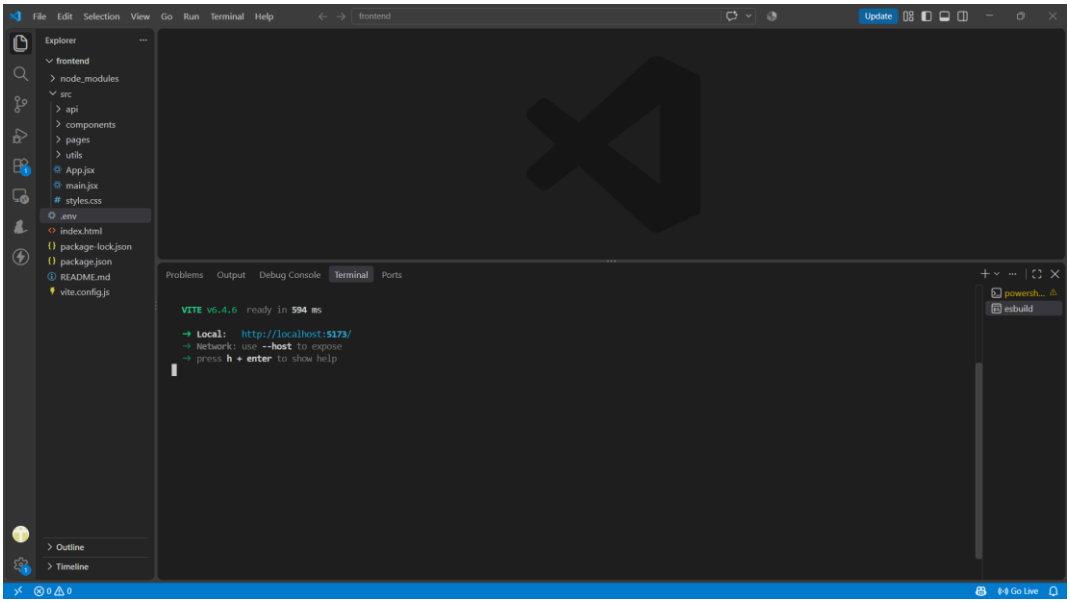


Figure 24. VS Code frontend project explorer showing the React/Vite source folders, components, pages and API utilities.

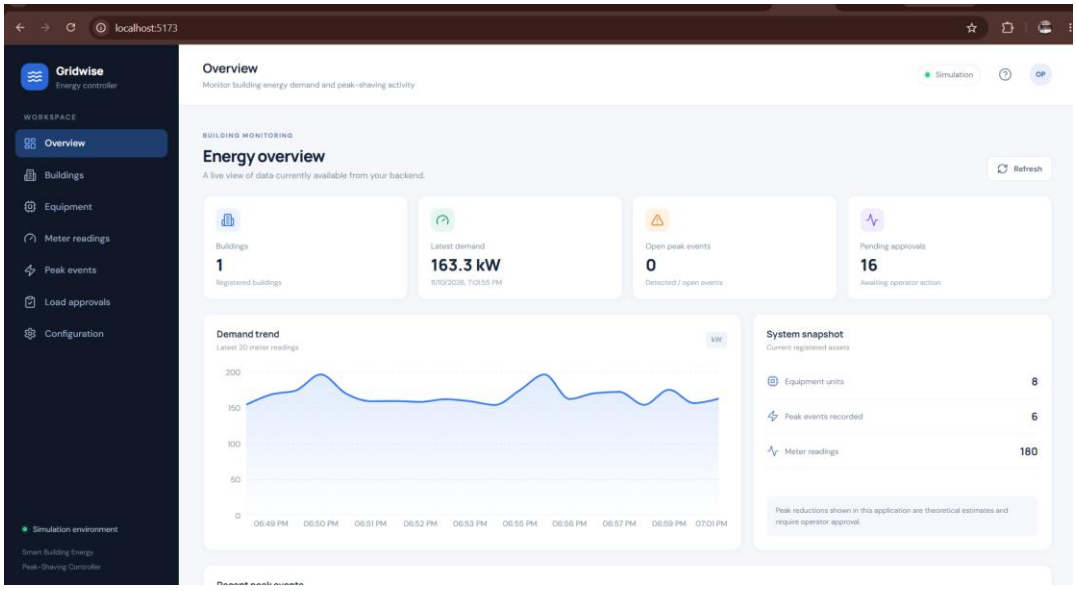


Figure 25. Deployed React dashboard showing the energy overview, summary cards and demand trend.

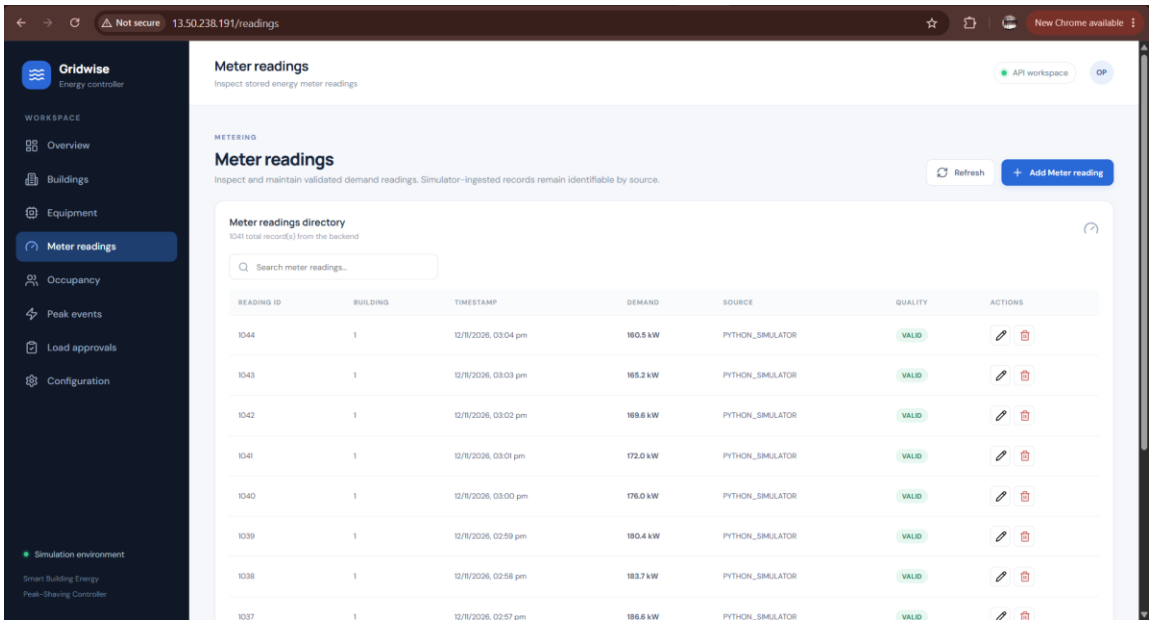


Figure 26. Meter readings page showing the readings table and simulator-ingested demand records.

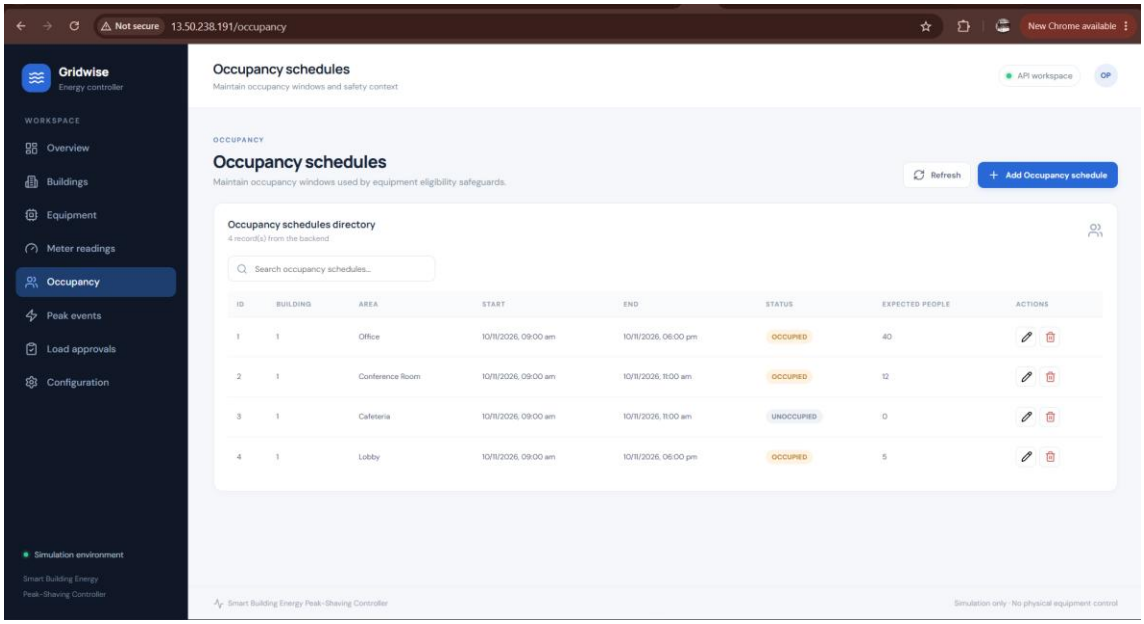


Figure 27. Occupancy schedules page showing the operator view for maintaining occupancy windows.

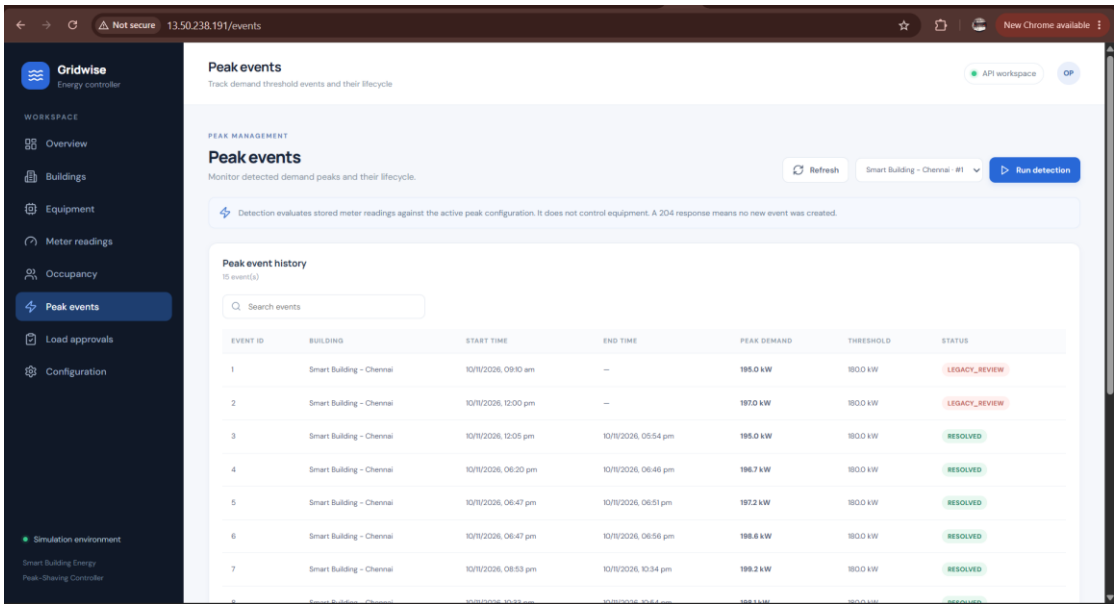


Figure 28. Peak events page showing detected demand events and their status.

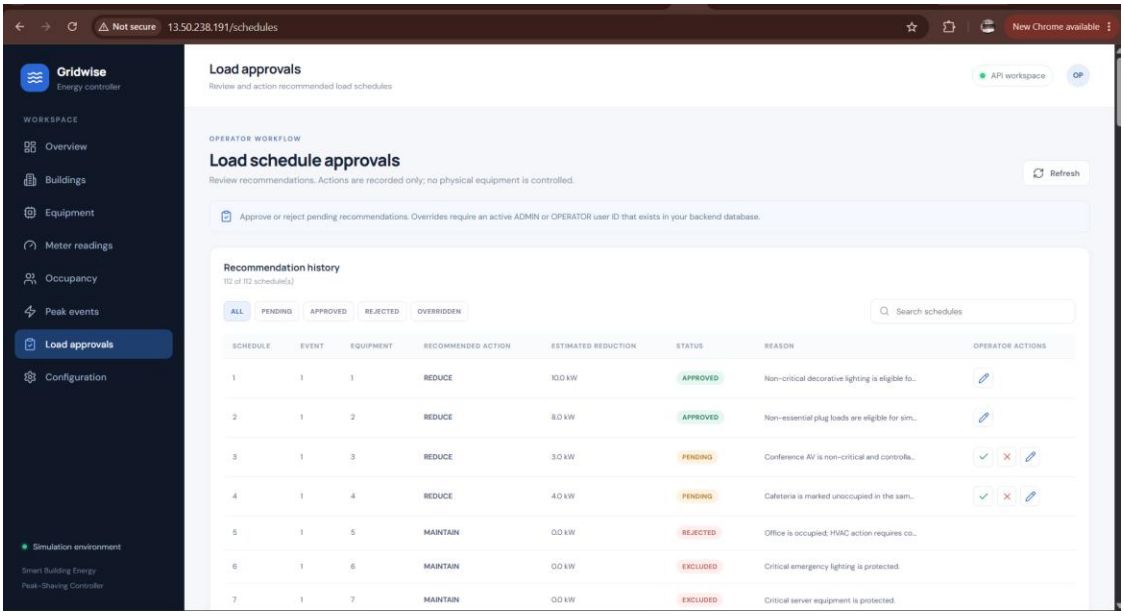


Figure 29. Load schedule approvals page showing recommendations available for operator review.

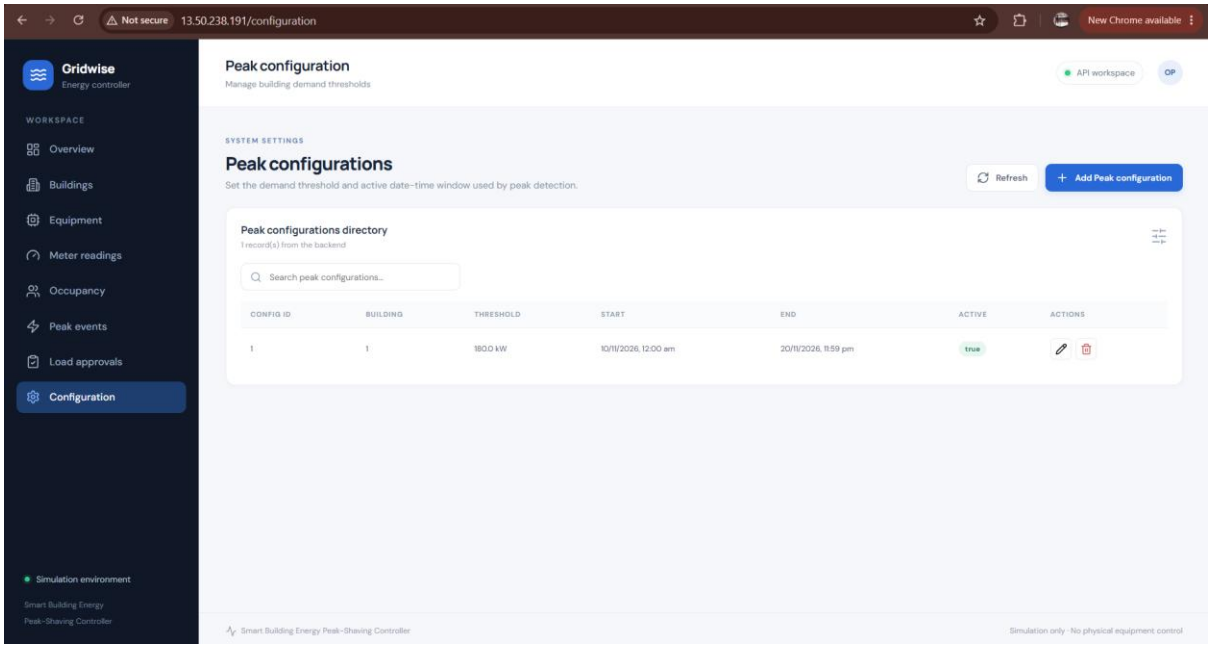


Figure 30. Peak configuration page showing the building demand threshold configuration.

11.1 Local Setup

11.1.1 Install and configure
PowerShell — from the project root:

cd frontend
npm ci
Copy-Item .env.example .env

Open frontend/.env and set:
VITE_API_BASE_URL=http://localhost:8082/api

11.1.2 Start and verify

npm run dev
Open http://localhost:5173.
Check the browser Network tab and confirm API requests go to http://localhost:8082/api.

11.1.3 Production build check

npm run build
Confirm the build completes and the dist folder is generated. Do not commit local .env files.

Use the commands in Section 11.1.1 to install dependencies and configure the local API URL.

RUN IN A TERMINAL IN THE FRONTEND DIRECTORY:

npm install

npm run dev

Open http://localhost:5173 in the browser. Check the browser console and Network panel if a page loads but its data does not. Common causes include an incorrect VITE_API_BASE_URL, a stopped backend, CORS mismatch or an API response error.

```
cd frontend
npm ci
npm run dev
```

Open http://localhost:5173. If the API base URL is overridden, copy .env.example to .env and set VITE_API_BASE_URL=http://localhost:8082/api. Do not commit .env files containing secrets.

12. Snowflake Analytics and Data Integration

Snowflake is the analytical destination, separate from MySQL which remains the operational source of truth. The integration process should extract operational records, transform and validate them, load Snowflake tables and reconcile counts/keys. Power BI connects to Snowflake rather than depending on browser-only calculations.

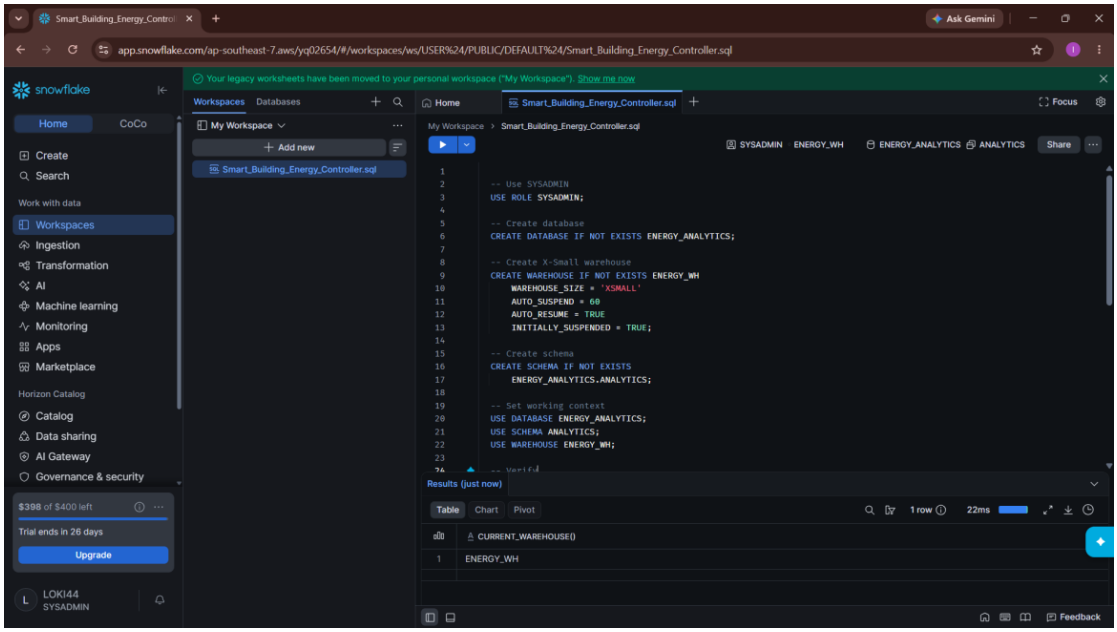


Figure 31. Snowflake worksheet showing the project SQL workspace and analytical database setup.

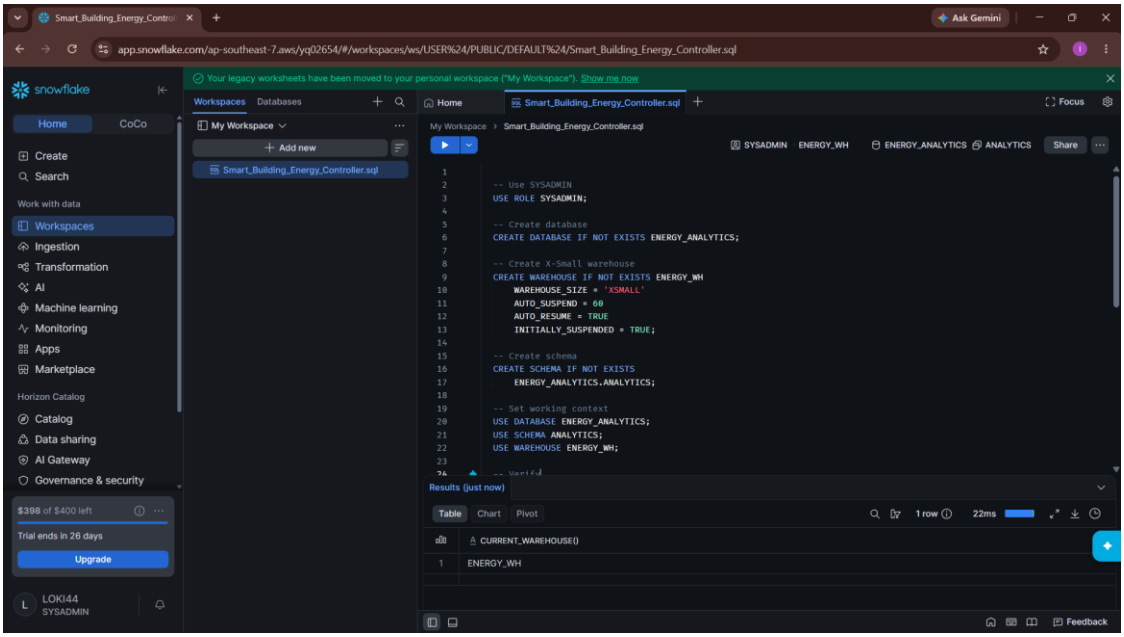


Figure 32. Snowflake worksheet showing the project SQL workspace and analytical database setup.

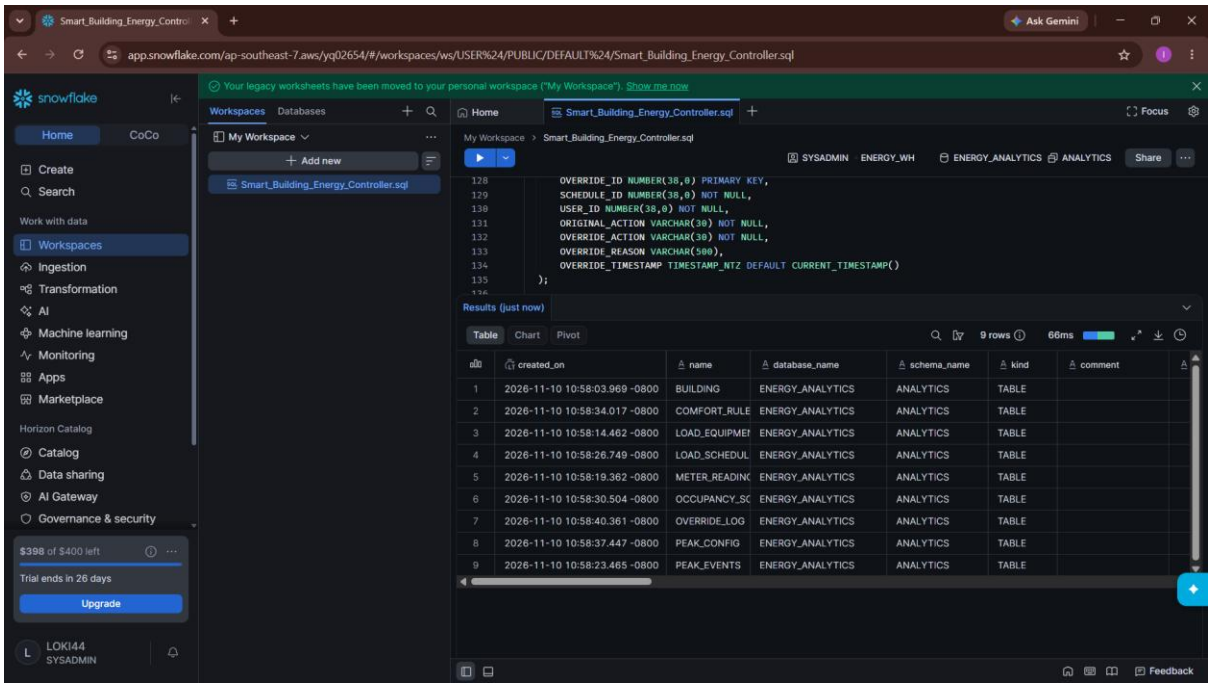


Figure 33. Snowflake worksheet showing database/schema or warehouse configuration for the analytics environment.

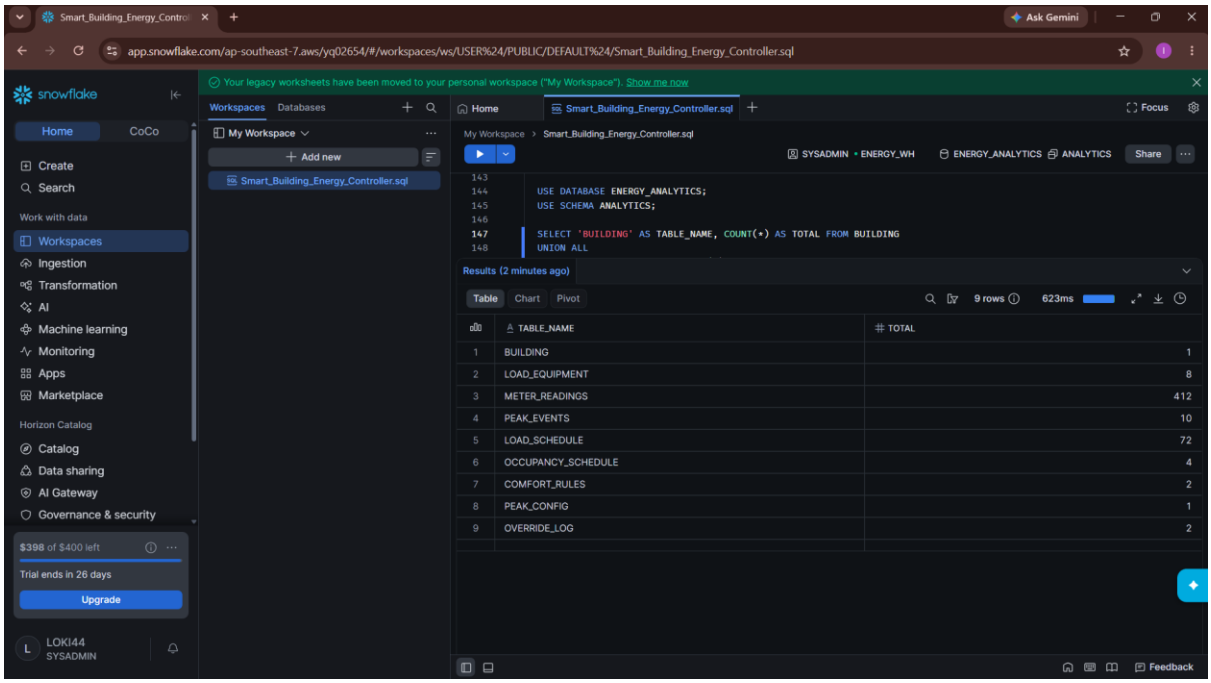


Figure 34. Snowflake worksheet showing the project table or file-format setup used for analytical loading.

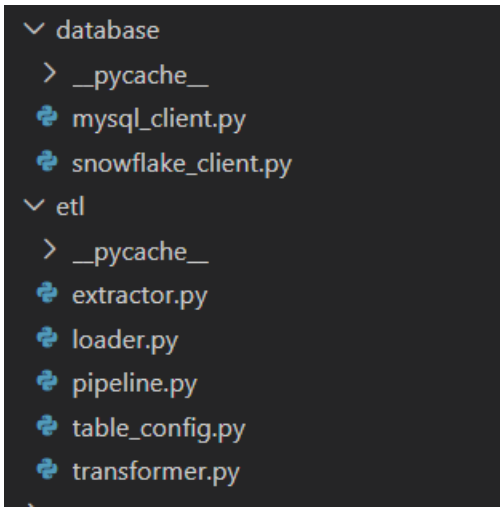


Figure 35. VS Code project explorer showing the Python Snowflake ETL modules, extractor, transformer, loader and pipeline.

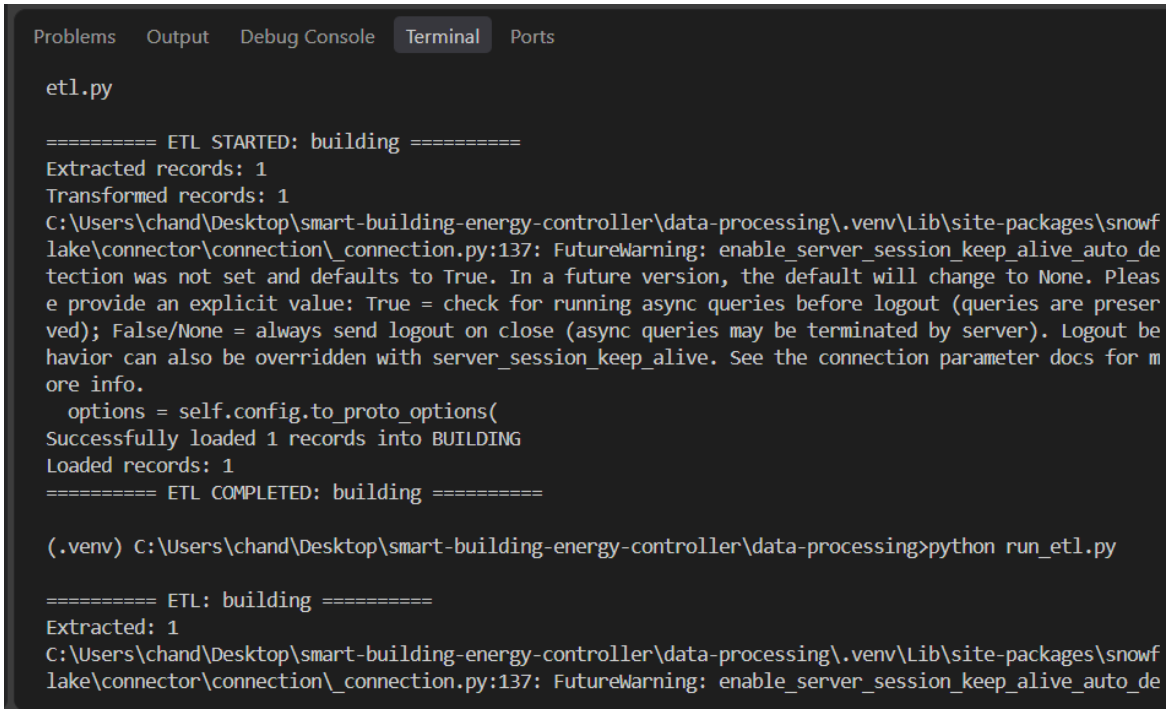


Figure 36. Python ETL execution output showing the start of the pipeline and extracted record counts.

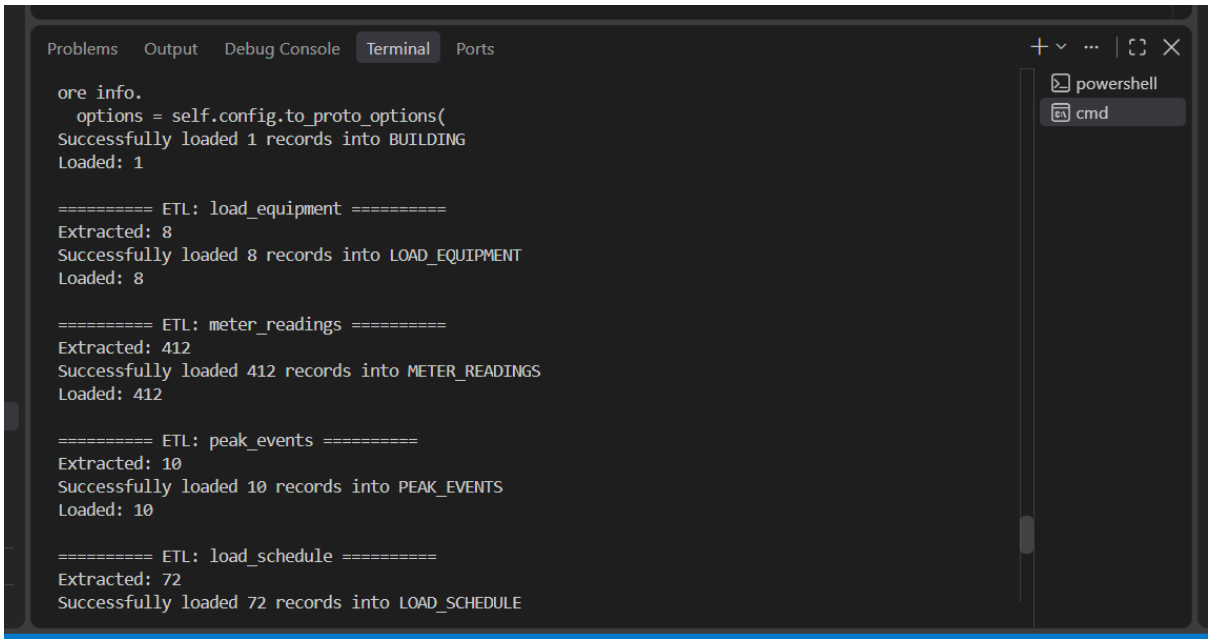


Figure 37. ETL execution output showing records successfully loaded into Snowflake analytical tables.

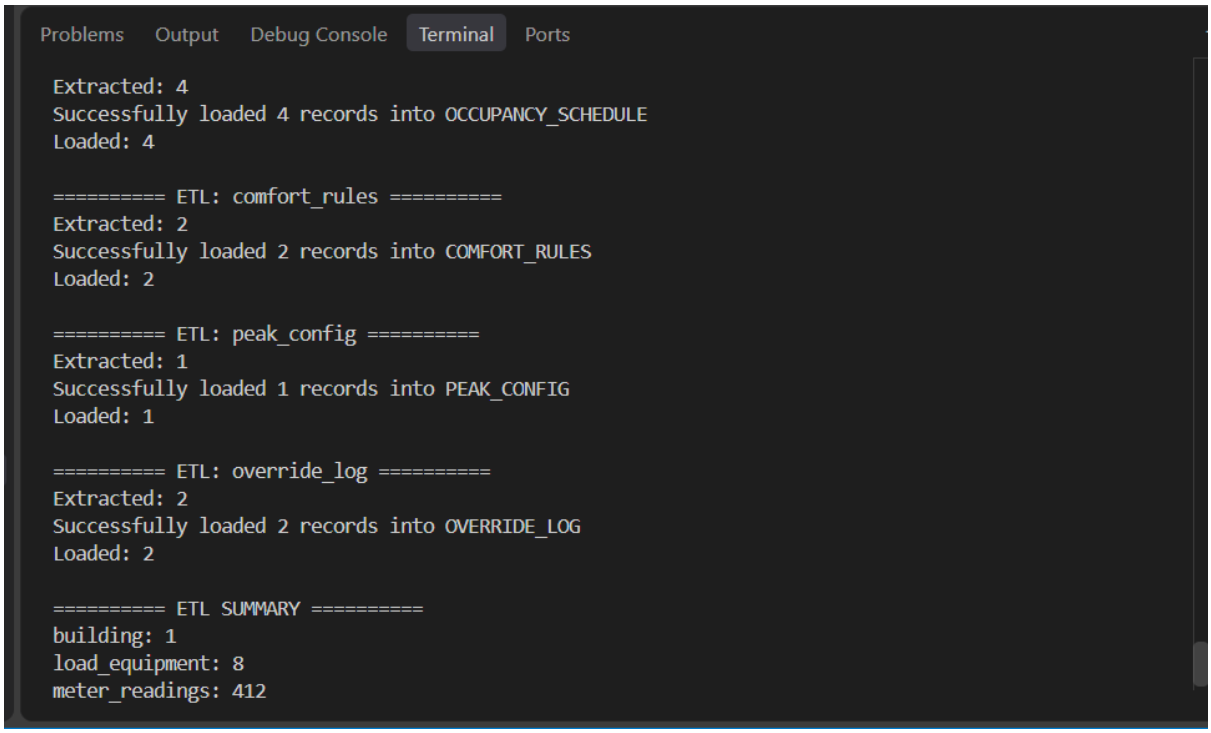


Figure 38. ETL execution output showing additional table loads and row counts for the project analytics dataset.

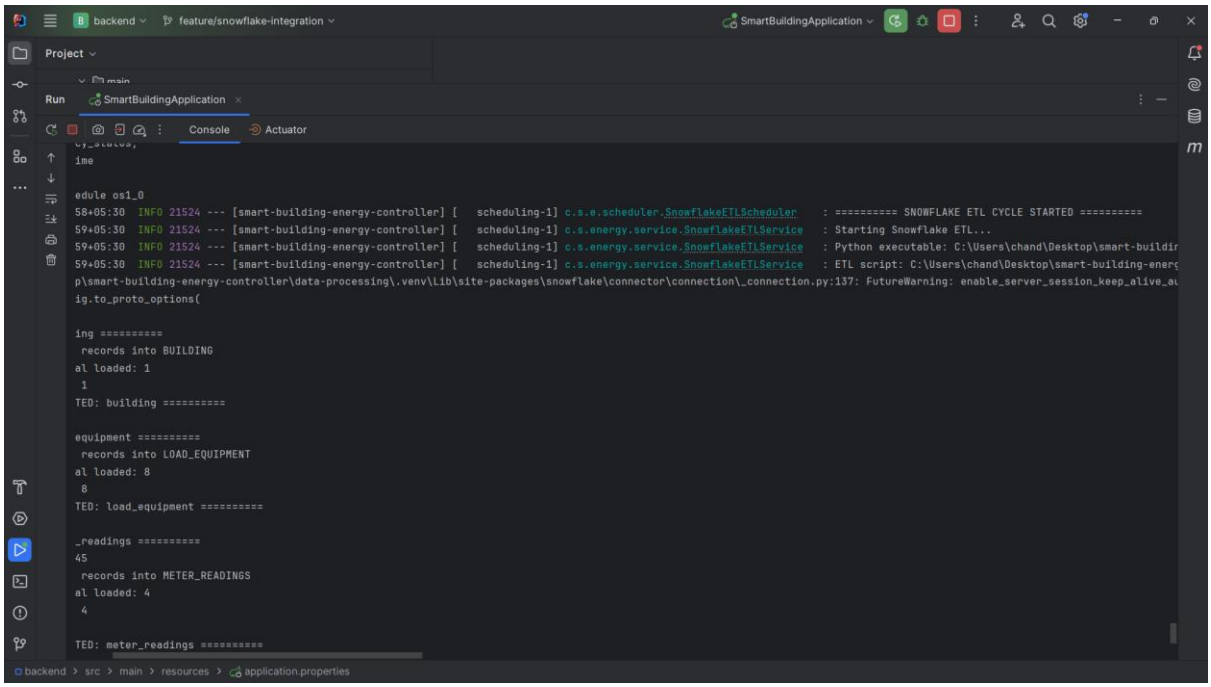


Figure 39. Spring Boot console output showing the application and Snowflake integration-related runtime activity.

12.1 Current Snowflake Environment

12.1.1 Select the recorded role and create/select project resources
Run in a Snowflake worksheet. Confirm the account and role are correct before executing:

```
USE ROLE SYSADMIN;

CREATE DATABASE IF NOT EXISTS ENERGY_ANALYTICS;

CREATE SCHEMA IF NOT EXISTS ENERGY_ANALYTICS.ANALYTICS;

CREATE WAREHOUSE IF NOT EXISTS ENERGY_WH
  WAREHOUSE_SIZE = 'XSMALL'
  AUTO_SUSPEND = 60
  AUTO_RESUME = TRUE
  INITIALLY_SUSPENDED = TRUE;

USE WAREHOUSE ENERGY_WH;
USE DATABASE ENERGY_ANALYTICS;
USE SCHEMA ANALYTICS;
```

12.1.2 Verify the resources

```
SHOW DATABASES LIKE 'ENERGY_ANALYTICS';

SHOW SCHEMAS IN DATABASE ENERGY_ANALYTICS;

SHOW WAREHOUSES LIKE 'ENERGY_WH';

SHOW TABLES IN SCHEMA ENERGY_ANALYTICS.ANALYTICS;
```

Use the existing table definitions and ETL mappings for table creation/loading. Do not create replacement tables with guessed columns.

Setting	Project value recorded
Account identifier	TWLZHPD-LT48562
User	LOKI44
Warehouse	ENERGY_WH
Database	ENERGY_ANALYTICS
Schema	ANALYTICS
Role used during setup	SYSADMIN

These names are environment-specific. Verify they still exist before executing SQL. Never include passwords or authentication tokens in the document.

12.2 Pipeline Flow

A practical run should be checked at each boundary rather than treating a successful script start as proof that the full analytics refresh completed.

- Extract: confirm the expected source tables and date/watermark range were read from MySQL.
- Transform: check required columns, timestamp parsing, null handling, data types and duplicate business keys.
- Load: confirm the target Snowflake tables were written and capture inserted/updated row counts from the actual run output.
- Reconcile: compare source and target counts for the same extraction window and investigate missing or duplicated keys.
- Report: refresh the Power BI dataset and record its refresh time before presenting the latest figures.

12.2.1 Reconcile the Snowflake load

Run a count query against each target table after the ETL finishes. Example pattern (replace <TARGET_TABLE> with an actual table shown by SHOW TABLES):

```
SELECT COUNT(*) AS ROW_COUNT
FROM ENERGY_ANALYTICS.ANALYTICS.<TARGET_TABLE>;
```

Compare the result with the source count for the same extraction window. Record the ETL run time, source window and inserted/updated counts.

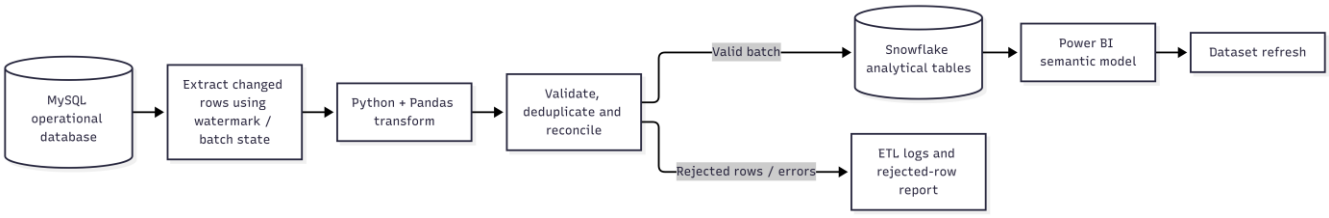


Figure 40. Data pipeline diagram showing MySQL extraction, Python/Pandas validation and transformation, Snowflake loading and Power BI refresh.

The project history describes incremental extraction and periodic ETL. Document the exact deployed schedule, watermark state, retry strategy, S3 staging role and Snowflake load command from the current ETL script and logs before claiming those mechanisms are production-grade. Keep analytics failures independent from the operational API.

13. Power BI Dashboard

The Power BI report was developed with Snowflake as its data source and contains two principal pages: an energy overview and peak-demand/operations analysis. The report is intended to help users inspect demand patterns, event status, thresholds and equipment classifications.

13.1 Page 1 – Energy Overview

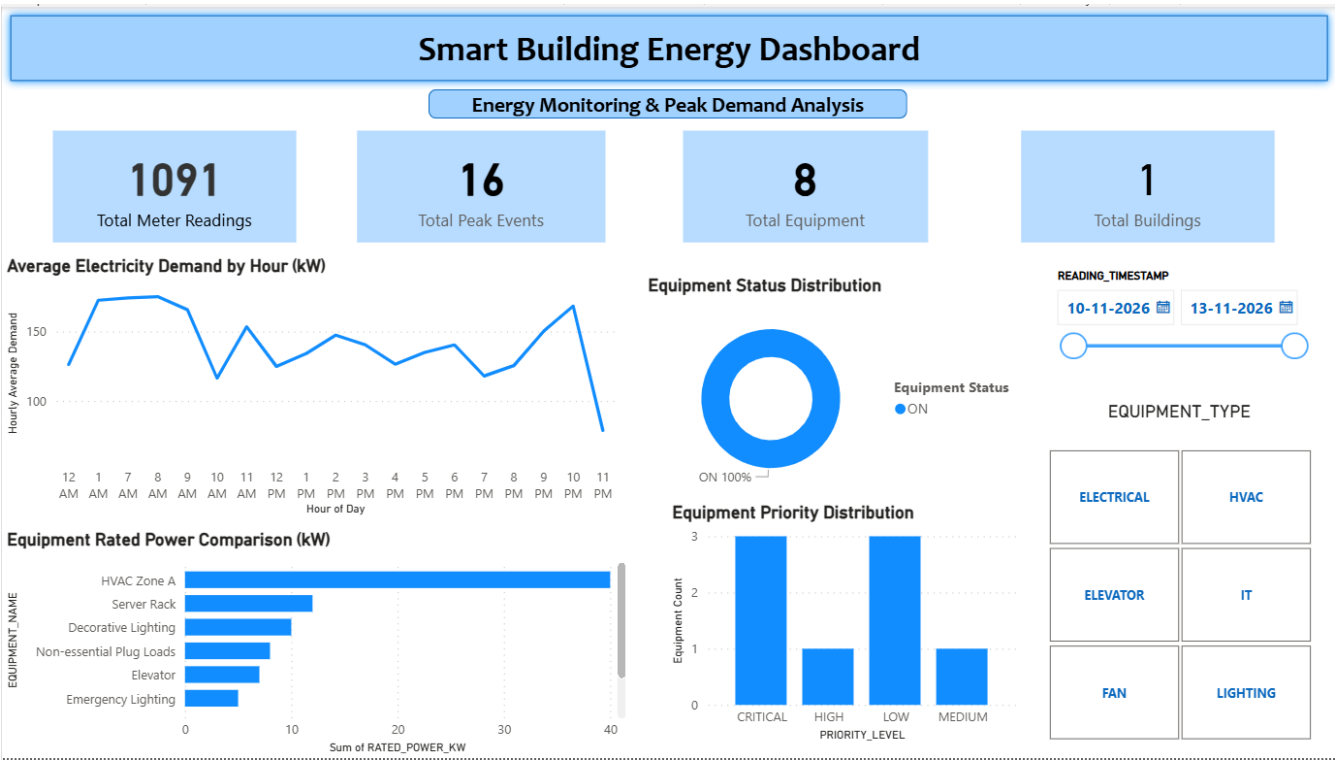


Figure 41. Power BI Energy Monitoring dashboard showing meter readings, peak events, equipment/building KPIs and hourly demand analysis.

- KPI cards include total meter readings, peak event count, equipment count and building count (values vary with dataset refresh).
- Demand trend visual groups demand by hour/time dimension where available.
- Equipment status distribution and rated power by equipment provide operational context. Rated power is not the same as measured demand.
- Equipment priority distribution helps distinguish protected and non-critical classes.
- Date and equipment-type slicers are included; confirm model relationships before assuming every slicer filters every visual.

13.2 Page 2 – Peak Demand and Operations

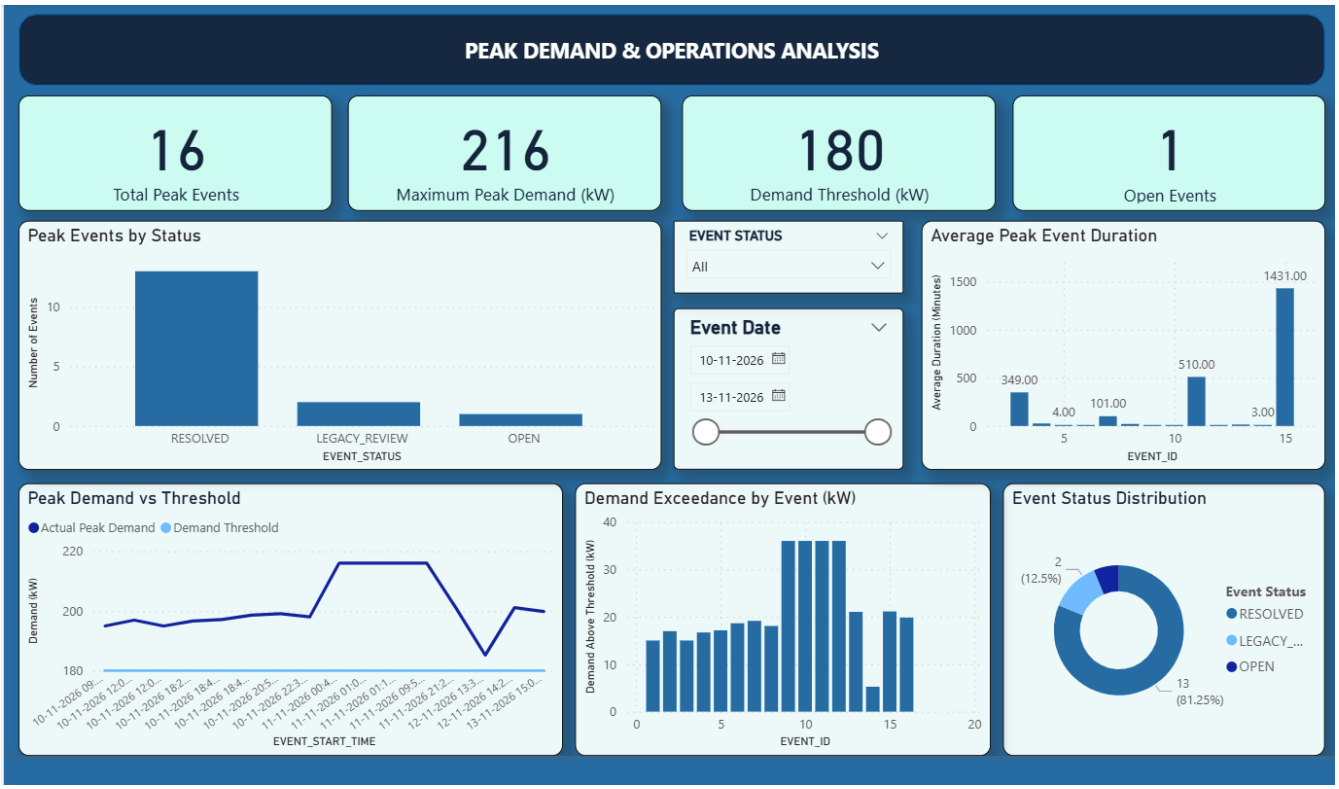


Figure 42. Power BI Peak Demand and Operations page showing event KPIs, event status and demand-threshold analysis.

- Peak event KPIs include event count, maximum peak demand, configured threshold and open event count.
- Peak demand versus threshold chart highlights demand excursions.
- Event status distribution distinguishes OPEN, RESOLVED and LEGACY_REVIEW states in the captured dataset.
- Event duration and demand exceedance calculations support operational review.

Power BI Import mode requires a dataset refresh to reflect new Snowflake records. KPI values in screenshots are historical snapshots, not guaranteed current values. Record refresh date and reconcile against Snowflake before final presentation.

14. Docker Containerization

Docker packages the React frontend and Spring Boot backend into application images. Local Docker Desktop is used for image building and validation. The published images are then consumed by the AWS EC2 deployment process, which is documented separately in Section 15.

14.1 Docker Image Creation

Docker images were created for the Smart Building Energy Peak-Shaving Controller application. Separate images were maintained for the React frontend and Spring Boot backend. MySQL 8.0 was used as the database image.

The application images were built using the Docker configuration available in the project repository.

RUN IN LOCAL TERMINAL (PROJECT ROOT DIRECTORY):

docker compose build

docker images

The docker compose build command builds the application images based on the configured services and their Dockerfiles. The docker images command displays the images available in the local Docker environment.

Docker images used in the project:

Component	Docker Image
Frontend	loki1100/smart-building-frontend
Backend	loki1100/smart-building-backend
Database	mysql:8.0

14.2 Docker Container Verification

After building the images, the Docker containers were checked to verify their availability and execution status.

RUN IN LOCAL TERMINAL:

docker ps -a

docker compose ps

docker images

The commands display the container status, Docker Compose service status and locally available images.

14.3 Docker Hub Image Publishing

Docker Hub was used as the container image registry to store the frontend and backend application images. The Docker Hub username used for publishing was loki1100.

The application images were identified using the following repository names:

Frontend image:

loki1100/smart-building-frontend:aws

Backend image:

loki1100/smart-building-backend:latest

Docker Hub authentication:

docker login

The images were published to their respective Docker Hub repositories. The configured GitHub Actions workflow also builds and publishes application images as part of the CI/CD process.

14.4 Docker Containerization Summary

The containerization process involved building and validating the frontend and backend images, identifying the required image tags, and publishing the application images to Docker Hub.

Local Docker was used for development and image validation. AWS EC2 deployment and runtime verification are covered in Section 15, while automated image publishing and deployment workflow details are covered in Section 16.

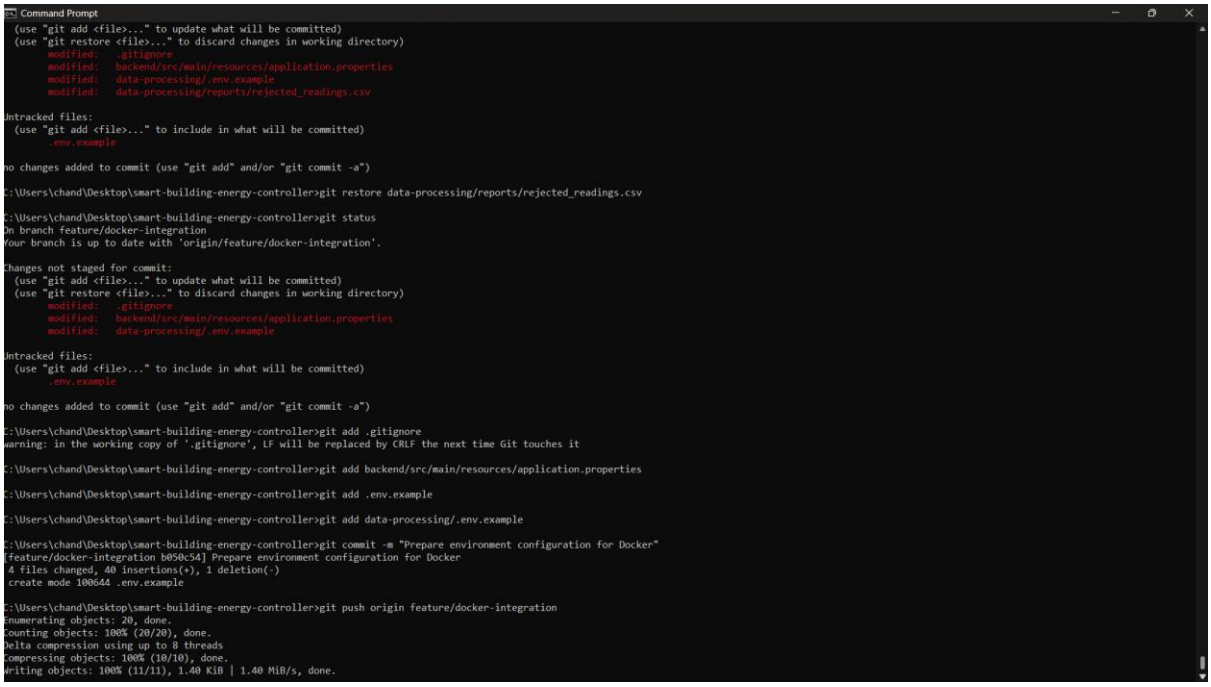


Figure 43. Git status output showing the project files modified before committing Docker and deployment changes.

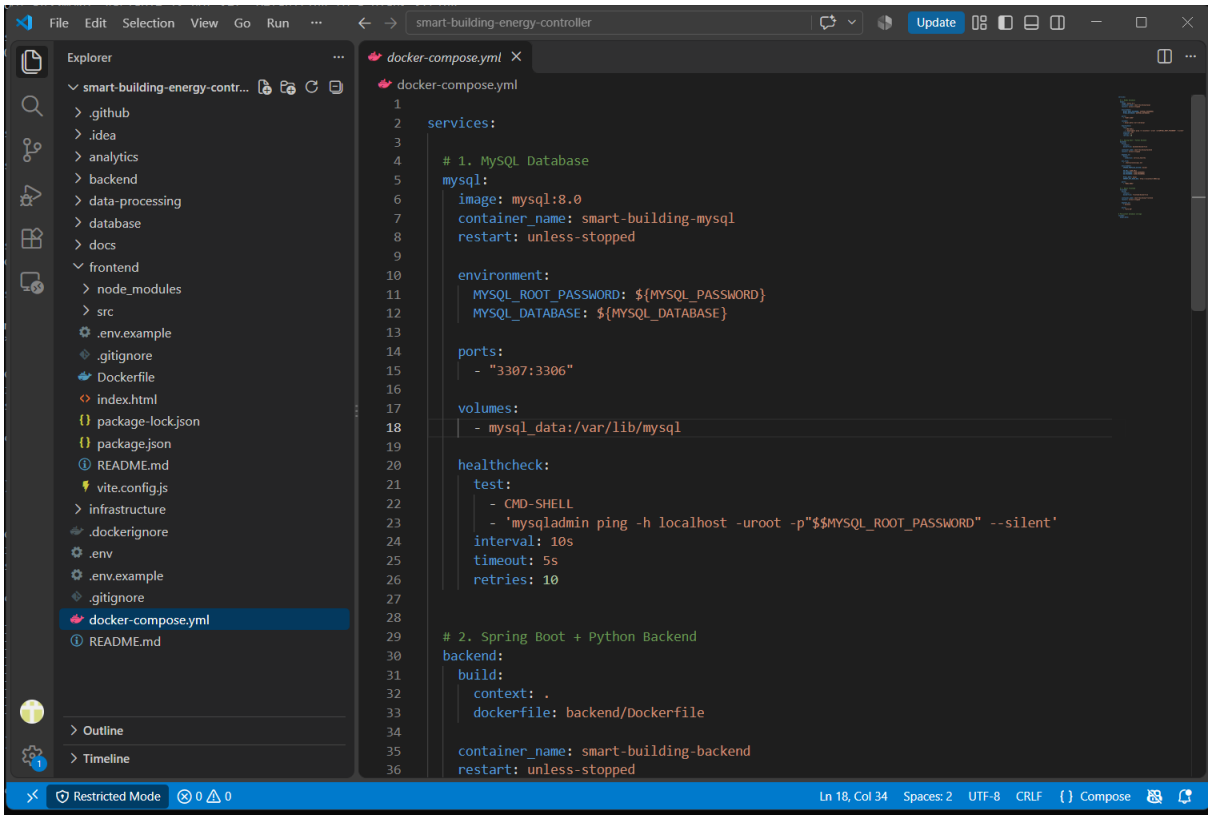


Figure 44. Docker Compose configuration in the IDE showing the application services and their container settings.

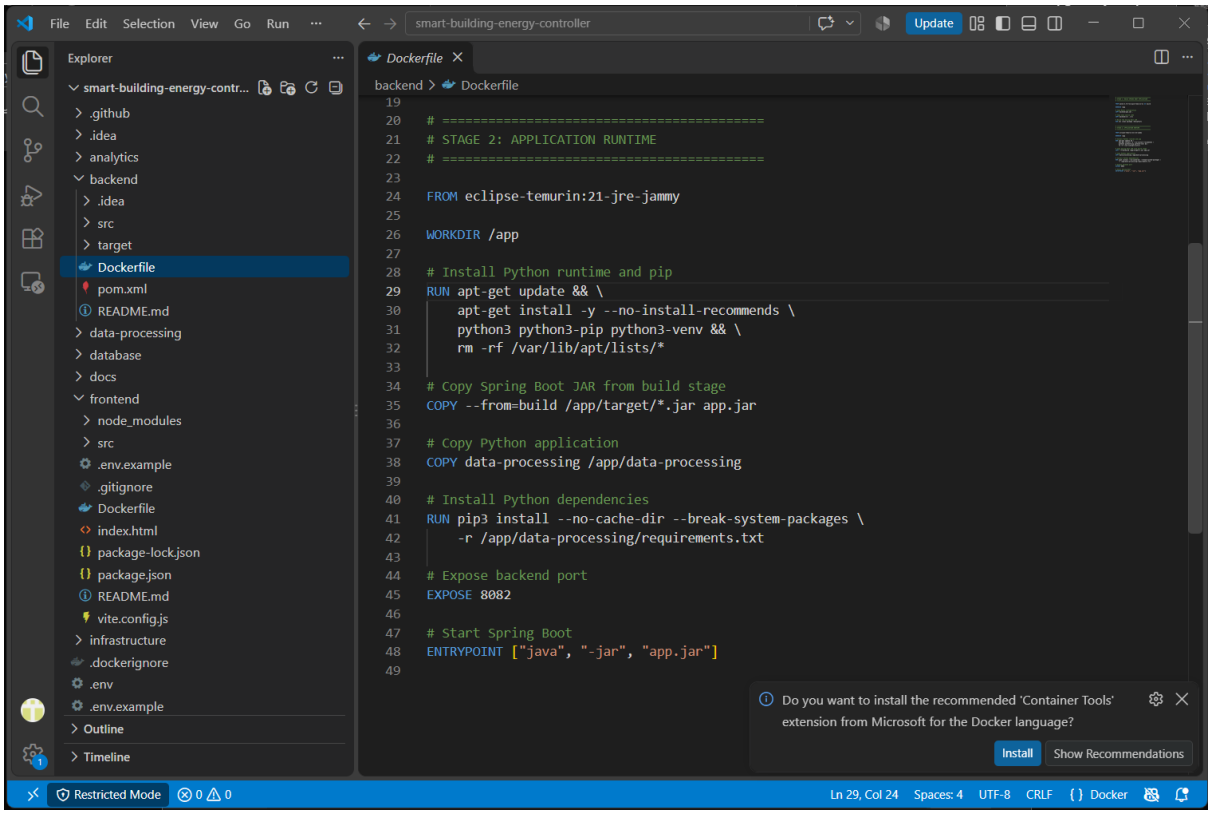


Figure 45. Backend Dockerfile showing the container build instructions for the Spring Boot application.

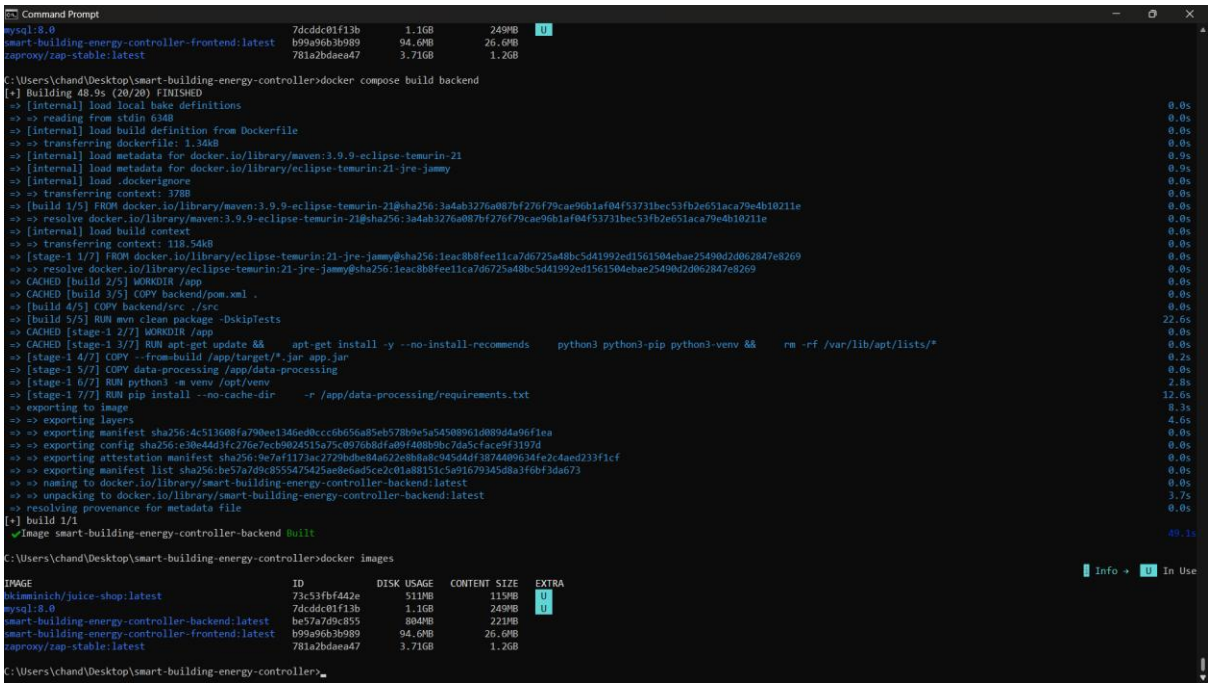


Figure 46. Docker image listing showing the local MySQL and application images available on the developer machine.

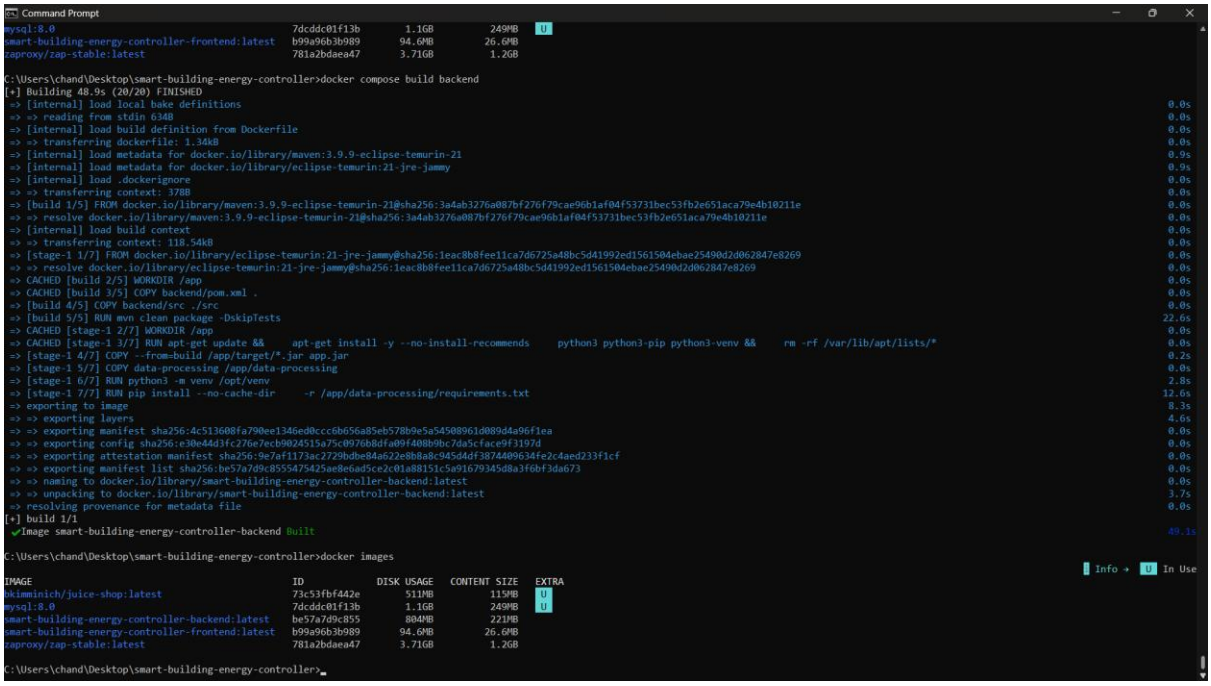


Figure 47. Docker image listing showing the local MySQL and application images available on the developer machine.

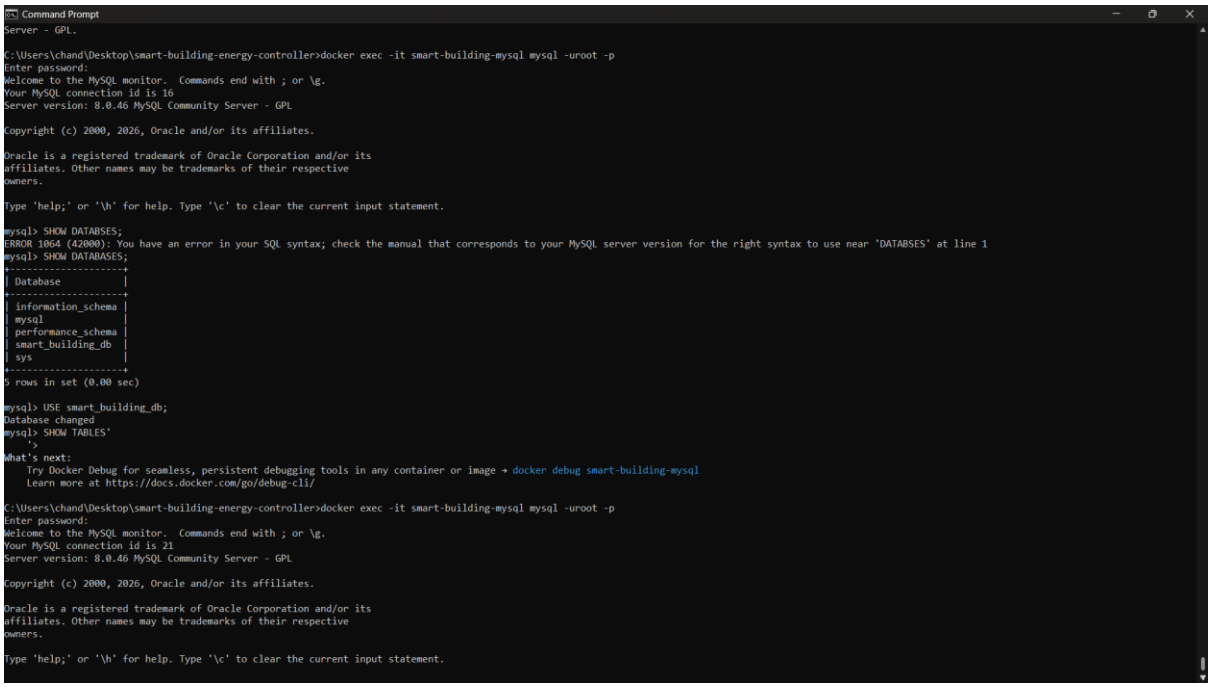


Figure 48. MySQL client session opened inside the smart-building-mysql container for database verification.

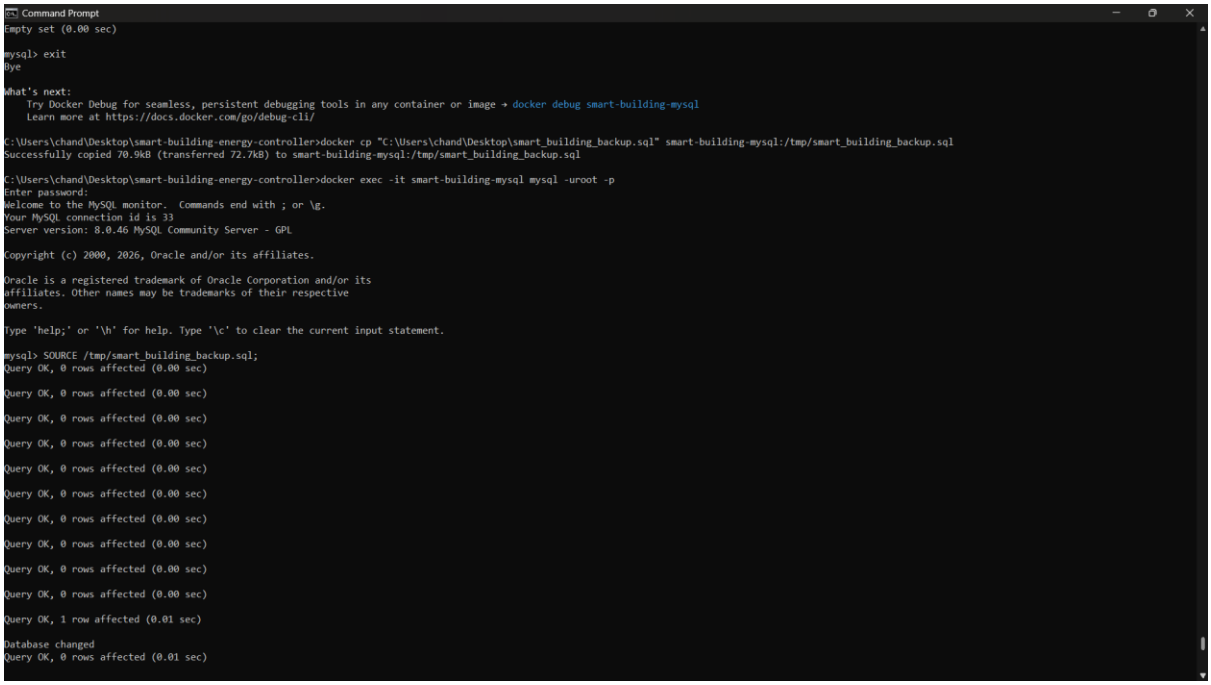


Figure 49. MySQL client output showing the result of a database inspection query and a clean exit from the client.



Figure 50. Spring Boot backend container startup output showing the application process running in Docker.

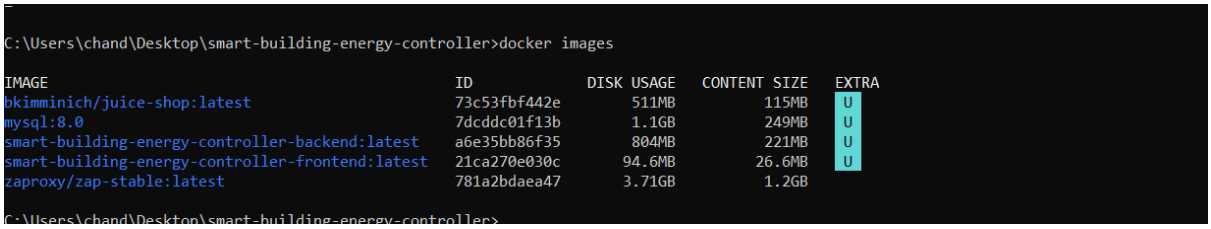


Figure 51. docker images output showing the built backend and frontend images and supporting local images.

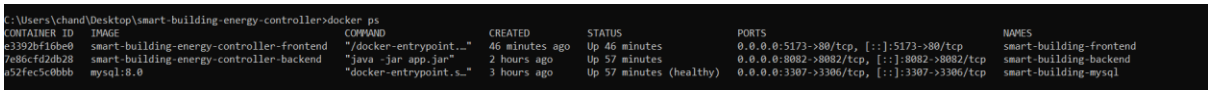


Figure 52. docker ps output showing the running frontend container and its published local port.

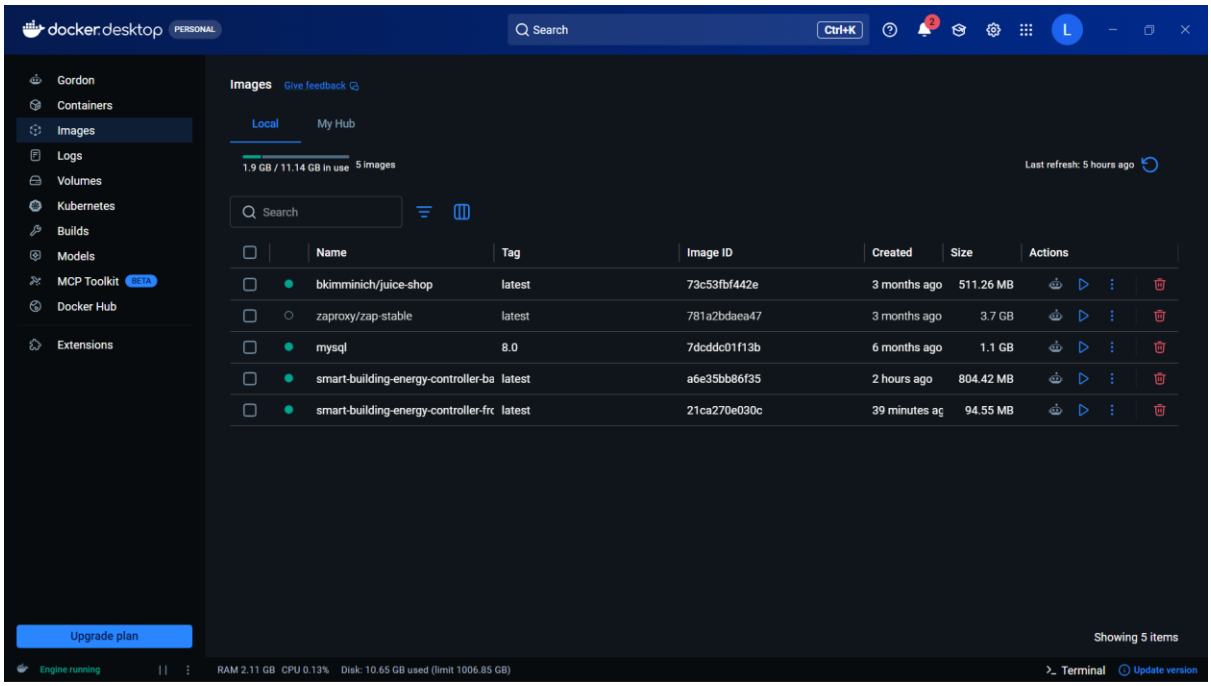


Figure 53. Docker Desktop Images view showing locally built application images and image storage details.

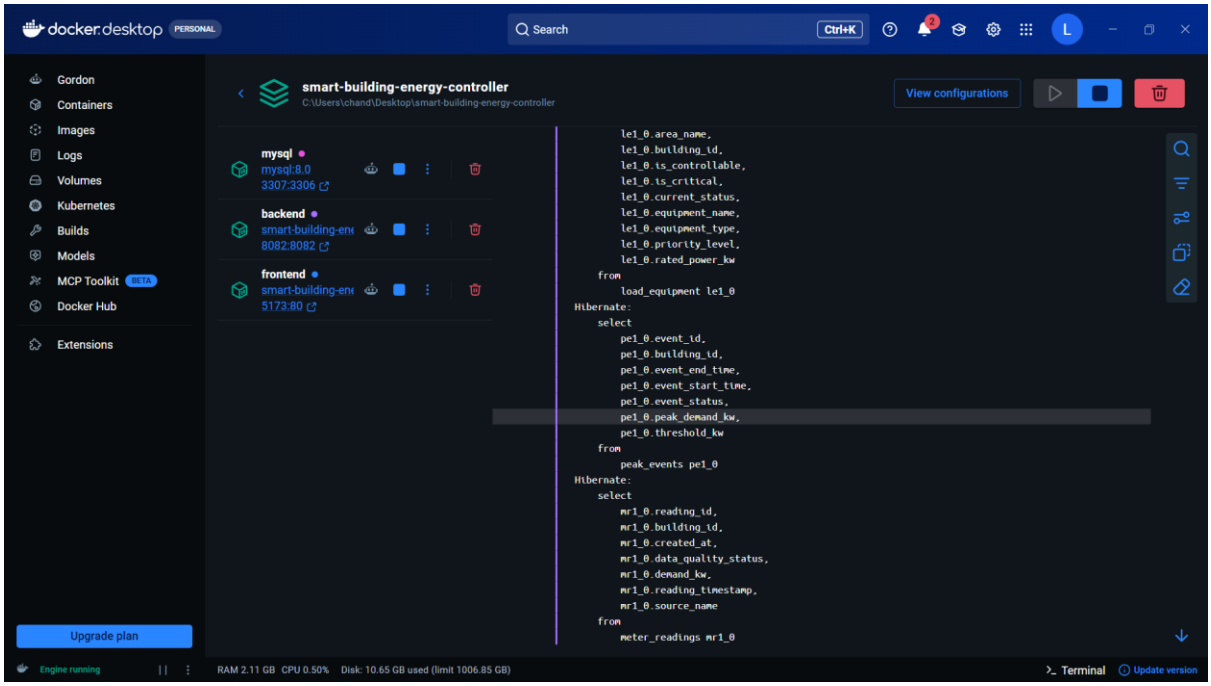


Figure 54. Docker Desktop Containers view showing the project containers and their service/port information.

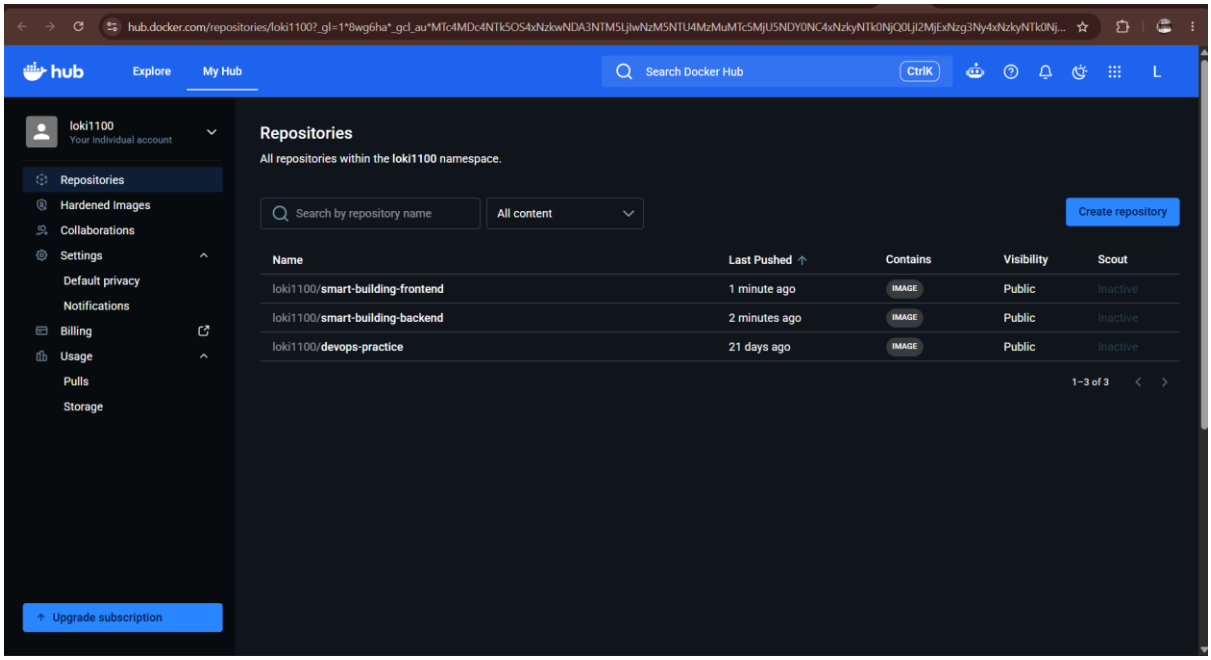


Figure 55. Docker Hub account repository page showing the project image repositories.

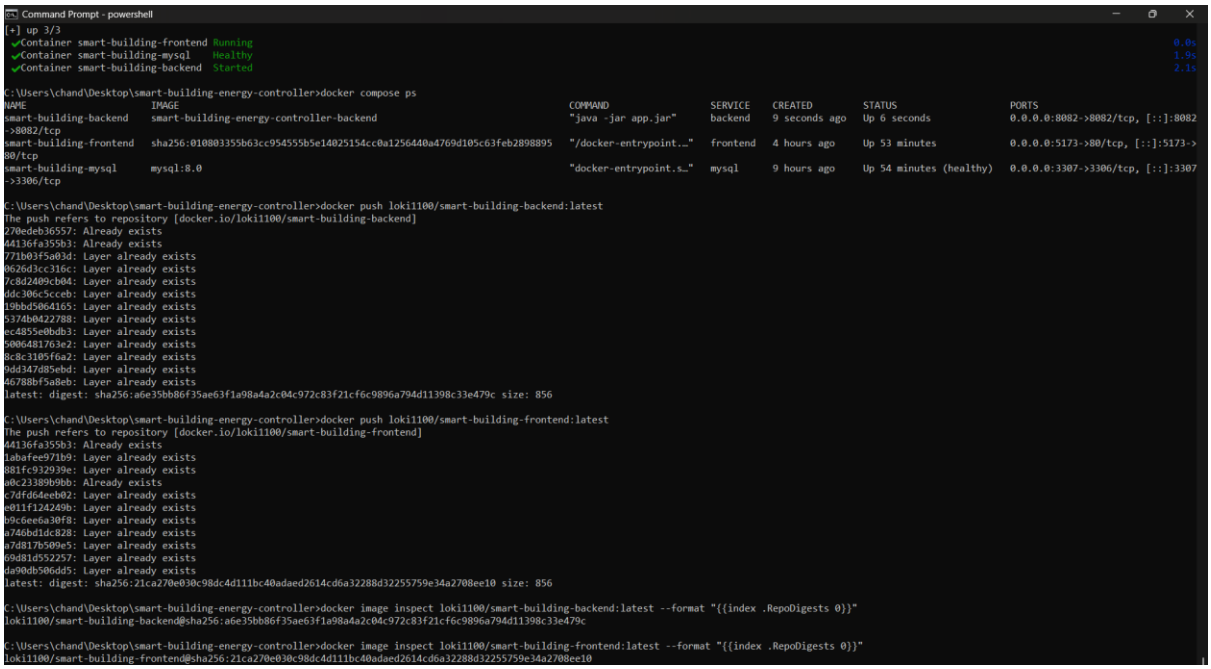


Figure 56. Docker Compose output showing the frontend, backend and MySQL services starting, with the database health check.

15. AWS EC2 Deployment

The Smart Building Energy Peak-Shaving Controller is deployed on an Ubuntu-based AWS EC2 instance in the eu-north-1 region. Docker Compose runs the React frontend, Spring Boot backend and MySQL database containers. The project directory on the EC2 host is /home/ubuntu/smart-building-energy-controller.

The frontend is exposed through host port 80. The backend (container port 8082) and MySQL (container port 3306) are not host-published in the recorded deployment. Use the current EC2 public IPv4 address shown in the AWS console; it may change after a stop/start unless an Elastic IP is assigned.

15.1 EC2 Instance Configuration

Create the EC2 host:
1. In AWS Console, open EC2 → Launch instance and select the approved Ubuntu Server AMI.
2. Select the instance type and storage size approved for the project.
3. Create or select the SSH key pair and keep the private key secure.
4. Select the project security group and attach SmartBuildingEC2S3Role.
5. Launch the instance, wait for status checks to pass, then connect using SSH/PuTTY.
6. Install and verify Docker/Compose before deploying the application.

Configuration	Recorded value
Cloud provider	AWS
Region	eu-north-1
Instance label	smart-building-server
Operating system	Ubuntu
Linux account	ubuntu
Project directory	/home/ubuntu/smart-building-energy-controller

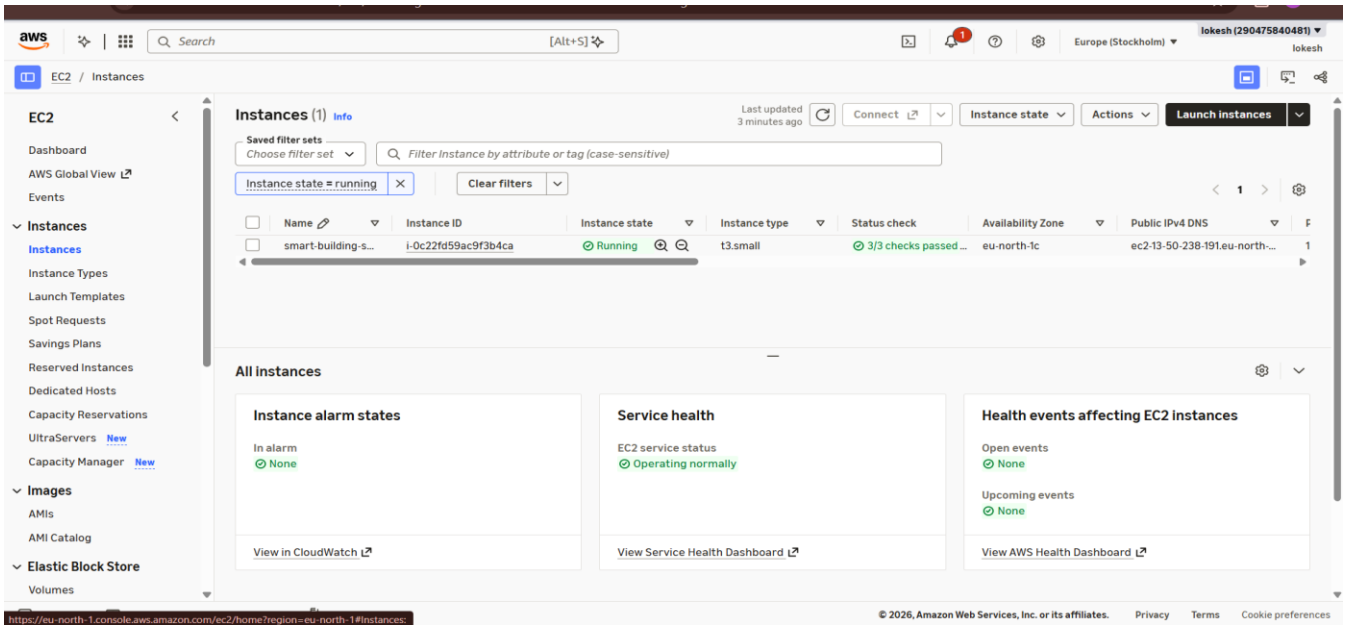


Figure 57. AWS EC2 console showing the project instance and its instance state/health information.

EC2 console showing the instance and its running/health status.

15.2 Network and Security Configuration

Configure the EC2 security group:
1. Add HTTP TCP 80 from the intended website audience (0.0.0.0/0 only if the site is public).
2. Add HTTPS TCP 443 only after HTTPS is configured.
3. Add SSH TCP 22 from the administrator's current public IP only.
4. Do not add public inbound rules for backend 8082 or MySQL 3306.

Allow HTTP TCP port 80 for frontend access.

Allow HTTPS TCP port 443 only when HTTPS is configured.

Restrict SSH TCP port 22 to the administrator’s current public IP.

Do not expose MySQL port 3306 publicly.

Do not expose backend port 8082 publicly unless an approved secured access method is configured.

Keep application credentials, tokens and private keys outside source control and screenshots.

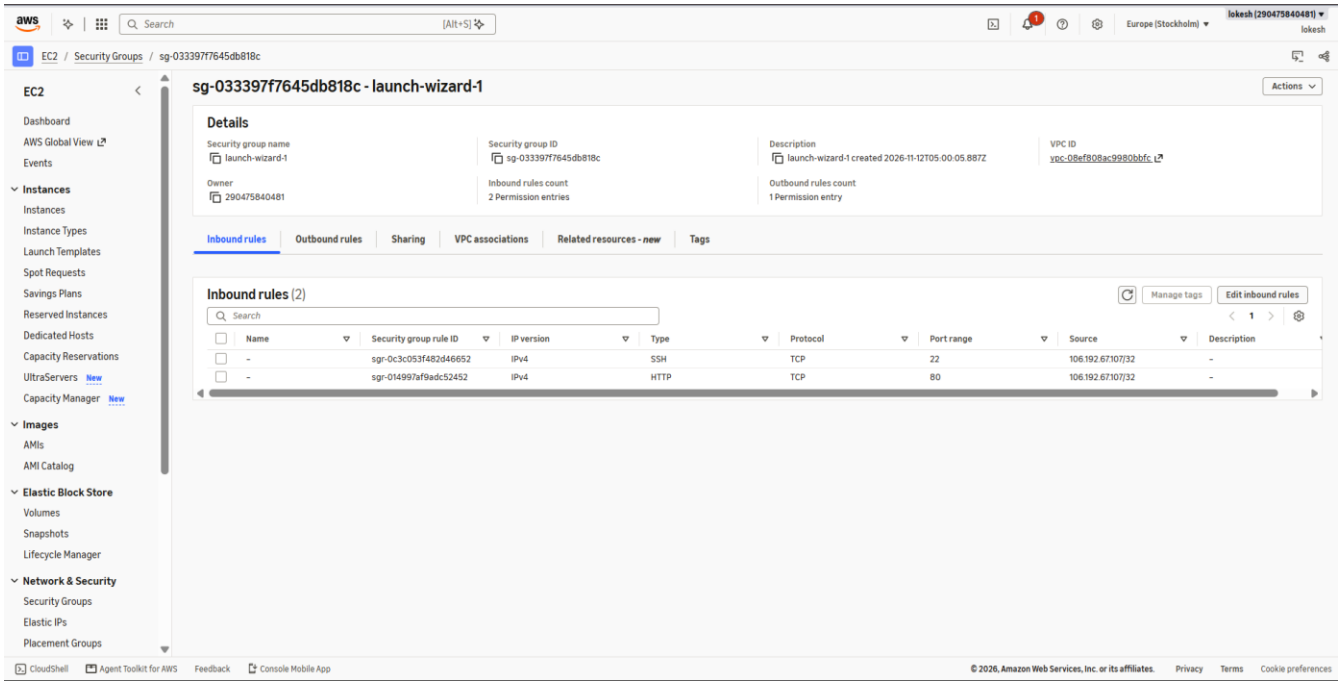


Figure 58. AWS security group inbound rules showing the network access configuration for the EC2 application host.

15.3 AWS IAM Role and Instance Permissions

Attach SmartBuildingEC2S3Role to the EC2 instance. It is used by AWS CLI for the private S3 backup workflow.

Project values: role SmartBuildingEC2S3Role | policy SmartBuildingS3BackupPolicy | bucket smart-building-energy-lokesh-2026 | prefix mysql-backups/.

Steps: create the S3 policy, attach it to the EC2 role, then attach the role to the EC2 instance.

The captured policy object ARN is bucket-wide (/*). Change it to the mysql-backups/* prefix.

AmazonS3FullAccess is also attached in the screenshot. Remove it after the custom policy passes the backup tests.

Create and attach the EC2 role:
1. IAM → Roles → Create role.
2. Trusted entity: AWS service → EC2.
3. Attach SmartBuildingS3BackupPolicy; role name SmartBuildingEC2S3Role.
4. Review and create.
5. EC2 → Instances → select instance → Actions → Security → Modify IAM role → select SmartBuildingEC2S3Role

→ Save.

6. Reconnect and run aws sts get-caller-identity.

1. Open IAM → Roles → Create role.

2. Select AWS service → EC2 as the trusted entity/use case.

3. Attach SmartBuildingS3BackupPolicy and name the role SmartBuildingEC2S3Role.

4. Review the permissions and create the role.

5. Open EC2 → Instances → select the instance → Actions → Security → Modify IAM role. Select SmartBuildingEC2S3Role and save.

6. Connect to EC2 and complete the checks in Section 17.2.

For policy scope and S3 verification commands, follow Section 17.2.

After attaching the role, run this command on EC2:

RUN IN PuTTY CONNECTED TO AWS EC2:

```
aws sts get-caller-identity
```

If access is denied, recheck the attached role, policy, bucket name, region and prefix. Do not grant administrator access as a workaround.

The IAM screenshots and S3 permission checks are shown in Section 17.

15.4 Connecting to EC2 and Accessing the Project Directory

The EC2 instance is accessed remotely using PuTTY and the configured SSH key.

RUN IN PuTTY CONNECTED TO AWS EC2:

```
cd /home/ubuntu/smart-building-energy-controller
pwd
ls -la
```

These commands open the project directory, confirm the current working directory and list the project files.

```
ubuntu@ip-172-31-5-156: ~/smart-building-energy-controller
remote: Enumerating objects: 381, done.
remote: Counting objects: 100% (381/381), done.
remote: Compressing objects: 100% (221/221), done.
remote: Total 381 (delta 123), reused 347 (delta 96), pack-reused 0 (from 0)
Receiving objects: 100% (381/381), 2.74 MiB | 27.50 MiB/s, done.
Resolving deltas: 100% (123/123), done.
ubuntu@ip-172-31-5-156:~$ cd smart-building-energy-controller
ubuntu@ip-172-31-5-156:~/smart-building-energy-controller$ ls -la
total 60
drwxrwxr-x 9 ubuntu ubuntu 4096 Nov 12 05:16 .
drwxr-x--- 5 ubuntu ubuntu 4096 Nov 12 05:16 ..
-rw-rw-r-- 1 ubuntu ubuntu  305 Nov 12 05:16 .dockerignore
-rw-rw-r-- 1 ubuntu ubuntu  232 Nov 12 05:16 .env.example
drwxrwxr-x 7 ubuntu ubuntu 4096 Nov 12 05:16 .git
drwxrwxr-x 3 ubuntu ubuntu 4096 Nov 12 05:16 .github
-rw-rw-r-- 1 ubuntu ubuntu  304 Nov 12 05:16 .gitignore
-rw-rw-r-- 1 ubuntu ubuntu 4108 Nov 12 05:16 README.md
drwxrwxr-x 3 ubuntu ubuntu 4096 Nov 12 05:16 backend
drwxrwxr-x 9 ubuntu ubuntu 4096 Nov 12 05:16 data-processing
drwxrwxr-x 2 ubuntu ubuntu 4096 Nov 12 05:16 database
-rw-rw-r-- 1 ubuntu ubuntu 1393 Nov 12 05:16 docker-compose.yml
drwxrwxr-x 2 ubuntu ubuntu 4096 Nov 12 05:16 docs
drwxrwxr-x 3 ubuntu ubuntu 4096 Nov 12 05:16 frontend
ubuntu@ip-172-31-5-156:~/smart-building-energy-controller$ cat docker-compose.yml

services:

  # 1. MySQL Database
  mysql:
    image: mysql:8.0
    command: --lower_case_table_names=1
    container_name: smart-building-mysql
    restart: unless-stopped

    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_PASSWORD}
      MYSQL_DATABASE: ${MYSQL_DATABASE}

    ports:
      - "3307:3306"

    volumes:
      - mysql_data:/var/lib/mysql

    healthcheck:
      test:
        - CMD-SHELL
        - 'mysqladmin ping -h localhost -uroot -p"$$MYSQL_ROOT_PASSWORD" --silent'
      interval: 10s
      timeout: 5s
      retries: 10
```

Figure 59. Git command output showing the project repository being cloned or updated on the EC2 host.

```
ubuntu@ip-172-31-5-156: ~/smart-building-energy-controller
login as: ubuntu
Authenticating with public key "smart-building-key"
Welcome to Ubuntu 26.04.1 LTS (GNU/Linux 7.0.0-1014-aws x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Thu Nov 12 05:13:56 UTC 2026

  System load:  0.02              Temperature:          -273.1 C
  Usage of /:   12.6% of 23.10GB  Processes:            123
  Memory usage: 23%               Users logged in:      0
  Swap usage:   0%                IPv4 address for ens5: 172.31.5.156


Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


*** System restart required ***
Last login: Thu Nov 12 05:05:46 2026 from 14.99.219.164
ubuntu@ip-172-31-5-156:~$ groups
ubuntu adm cdrom sudo dip lxd docker
ubuntu@ip-172-31-5-156:~$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
d5e71e642bf5: Download complete
Digest: sha256:cf1b20a0c89a666b6950fecda7ccbc1b1a86520e7c53f908f3a21e79d0d60458
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
```

Figure 60. PuTTY/SSH terminal showing a successful connection to the Ubuntu EC2 instance and the project directory.

15.5 Docker Compose Deployment on EC2

The docker-compose.aws.yml file defines the EC2 services. From the project directory, check the configured image names/tags, pull the images and start the services:

RUN IN PuTTY CONNECTED TO AWS EC2 FROM THE PROJECT DIRECTORY:

```
docker images

docker ps

docker compose -f docker-compose.aws.yml ps

docker compose -f docker-compose.aws.yml pull
docker compose -f docker-compose.aws.yml up -d
docker compose -f docker-compose.aws.yml ps
```

These commands display available images, running containers and the status of the Compose services.

Service / container	Purpose	Network exposure recorded
smart-building-frontend	Serves the React web UI.	Host port 80 mapped to container port 80.
smart-building-backend	Spring Boot REST API and scheduler.	Container port 8082; not host-published.
smart-building-mysql	MySQL operational database.	Container port 3306 inside the Compose network; not host-published.

15.6 Application Logs and Runtime Verification

Docker logs help verify service startup and investigate runtime errors.

RUN IN PuTTY CONNECTED TO AWS EC2:

```
docker compose -f docker-compose.aws.yml logs --tail=100 backend

docker compose -f docker-compose.aws.yml logs --tail=100 frontend

docker compose -f docker-compose.aws.yml logs --tail=100 mysql
```

Confirm the Compose service keys in docker-compose.aws.yml before running these commands; service keys can differ from container names.

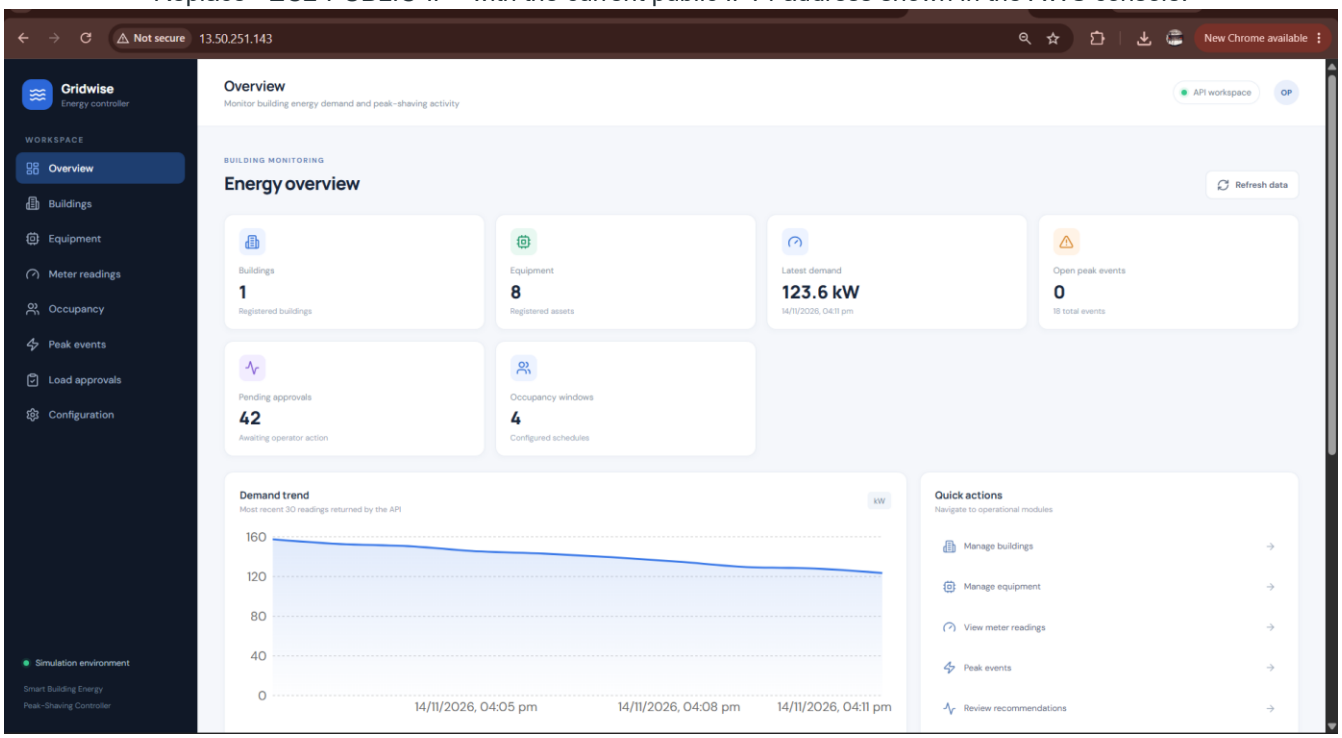
15.7 Frontend Application Access

After the frontend container is running and the EC2 security group allows HTTP traffic, open the current EC2 public IPv4 address in a browser.

BROWSER ADDRESS:

http://<EC2-PUBLIC-IP>

Replace <EC2-PUBLIC-IP> with the current public IPv4 address shown in the AWS console.



15.8 GitHub Actions Self-Hosted Runner Verification

The EC2 instance hosts a GitHub Actions self-hosted runner that executes the deployment job on the EC2 host. The runner is installed as a Linux systemd service.

RUN IN PuTTY CONNECTED TO AWS EC2:

sudo systemctl status actions.runner.Loki-0110-smart-building-energy-controller.smart-building-ec2-runner.service

This command checks whether the runner service is active.

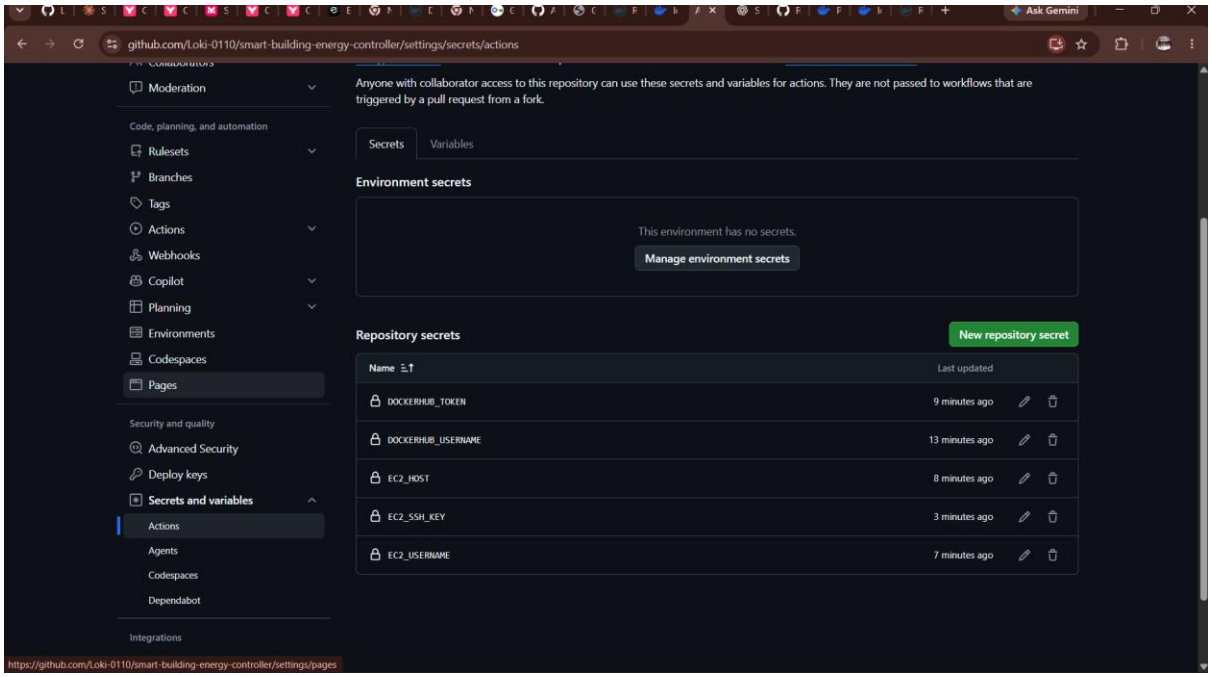


Figure 61. GitHub Actions repository settings page showing the Actions secrets configuration area.

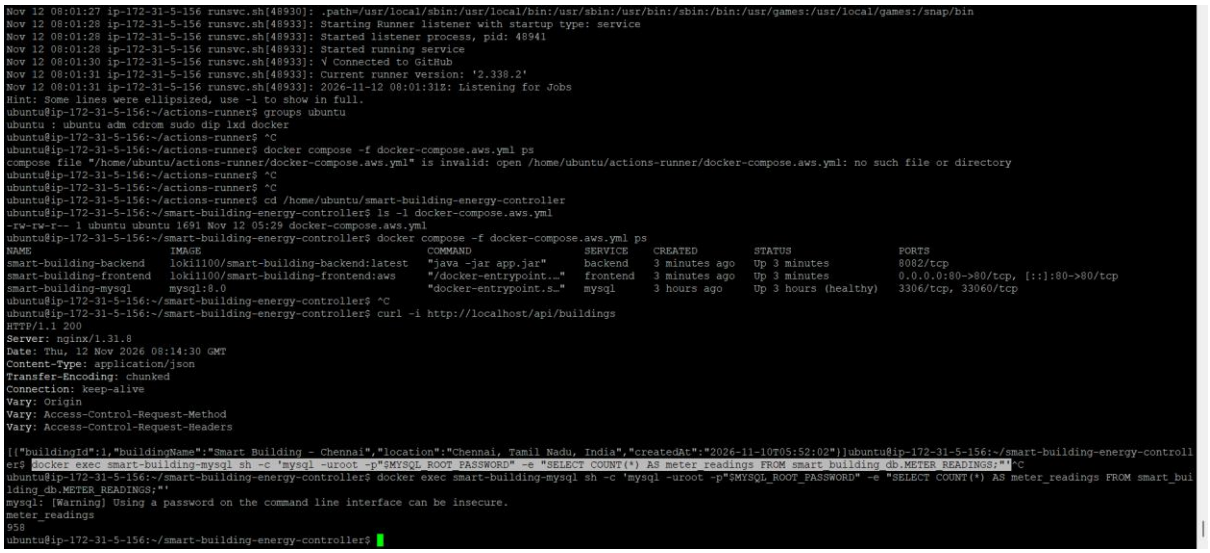


Figure 62. EC2 terminal log showing the GitHub Actions self-hosted runner service starting and connecting.

15.9 EC2 Start, Stop and Recovery

Stopping the EC2 instance interrupts the application because the containers run on that host. After the instance is started again, verify Docker and the application containers. Containers configured with an appropriate restart policy can restart automatically when Docker starts, provided the deployment remains intact and the containers were not manually stopped.

The GitHub Actions self-hosted runner should reconnect when the EC2 instance and its systemd service are running.

RUN IN PuTTY CONNECTED TO AWS EC2:

sudo systemctl status actions.runner.Loki-0110-smart-building-energy-controller.smart-building-ec2-runner.service

sudo systemctl restart actions.runner.Loki-0110-smart-building-energy-controller.smart-building-ec2-runner.service

docker ps

Use the status command to verify the runner. Restart it only when required, then check the application containers.

16. GitHub Actions and CI/CD

The project uses GitHub Actions for build and deployment automation. The recorded deployment approach uses an EC2 self-hosted runner service rather than relying solely on SSH from a hosted runner. The workflow builds/tests project components, builds and publishes Docker images to Docker Hub, then executes deployment steps on the EC2 runner.

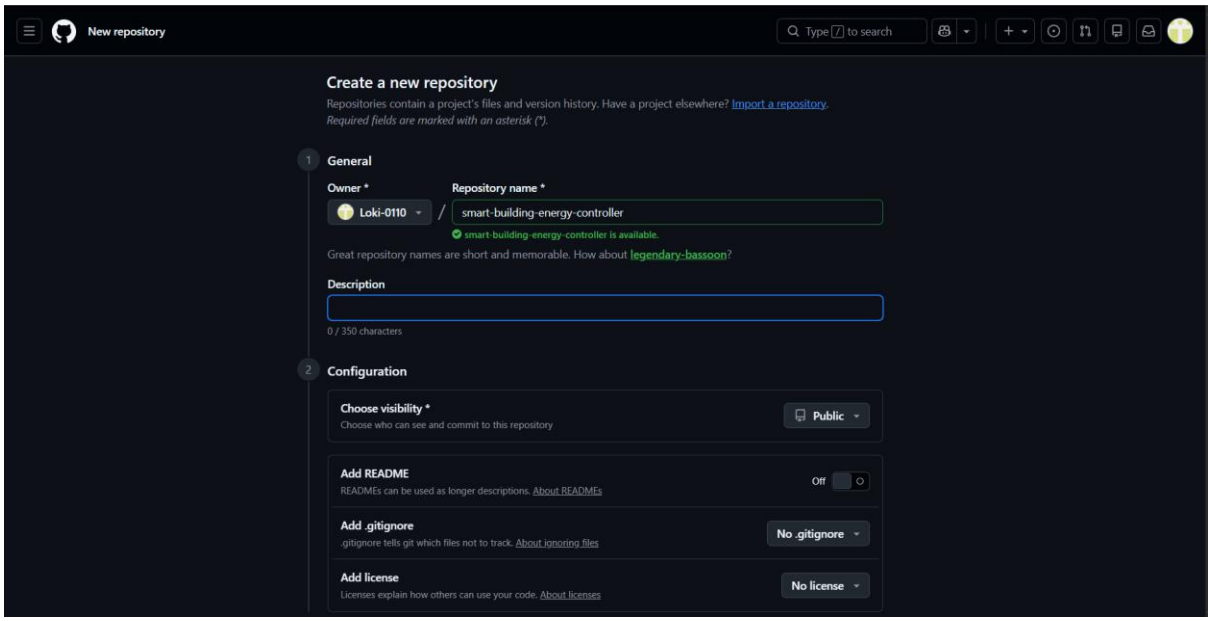


Figure 63. GitHub new repository page showing the creation of the project repository.

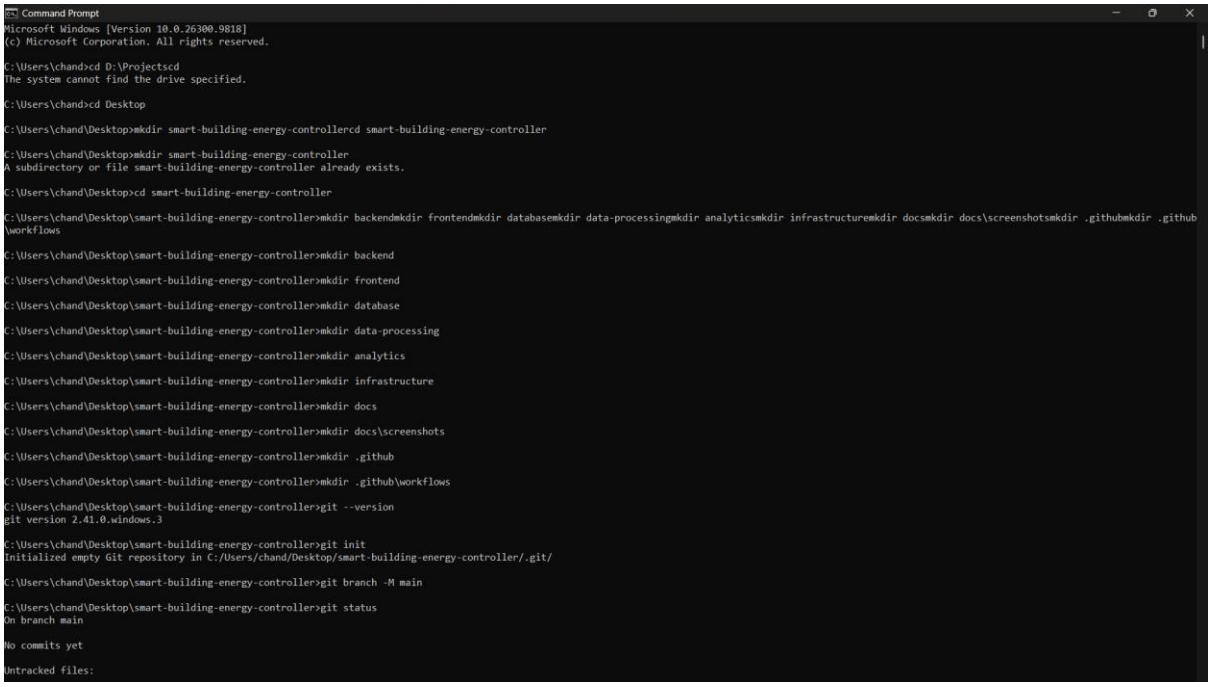


Figure 64. Windows Command Prompt showing navigation to the project directory and initial Git setup commands.

```
C:\Users\chand\Desktop\smart-building-energy-controller>git remote add origin https://github.com/Loki-0110/smart-building-energy-controller.git

C:\Users\chand\Desktop\smart-building-energy-controller>git remote -v
origin  https://github.com/Loki-0110/smart-building-energy-controller.git (fetch)
origin  https://github.com/Loki-0110/smart-building-energy-controller.git (push)
```

Figure 65. Git remote configuration output showing the GitHub origin URL for the project repository.

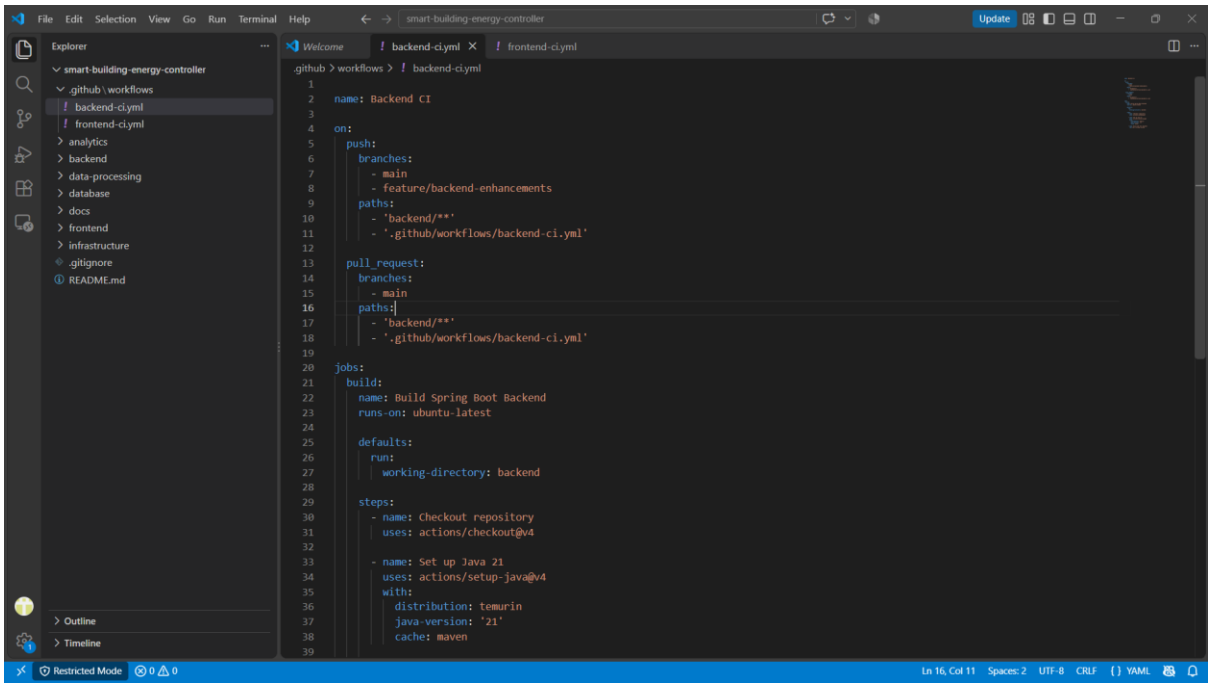


Figure 66. VS Code view of the backend GitHub Actions workflow YAML and its build/test steps.

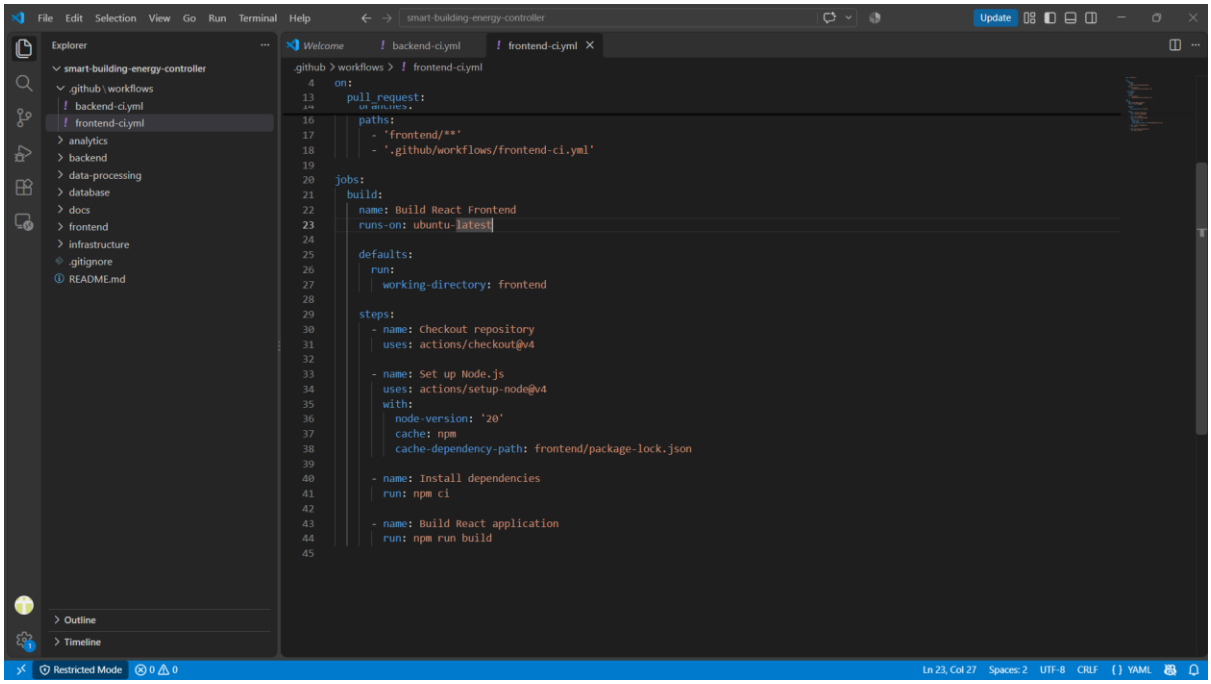


Figure 67. VS Code view of the frontend GitHub Actions workflow YAML and its build steps.

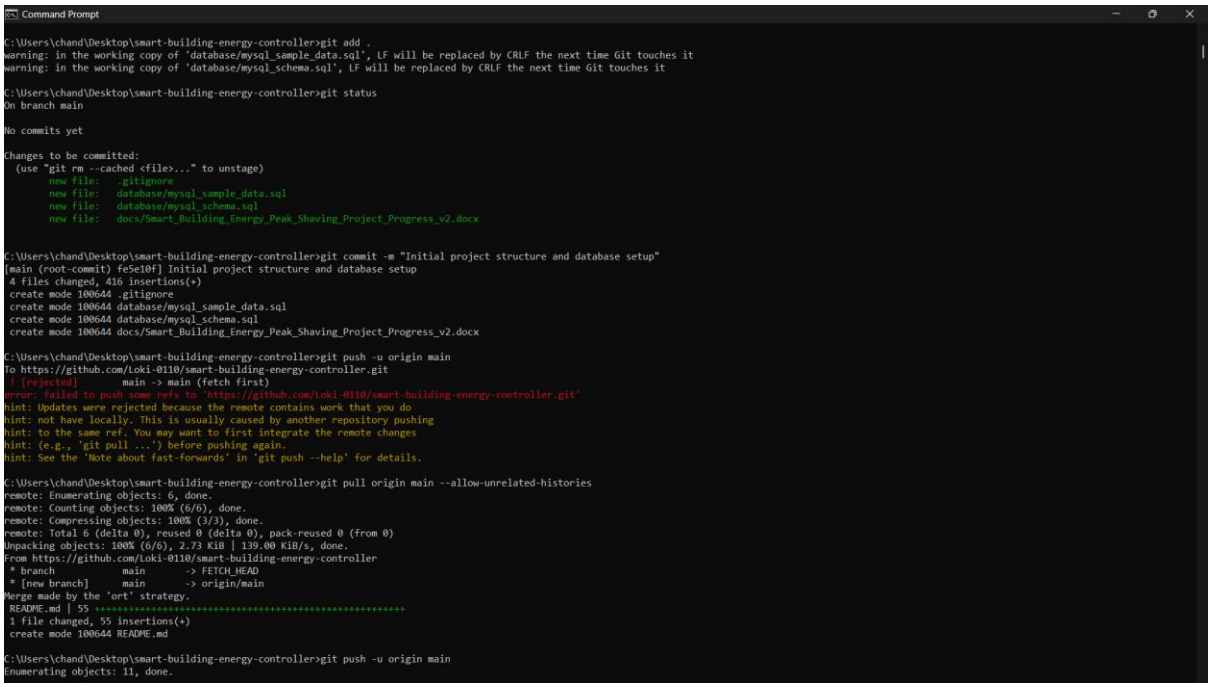


Figure 68. Git staging output showing project files added before a repository commit.

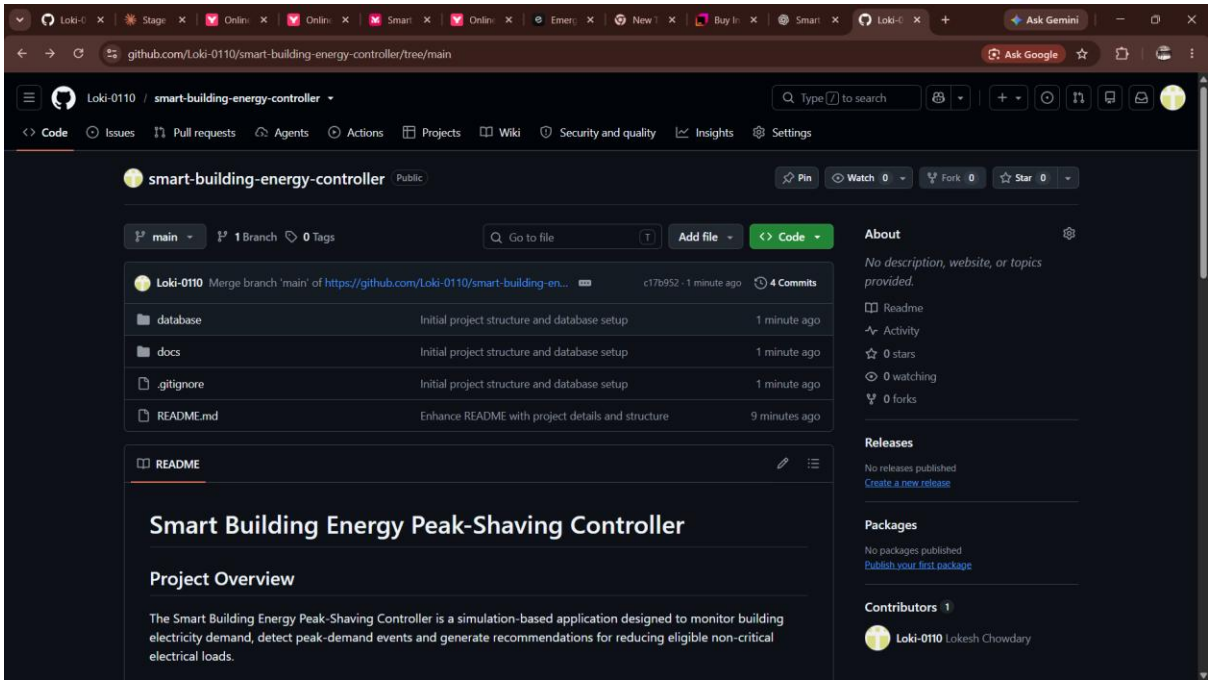


Figure 69. GitHub repository page showing the project source tree and repository navigation.

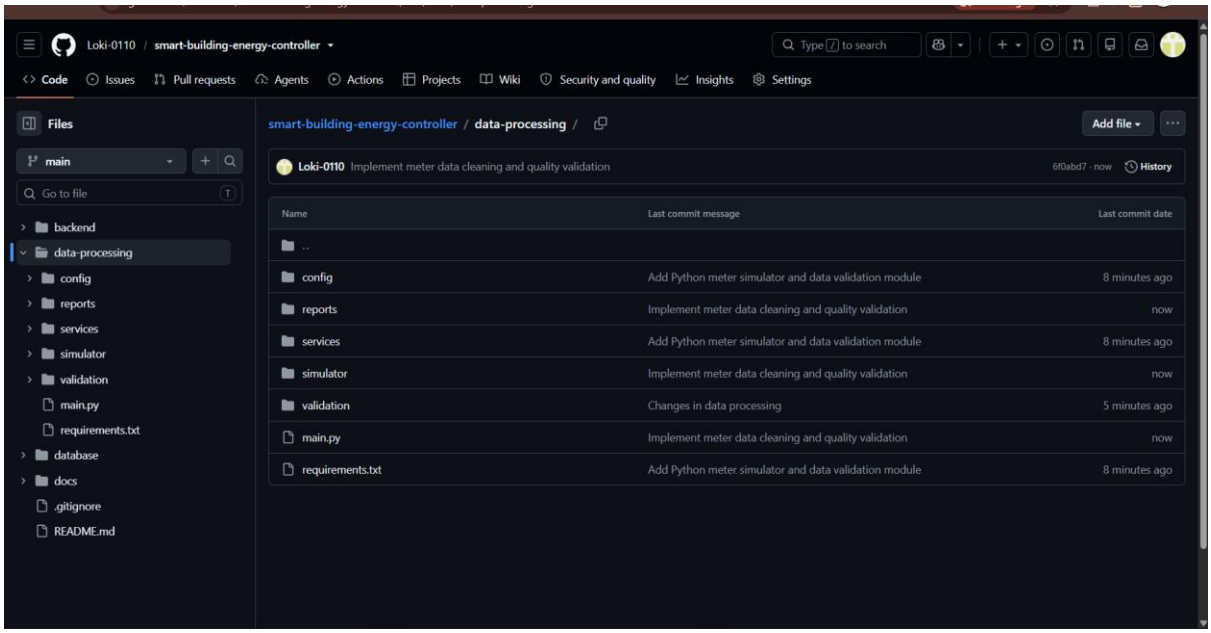


Figure 70. GitHub repository main branch page showing the committed project files.

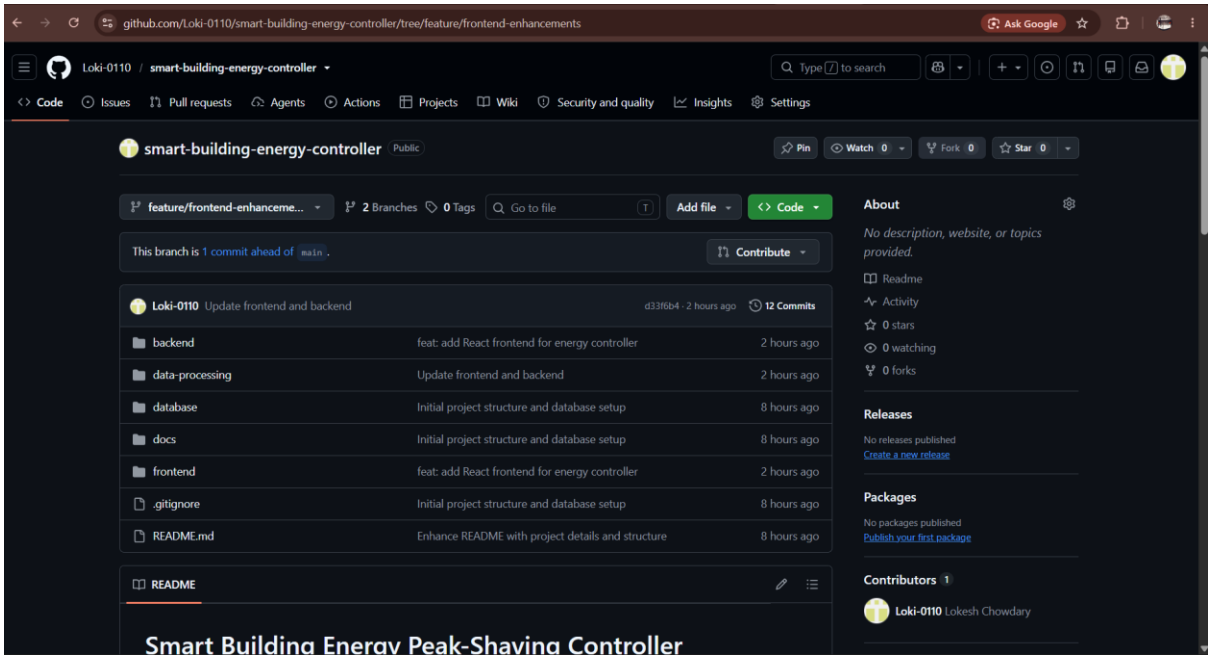


Figure 71. GitHub feature branch page showing frontend enhancement work.

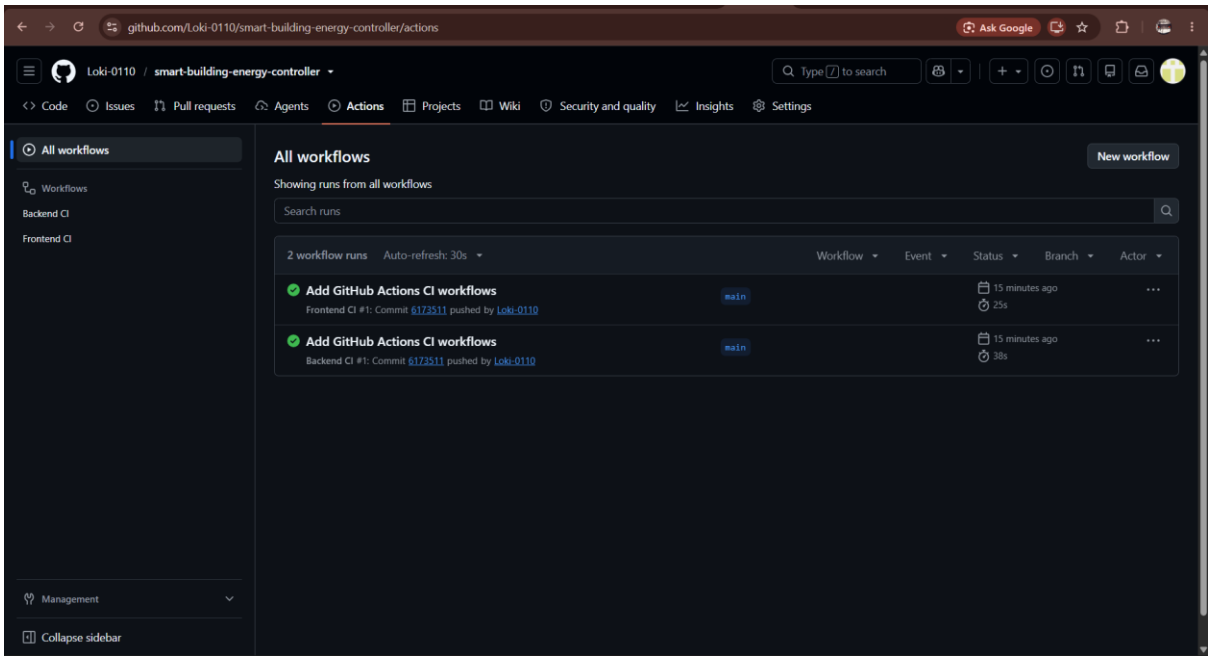


Figure 72. GitHub Actions page showing the repository workflow run history.

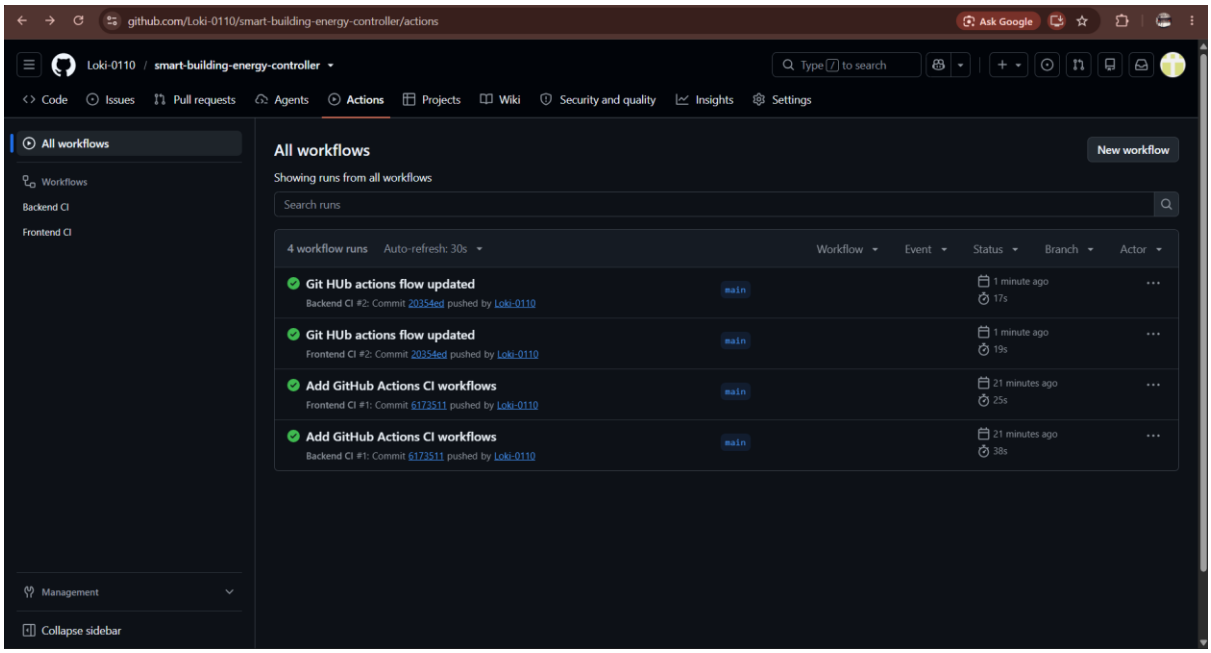


Figure 73. GitHub Actions workflow details showing a run and its job execution status.

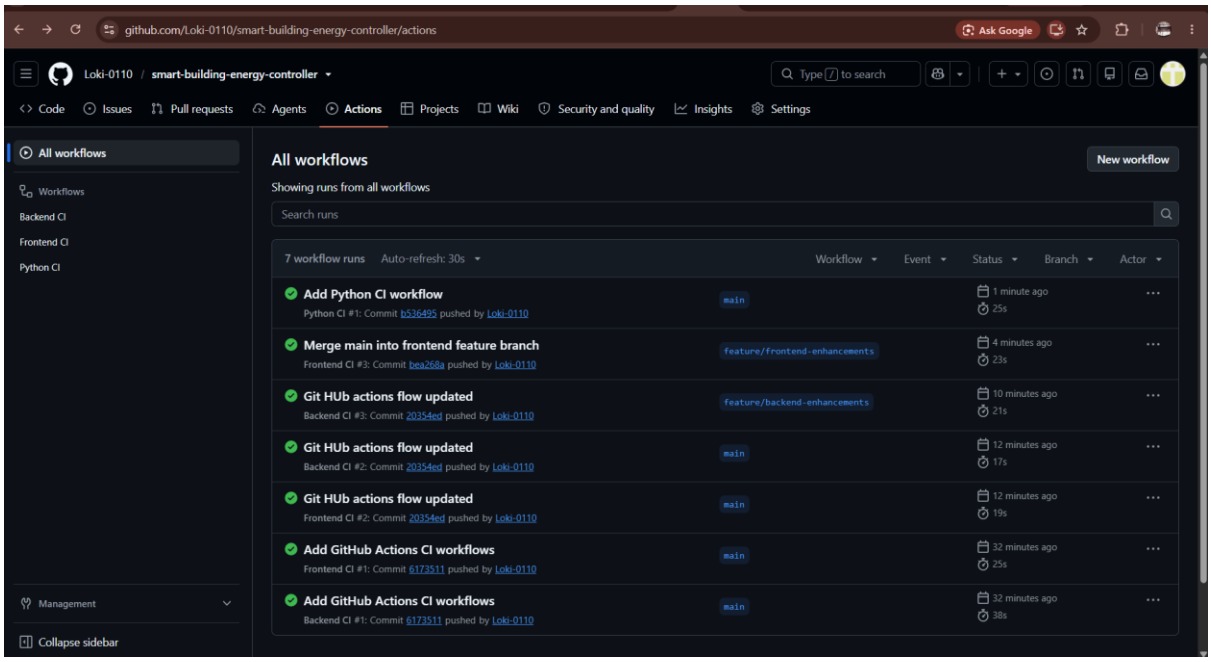


Figure 74. GitHub Actions job view showing the workflow steps and their results.

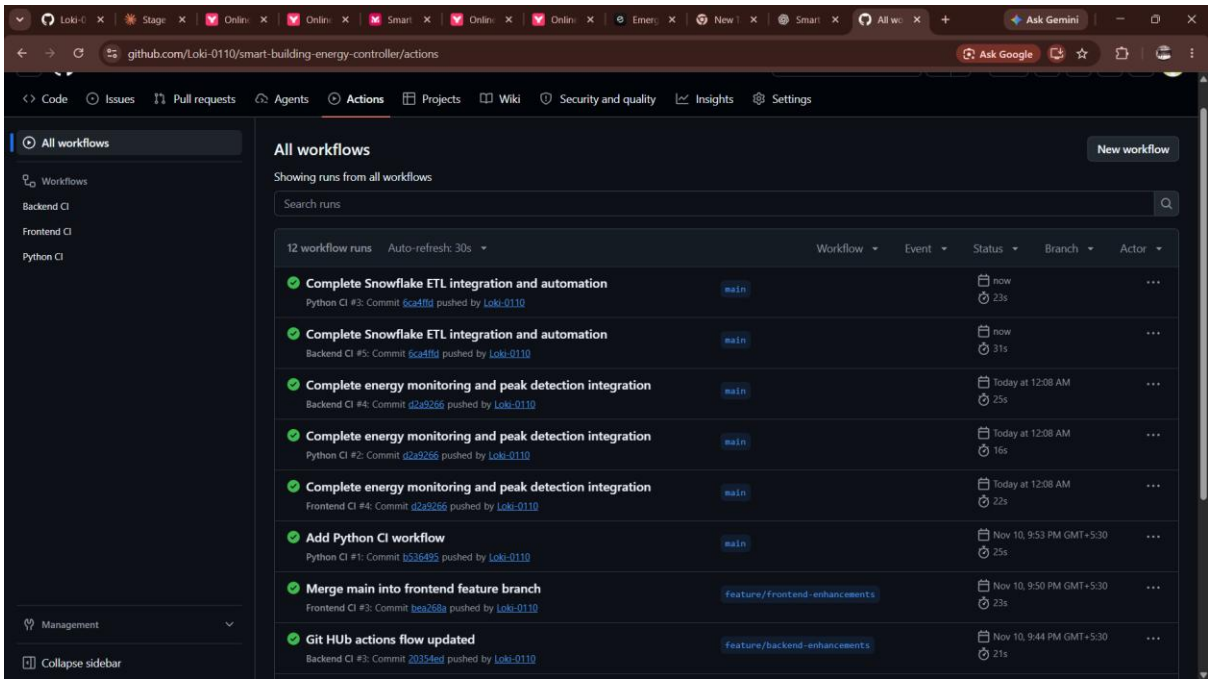


Figure 75. GitHub Actions workflow list showing the available CI/CD workflows and recent run state.

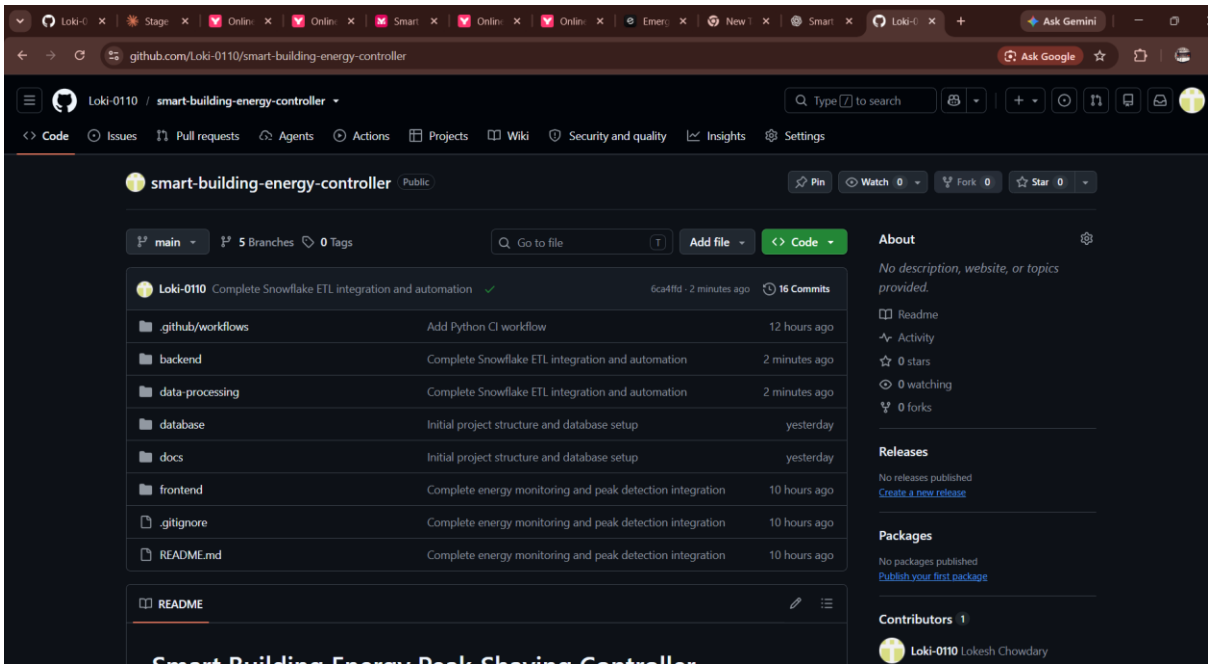


Figure 76. GitHub repository page showing the project files and branch context used for deployment.

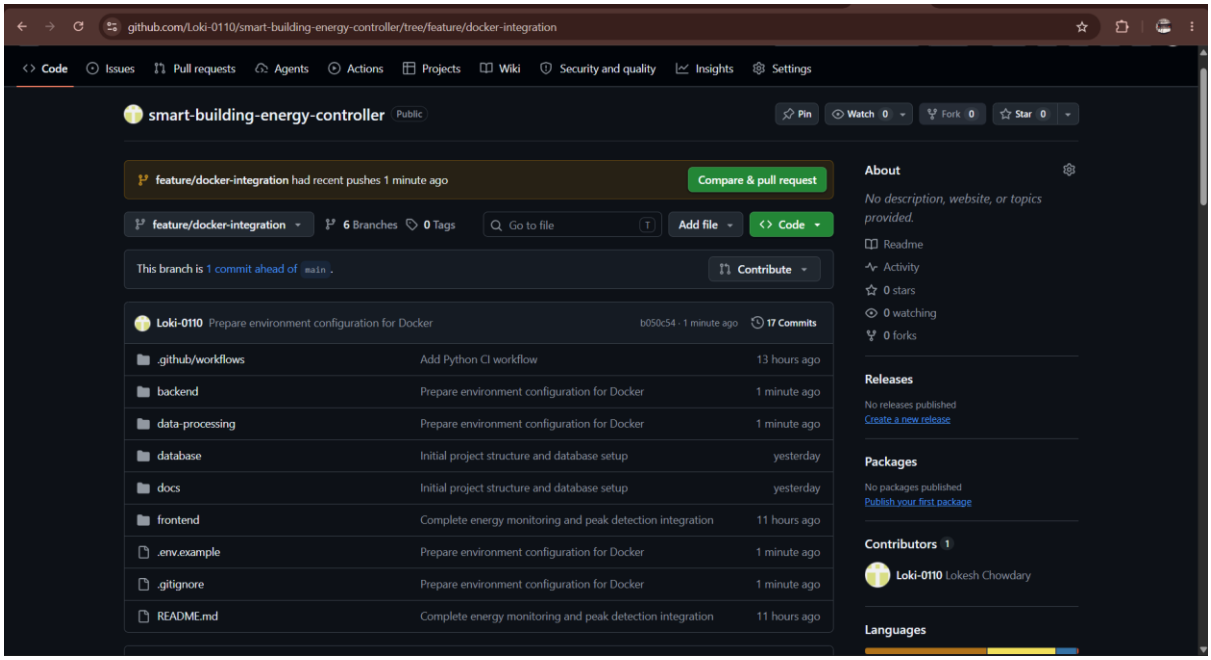


Figure 77. GitHub feature branch page showing Docker integration changes.

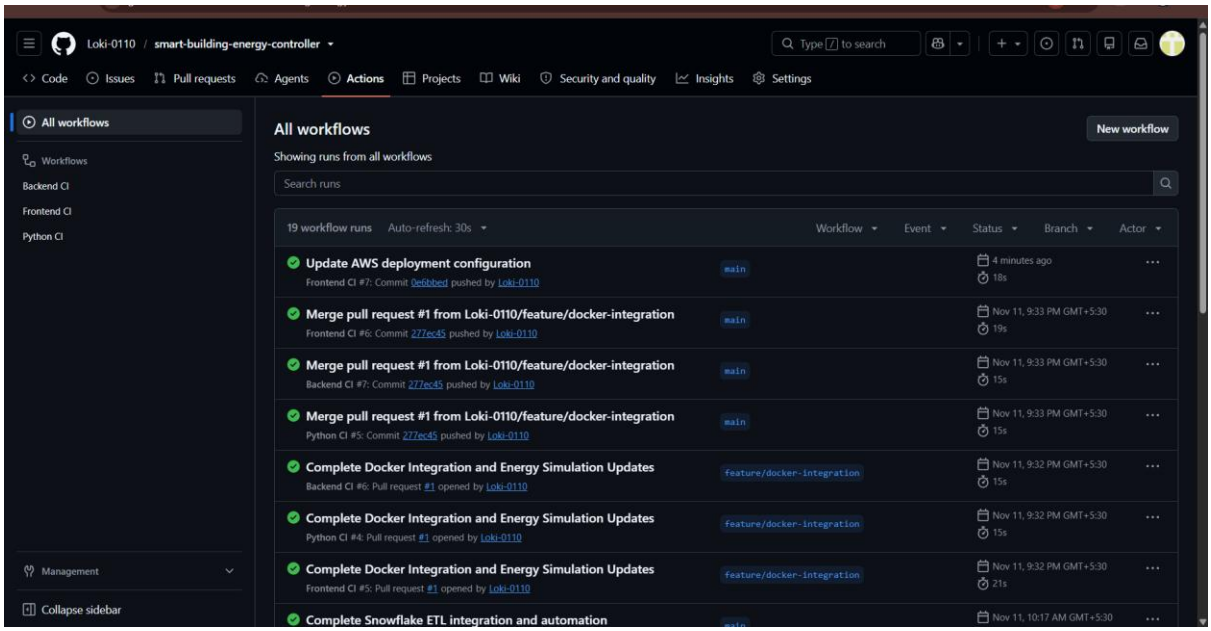


Figure 78. GitHub Actions run list showing the CI/CD workflow execution history.

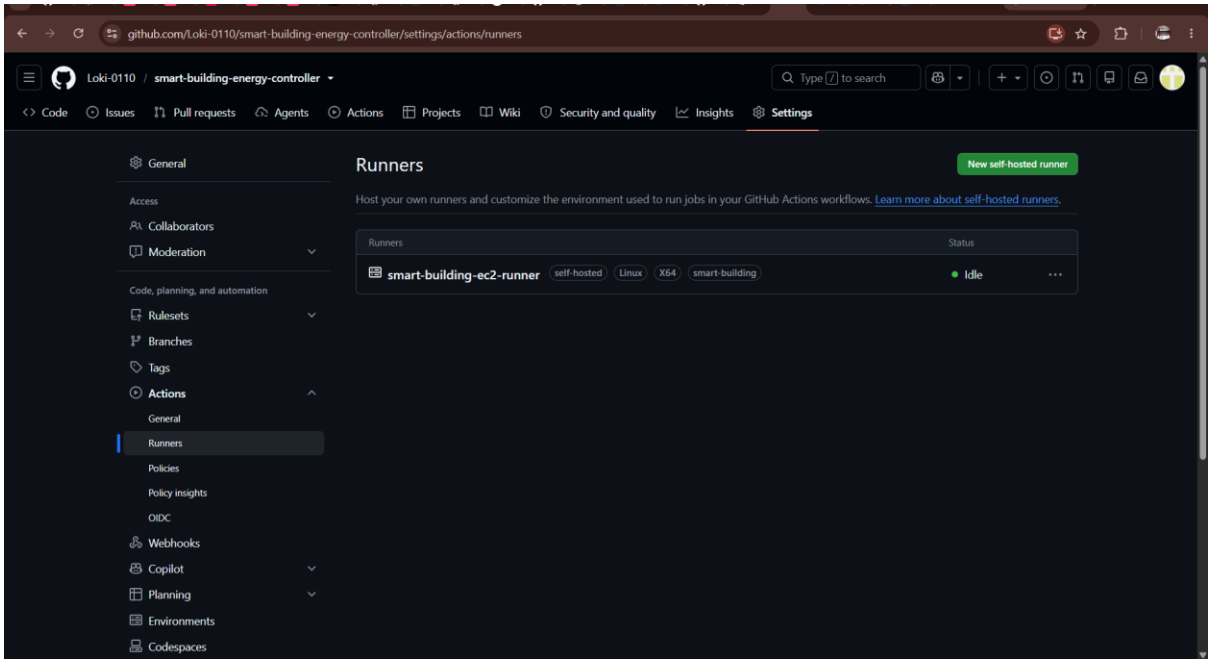


Figure 79. GitHub Actions self-hosted runner settings page showing the runner registration area.

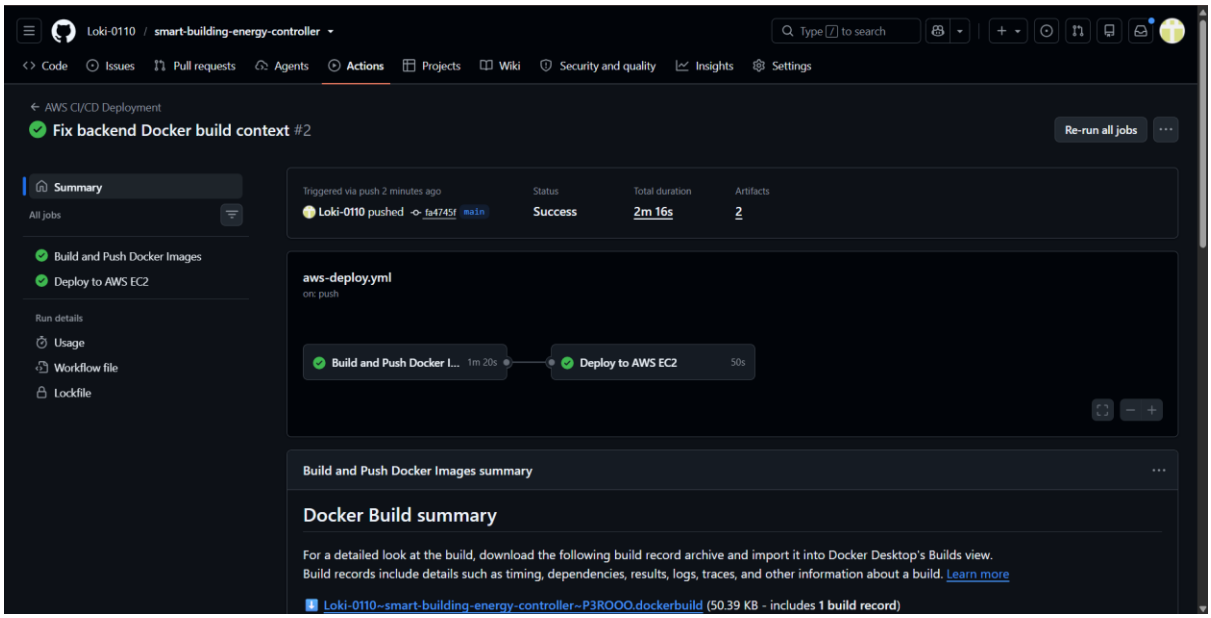


Figure 80. GitHub repository workflow evidence for the AWS EC2 deployment pipeline.

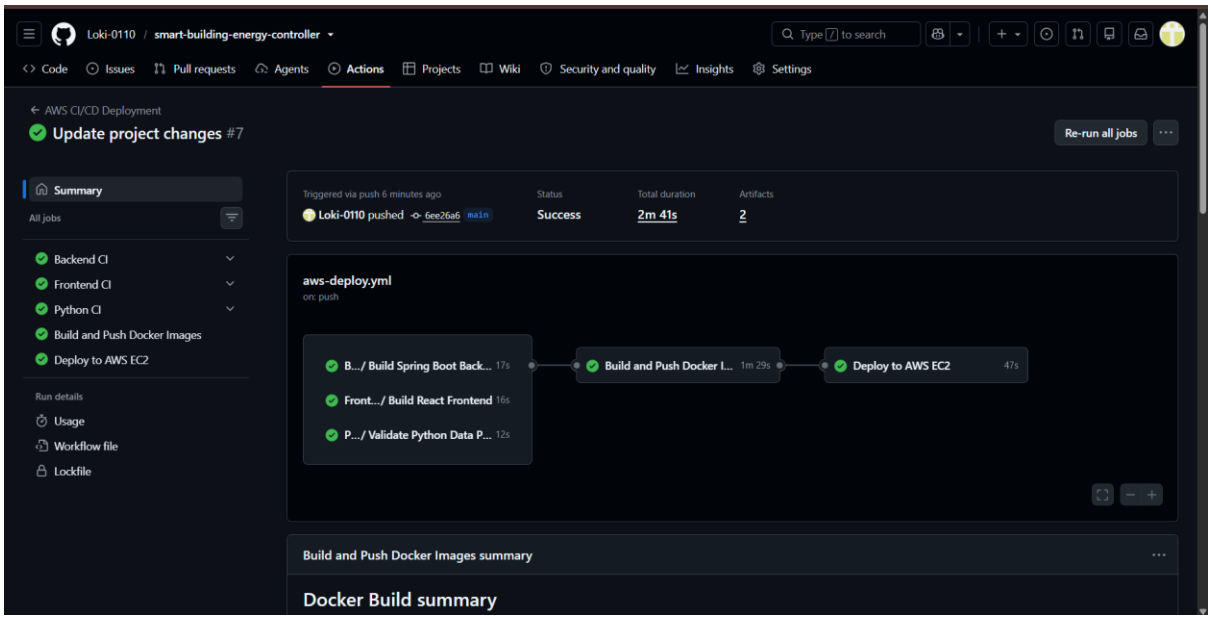


Figure 81. GitHub Actions workflow details showing the deployment-related update and execution status.

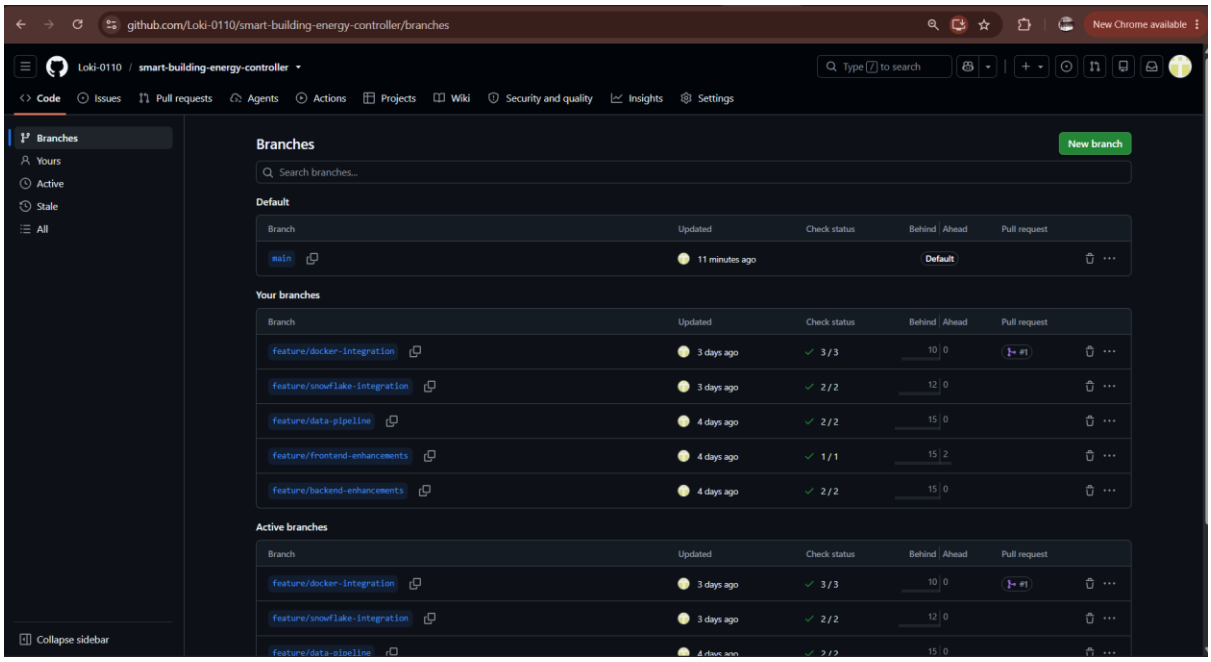


Figure 82. GitHub repository branch view showing the available project branches.

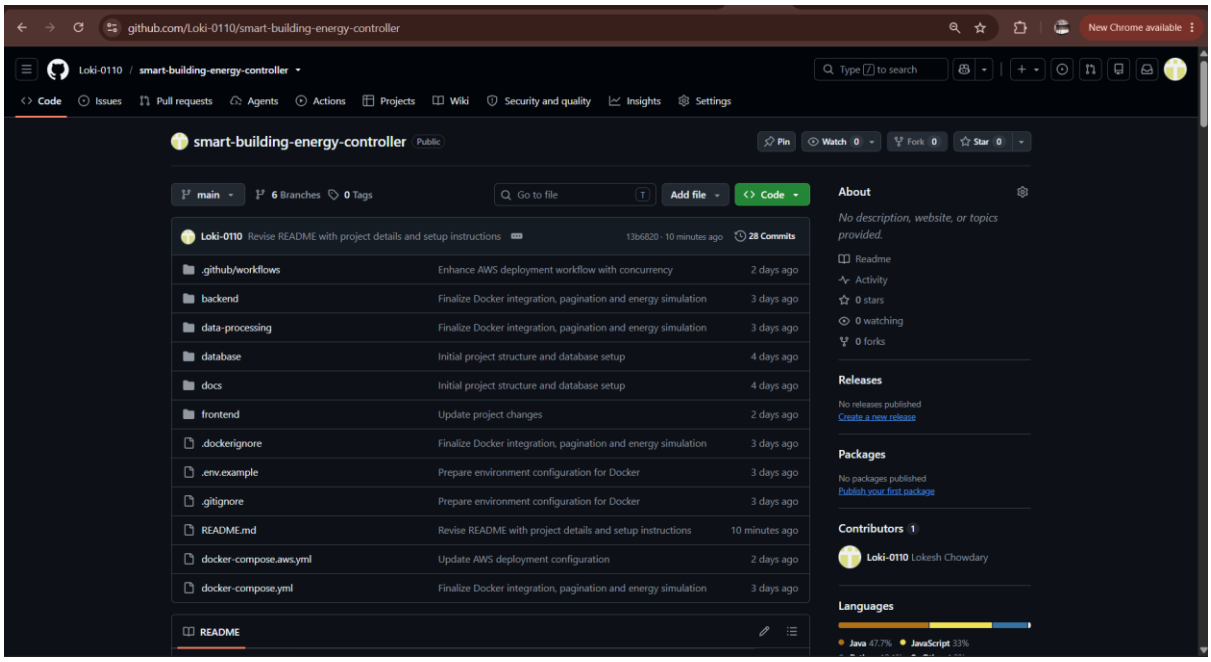


Figure 83. GitHub repository file view showing the project deployment configuration.

16.1 Deployment Flow

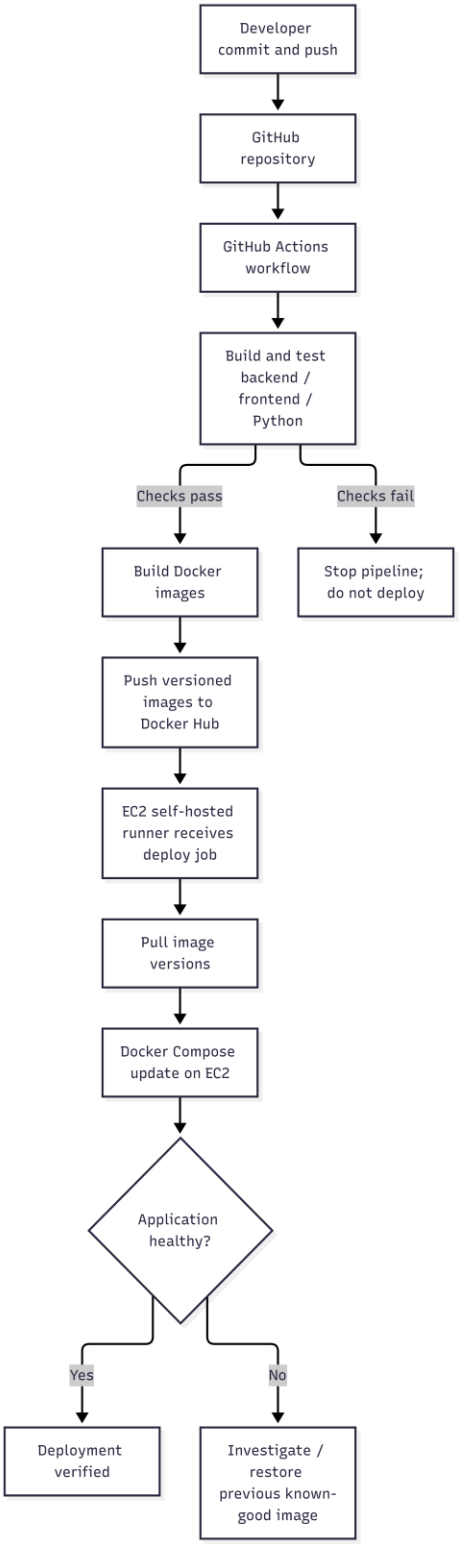


Figure 84. GitHub Actions deployment flow evidence showing the workflow execution associated with the EC2 deployment.

Verify each deployment:
1. Push the approved change to the branch monitored by the workflow.
2. Open GitHub → Actions and confirm the relevant workflow run succeeds.
3. On EC2, run docker compose -f docker-compose.aws.yml ps and inspect backend/frontend logs.
4. Open the EC2 public IP in a browser and test the application.
5. Record the workflow run and deployed image tags. A successful push alone does not prove deployment.

16.2 Secrets

- Add the registry secrets:
  1. Open the GitHub repository → Settings → Secrets and variables → Actions.
  2. Select New repository secret.
  3. Add DOCKERHUB_USERNAME and DOCKERHUB_TOKEN using the Docker Hub account values.
  4. Save the secrets and confirm the workflow references these exact secret names.
- Do not paste token values into documentation.
- Review obsolete EC2 SSH secrets and remove them if the current workflow no longer uses them.

17. AWS S3 Backup and Recovery

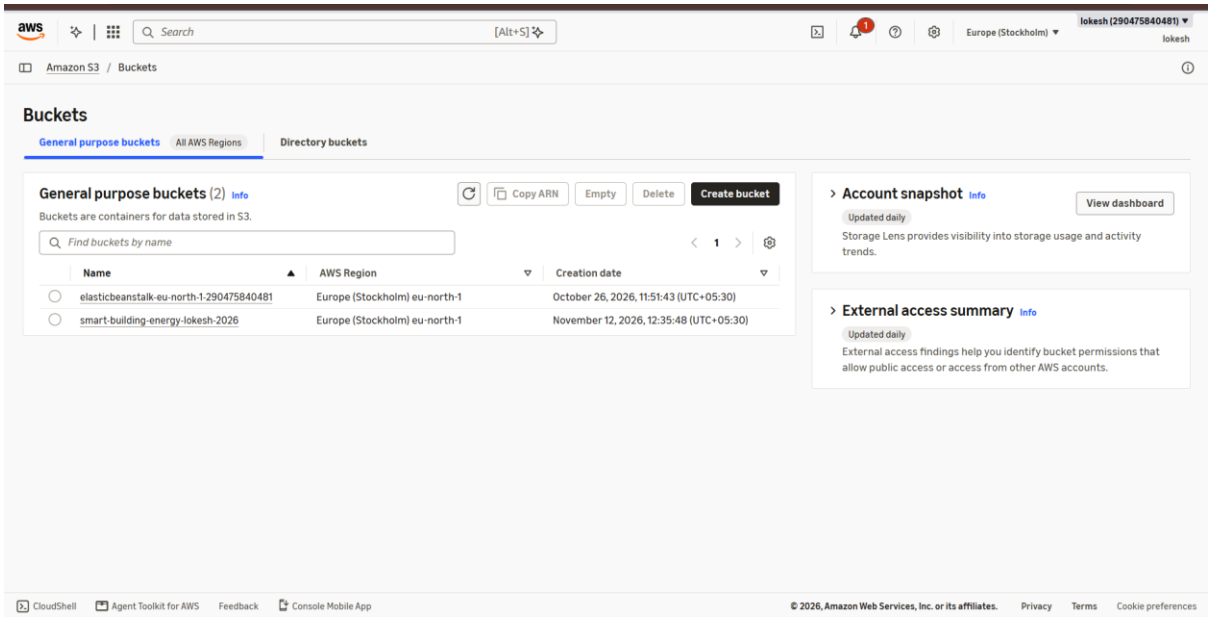


Figure 85. AWS S3 console showing the general-purpose project backup bucket list.

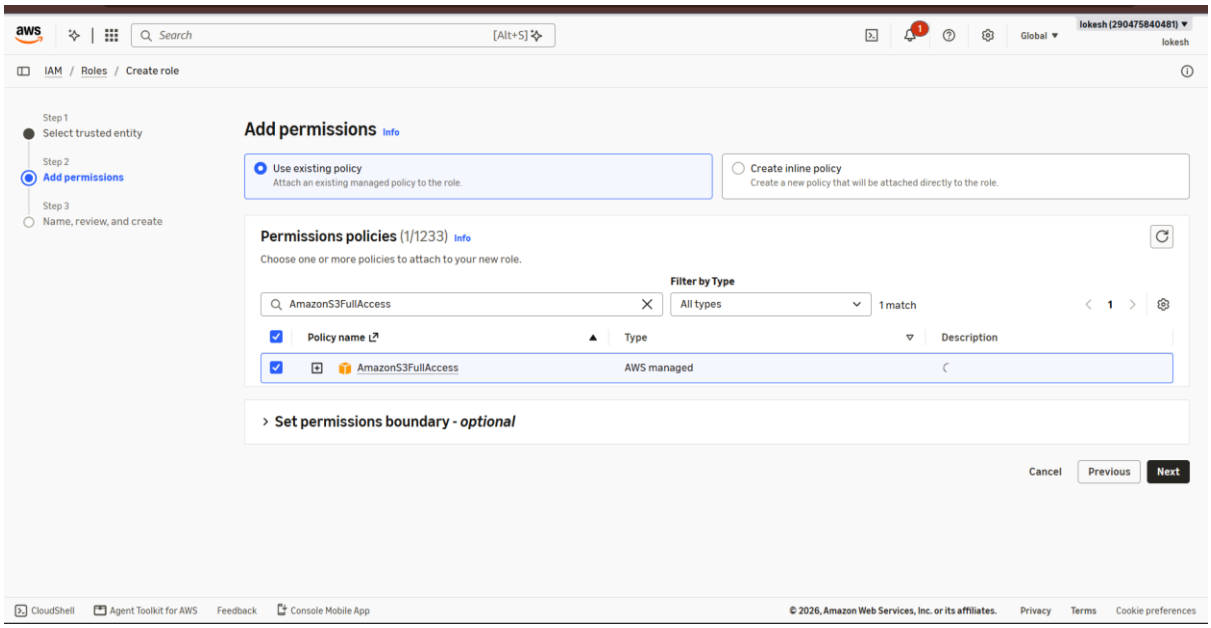


Figure 86. AWS IAM role creation page showing the EC2 trusted service/use case and permissions setup.

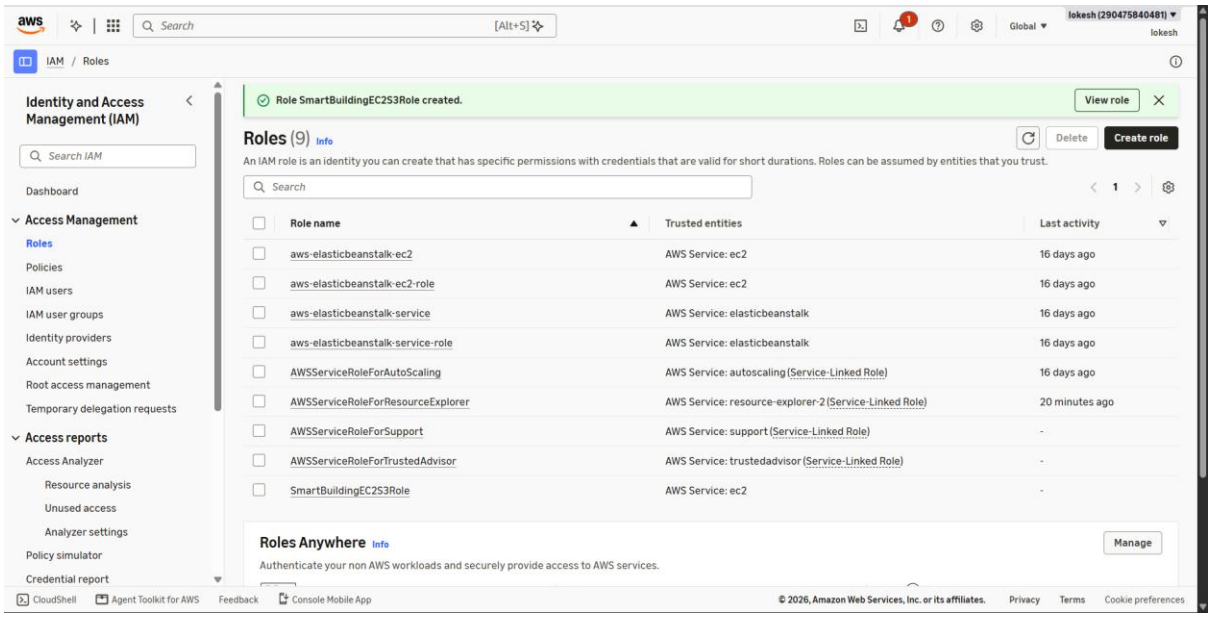


Figure 87. AWS IAM Roles list showing the SmartBuildingEC2S3Role role.

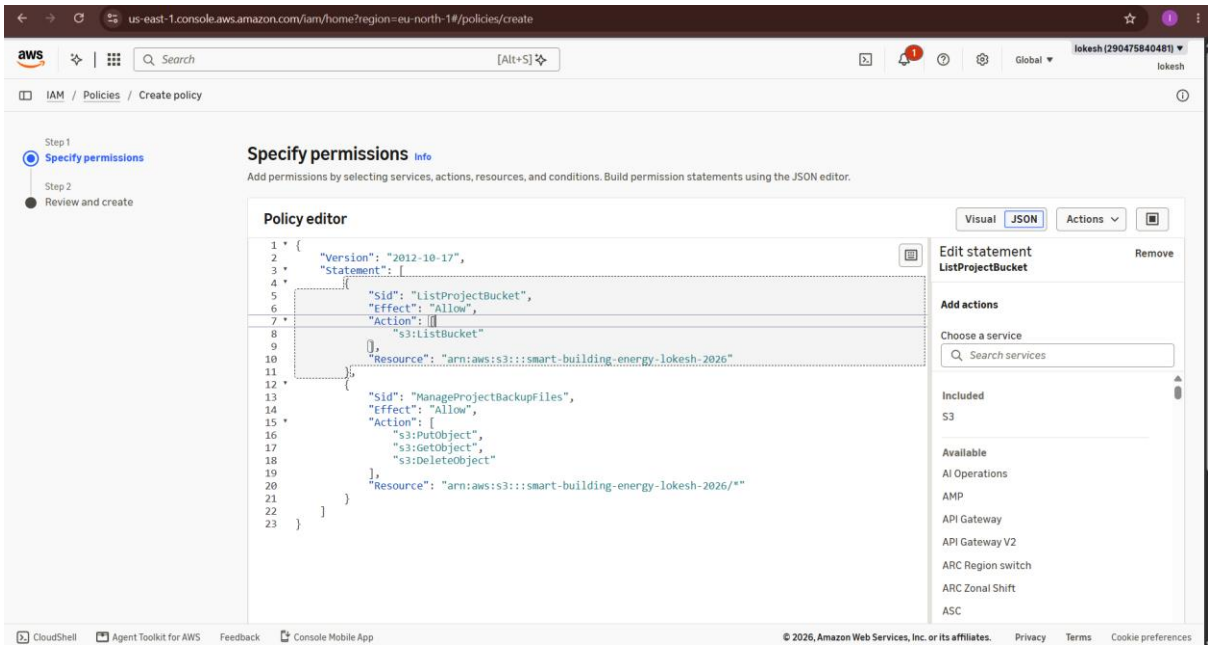


Figure 88. AWS IAM JSON policy editor showing SmartBuildingS3BackupPolicy and its S3 bucket/object actions.

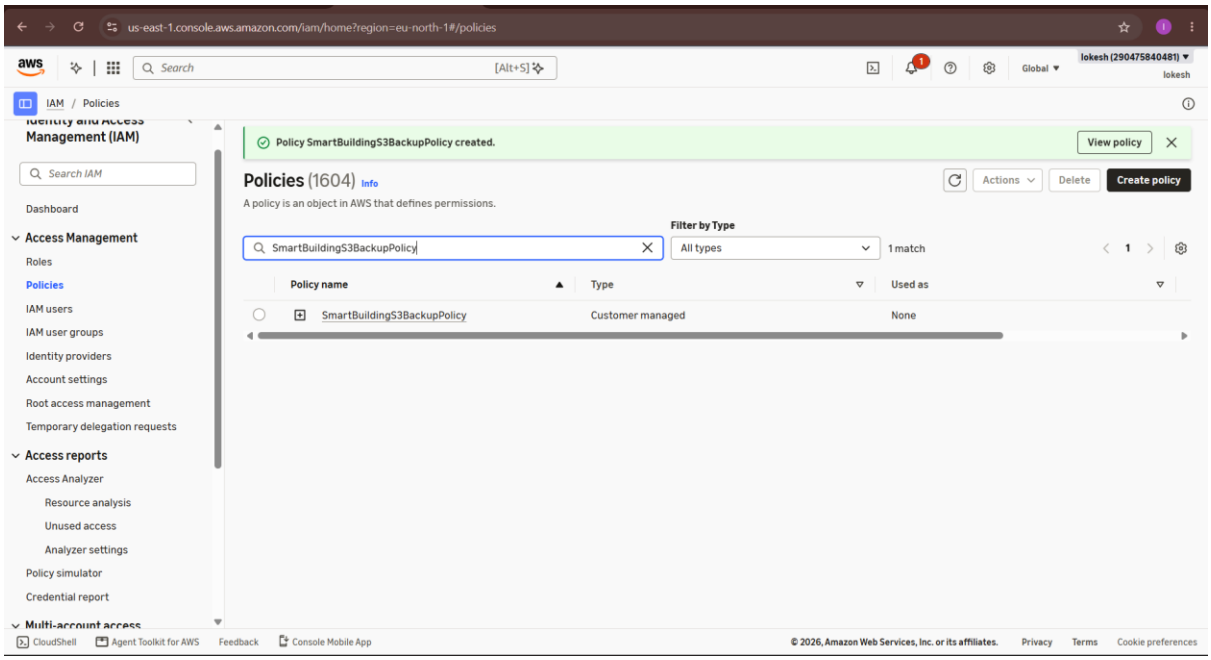


Figure 89. AWS IAM Policies page showing the customer-managed SmartBuildingS3BackupPolicy entry.

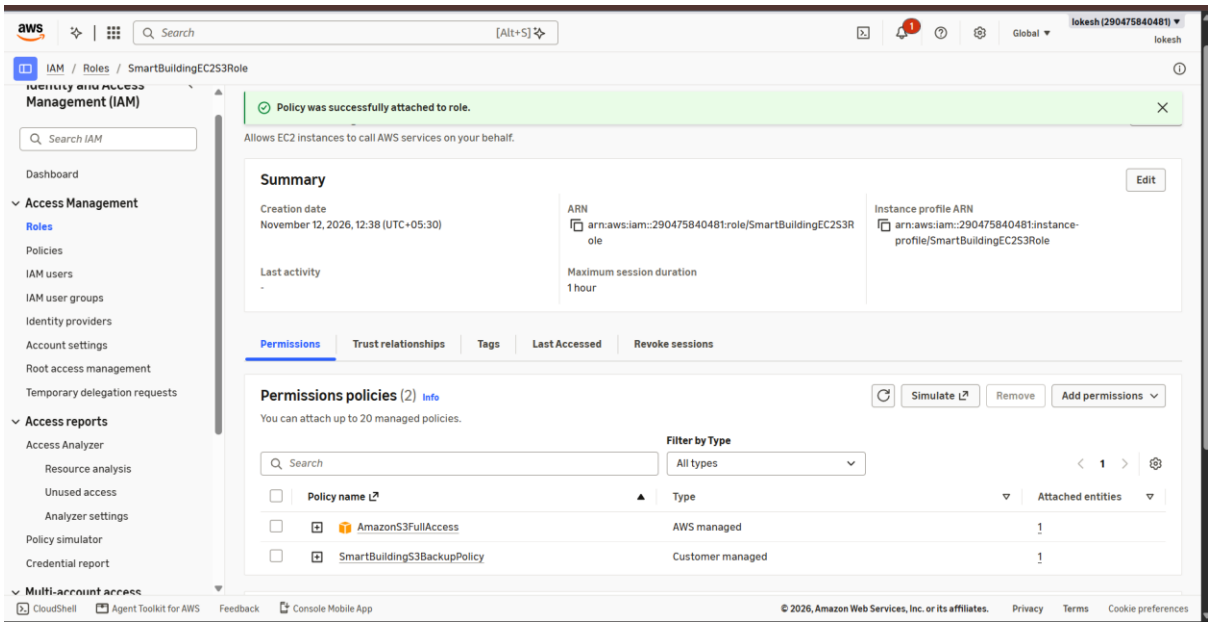


Figure 90. AWS IAM role summary showing SmartBuildingEC2S3Role and its attached permissions policies.

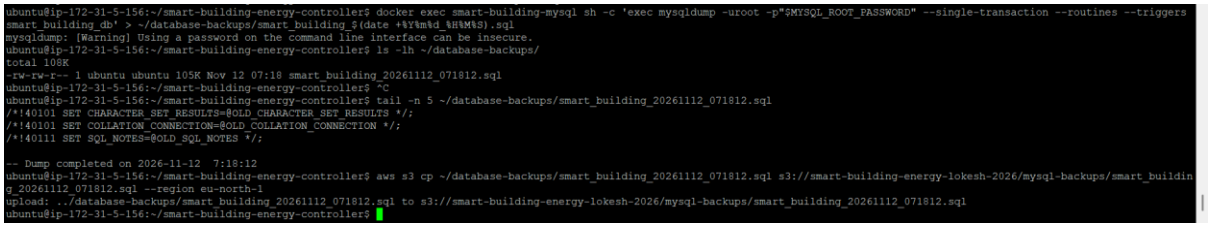


Figure 91. EC2 terminal showing the MySQL dump command that creates a database backup file for S3 storage.

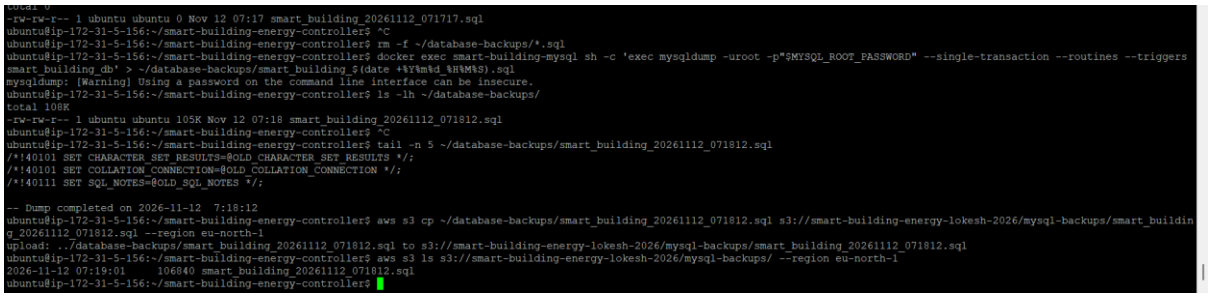


Figure 92. EC2 terminal showing the backup file verification and S3 upload/list commands.

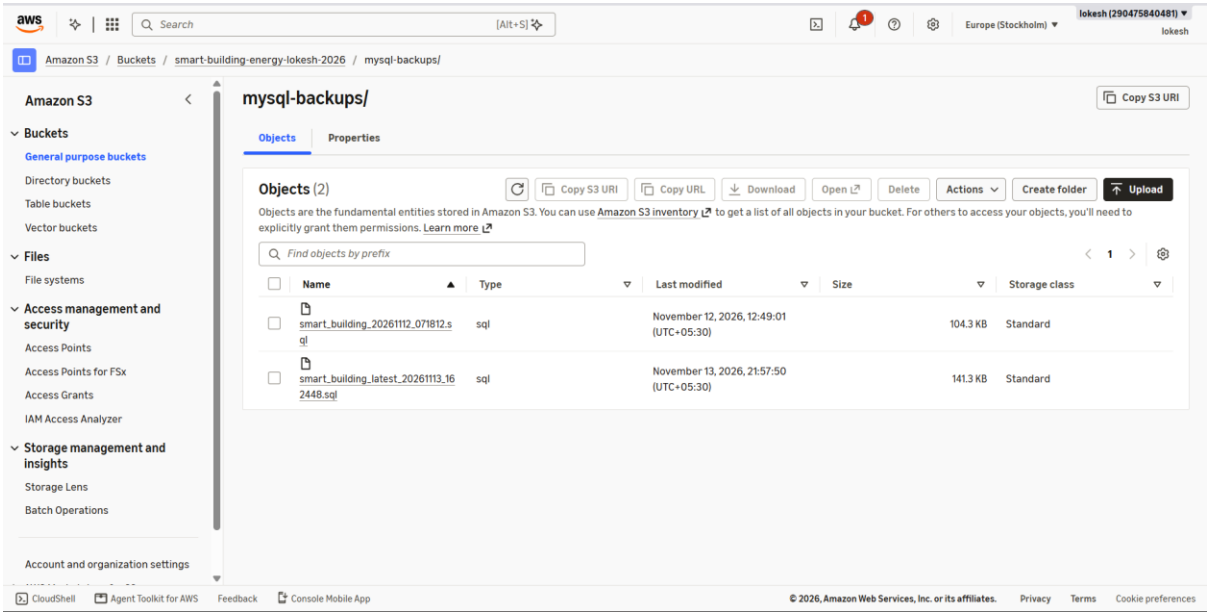


Figure 93. AWS S3 console showing the mysql-backups/ prefix and the uploaded SQL backup object.

17.1 Manual Backup Procedure

17.1.1 Create and check the backup
1. Connect to EC2 and move to the project directory:
cd /home/ubuntu/smart-building-energy-controller
2. Confirm the database container is running:
docker ps
3. Run the approved mysqldump command shown below and enter credentials only through the configured protected mechanism.
4. Check the dump file size and completion marker before uploading.

The backup is a manual database dump. Run it from the EC2 host only after confirming the database container and backup destination. Use an approved credential source; do not put a password directly in the command line or save it in shell history.

RUN IN PuTTY CONNECTED TO EC2 (EXAMPLE COMMAND — ADAPT THE CONTAINER AND FILE NAME AFTER VERIFYING THEM):

docker exec smart-building-mysql sh -c 'exec mysqldump --defaults-extra-file=/run/secrets/mysql-client.cnf smart_building_db' > smart_building_db_backup.sql

This is a template, not a guaranteed copy-and-run command. Confirm that the Compose service has a protected MySQL client option file at the stated path, that it is readable by the command, and that the database name is correct. If the project uses a different approved secret mechanism, use that mechanism instead. Do not place a database password directly in the command line or shell history.

```
mkdir -p ~/mysql-backups
# Create a timestamped dump from the MySQL container. Use a secure prompt/credential mechanism; do not paste
passwords into shell history.
docker exec -i smart-building-mysql mysqldump -u root -p --single-transaction --routines --triggers
smart_building_db > ~/mysql-backups/smart_building_YYYYMMDD_HHMMSS.sql
ls -lh ~/mysql-backups/
tail -n 5 ~/mysql-backups/<backup-file>.sql
```

Verify the dump before uploading:
1. Check that the file exists and is not empty: ls -lh smart_building_db_backup.sql
2. Check the dump's ending for the completion marker: tail -n 5 smart_building_db_backup.sql

3. If the command failed, the file is empty or the dump is incomplete, do not upload it. Re-run the backup after correcting the cause.

17.2 Upload and Verify

17.2.1 Upload the verified backup
1. Confirm the active EC2 identity:
aws sts get-caller-identity
2. Upload the SQL file:
aws s3 cp smart_building_db_backup.sql s3://smart-building-energy-lokesh-2026/mysql-backups/ --region eu-north-1
3. List the backup folder:
aws s3 ls s3://smart-building-energy-lokesh-2026/mysql-backups/ --region eu-north-1
4. In AWS Console → S3 → smart-building-energy-lokesh-2026 → mysql-backups/, confirm the uploaded object and size.

```
aws sts get-caller-identity
aws s3 cp ~/mysql-backups/<backup-file>.sql s3://smart-building-energy-lokesh-2026/mysql-backups/
aws s3 ls s3://smart-building-energy-lokesh-2026/mysql-backups/
```

Use the EC2 role and AWS CLI for uploads. Do not store access keys on EC2 or in source control.

IAM access design and verification

Policy actions:
1. s3:ListBucket — project bucket.
2. s3:PutObject — mysql-backups/*.
3. s3:GetObject — only if restore/download is required.
4. s3:DeleteObject — only if approved cleanup is required.

Resource ARNs:
• Bucket: arn:aws:s3:::smart-building-energy-lokesh-2026
• Objects: arn:aws:s3:::smart-building-energy-lokesh-2026/mysql-backups/*

Screenshot note: current object scope is /* and AmazonS3FullAccess is attached. Narrow the object scope and remove broad access after testing.

Verify:
1. IAM → Roles → SmartBuildingEC2S3Role → Permissions: confirm the custom policy.
2. Confirm object ARN ends with /mysql-backups/*.
3. Remove AmazonS3FullAccess after testing.
4. EC2 → Instances → Security: confirm the role is attached.

Run on EC2:
aws sts get-caller-identity
aws s3 ls s3://smart-building-energy-lokesh-2026/mysql-backups/ --region eu-north-1

Upload a test file, confirm it appears in the prefix, then upload the SQL backup and verify its name and size.

17.3 Restore Principles

```
# Restore only after confirming the target database and backup file. This overwrites data in the target schema
if objects conflict.
# Example pattern; execute only with a reviewed recovery plan:
mysql -u root -p smart_building_db < backup.sql
```

A restore must be tested in a non-production database first. Confirm schema version, row counts and application startup after restore. Do not run a restore command against the live database without explicit approval and a second backup.

18. Environment Configuration

Configuration values differ between local development and the EC2 deployment. Keep environment-specific values outside source control and document variable names rather than real values.

- Backend: SERVER_PORT, DB_URL, DB_USERNAME, DB_PASSWORD and FRONTEND_ORIGIN.
- Frontend: VITE_API_BASE_URL, set to the intended backend API base URL for the environment.
- Python ingestion/ETL: API base URL, MySQL connection settings and Snowflake connection settings should be supplied through the configured environment or protected secret store.
- Docker Compose: confirm service names, image tags, networks, health checks, restart policies and environment variable references in the active Compose file.
- Never commit .env files, database passwords, Snowflake credentials, AWS access keys, SSH private keys or Docker Hub tokens.

Variable	Purpose	Local default / example
DB_URL	JDBC URL for operational MySQL.	jdbc:mysql://localhost:3306/smart_building_db
DB_USERNAME	Database username.	root (local development only)
DB_PASSWORD	Database password.	Set privately in environment; never commit.
FRONTEND_ORIGIN	Allowed browser origin for CORS.	http://localhost:5173
SERVER_PORT	Backend listening port.	8082
VITE_API_BASE_URL	Frontend API base URL.	http://localhost:8082/api

```
PowerShell (local backend terminal):
$env:DB_USERNAME="root"
$env:DB_PASSWORD="<your-local-password>"
$env:DB_URL="jdbc:mysql://localhost:3306/smart_building_db"
$env:FRONTEND_ORIGIN="http://localhost:5173"
```

Do not share actual passwords in screenshots, support messages or GitHub. The project history included an exposed database credential; rotate it and remove hard-coded defaults. Use separate credentials for local and deployed environments.

19. Local Setup for a New Developer

19.0 Get the source code
1. Install Git and confirm access to the project repository.
2. Clone https://github.com/Loki-0110/smart-building-energy-controller.git.
3. Open the cloned project folder in the terminal or IDE.
4. Read the README and this guide, then follow the setup order below.

19.1 Prerequisites

- Windows 10/11
- Java 21 JDK
- Maven 3.9 or later
- Node.js 20 or later and npm
- Python 3.12
- MySQL 8 or Docker Desktop with MySQL container
- Git and access to the repository

19.2 Database Preparation

Prepare the local database:
1. Start MySQL 8 locally or start the MySQL service defined for local Docker development.
2. Create or obtain an authorized development database named smart_building_db.
3. Apply the project's reviewed schema and migration SQL in order; check database/PEAK_LIFECYCLE_MIGRATION.sql where applicable.
4. Load the required development seed data and confirm the expected tables exist.
5. Compare the schema with the backend entities before starting Spring Boot. Hibernate uses ddl-auto=validate and will fail if tables or columns are missing or mismatched.
6. Do not use production credentials or a production database for local setup.

19.3 Start Backend

PowerShell — Terminal 1:

cd backend
$env:DB_USERNAME="root"
$env:DB_PASSWORD="<local-password>"
$env:DB_URL="jdbc:mysql://localhost:3306/smart_building_db"
$env:FRONTEND_ORIGIN="http://localhost:5173"
mvn spring-boot:run
Expected API base URL: http://localhost:8082/api.

```
PowerShell – Terminal 1
cd backend
$env:DB_USERNAME="root"
$env:DB_PASSWORD="<local-password>"
mvn spring-boot:run
```

Expected base URL: http://localhost:8082/api. Check startup logs for datasource connectivity and Hibernate schema validation errors.

19.4 Start Frontend

PowerShell — Terminal 2:

cd frontend
npm ci
Copy-Item .env.example .env
Set VITE_API_BASE_URL=http://localhost:8082/api in frontend/.env
npm run dev
Open http://localhost:5173.

```
PowerShell – Terminal 2
cd frontend
npm ci
npm run dev
```

Open http://localhost:5173. Confirm browser network requests target the local backend URL and are not blocked by CORS.

19.5 Start Python Simulator

PowerShell — Terminal 3:
cd data-processing
py -3.12 -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python main.py

19.5.1 Run Analytics ETL
From data-processing, activate the virtual environment and set the MySQL/Snowflake connection values using the configuration names already read by the ETL code. Then run:
python run_etl.py
Check the actual ETL entry-point filename in the repository before running; if it differs, use the implemented entry point. Compare source and Snowflake counts for the same extraction window.

```
PowerShell – Terminal 3
cd data-processing
py -3.12 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python main.py
```

19.6 Verify the Workflow

22. Confirm at least one building exists.
23. Confirm equipment and peak configuration are present.
24. Run Python ingestion and inspect generated/valid/rejected/submitted counts.
25. Query meter readings through the UI or API.
26. Trigger peak detection or wait for the scheduler.
27. Review the event and generated schedule recommendations.
28. Approve/reject a PENDING recommendation and confirm its state changes.
29. Confirm no physical equipment control is implied or performed.

Refresh the Power BI dataset after the Snowflake load and confirm the report timestamp and key totals reflect the same extraction window.

20. Troubleshooting Guide

Use the following checks in order: confirm the failing component, inspect its logs, verify the configuration and network path, then retry with a small controlled test. Avoid destructive resets as a first response.

- For every incident, record the time, environment, affected component, relevant log excerpt (with secrets removed), root cause, corrective action and the verification performed after the fix.

Symptom	Likely cause	Action
Backend fails to connect to MySQL	Wrong DB_URL, username/password, MySQL not running, port mismatch.	Check Docker container, port mapping and local environment variables; test MySQL login.
Hibernate schema validation error	Database schema does not match entity mappings.	Inspect the exact missing/mismatched column/table; apply a reviewed migration, not ddl-auto update.
Frontend cannot reach API	Wrong VITE_API_BASE_URL, backend stopped, CORS mismatch.	Check browser Network tab, backend port and FRONTEND_ORIGIN.
Python reports no buildings	No building records or API URL/configuration issue.	Check GET /api/buildings and Python environment configuration.
Duplicate reading issue	Same building/timestamp submitted again.	Confirm idempotent handling and unique constraint; inspect stored row.
Peak not detected	Demand below threshold, reading stale, no active config.	Check latest reading timestamp/value and active peak configuration.
Recommendation not created	Equipment protected, occupancy unknown, comfort rule fails, or no eligible equipment.	Review eligibility reasons and equipment flags.
Approval returns conflict	Recommendation is not PENDING.	Fetch current schedule state before action.
EC2 website unavailable	Instance stopped, frontend container down, security group or IP changed.	Check EC2 state, public IP, port 80 ingress and docker ps.
GitHub deployment not running	Workflow failure or self-hosted runner offline.	Inspect Actions logs and systemctl status for runner service.
S3 upload denied	Missing IAM permissions or wrong bucket/region.	Run aws sts get-caller-identity; verify role and bucket permissions.
Power BI values stale	Import model not refreshed.	Refresh dataset and reconcile against Snowflake counts.

21. Testing and Verification

The following release checklist covers the main user journey and the boundaries between components. Mark an item complete only after running it in the target environment and retaining evidence.

- Create or retrieve a building and confirm it is visible through the API and frontend.
- Register equipment and verify criticality, controllability and rated-power fields.
- Configure occupancy windows, comfort constraints and the active demand threshold.
- Run the Python simulator and record generated, accepted, rejected/skipped and failed submission counts.
- Confirm accepted readings appear in MySQL and repeated building/timestamp submissions do not create duplicates.
- Test no-reading and stale-reading cases; confirm the scheduler exits safely without creating an invalid recommendation.
- Test demand below and above the configured threshold and verify event handling.
- Test critical, non-controllable, occupied and comfort-constrained equipment; verify protected loads are not recommended.
- Approve and reject a PENDING recommendation; verify invalid state transitions are rejected.
- Verify the UI reflects the updated event and recommendation status.
- Run the analytics ETL and reconcile a defined source window with Snowflake target records.
- Refresh Power BI and record the refresh time and key KPI reconciliation.
- Build and run the Docker Compose services locally; verify health and logs.

- For a release deployment, verify the workflow result, EC2 containers and browser access.
- Create a manual database backup and test restore in a separate non-production database.

For each test, capture: test ID, date/time, environment, input or setup, expected result, actual result, pass/fail and evidence reference. Automated rollback, production-grade authentication, scheduled backups and real equipment control are not part of the verified scope described in this guide.

Use a repeatable verification checklist for each release. Attach evidence (test output, API response, screenshots or workflow run) and record date, environment and software version.

Test area	Test case	Expected result
API	Create/list/update building and equipment.	Valid requests succeed; invalid data returns validation errors.
Meter ingestion	Submit a new reading and submit same building/timestamp again.	One logical reading is stored; retry is idempotent.
Peak detection	Demand below and above active threshold.	No peak action below threshold; event detection above threshold.
Critical protection	Include critical equipment in candidates.	No reduction recommendation for critical equipment.
Controllability	Include non-controllable equipment.	Equipment excluded with reason.
Occupancy	Unknown or occupied area.	Conservative rule blocks unsafe reduction.
Approval	Approve/reject pending and non-pending schedules.	Pending transition succeeds; invalid state returns conflict.
Python	Run simulator with existing records.	New timestamps follow latest stored record; rejected report is generated when needed.
Cloud	Verify all Compose services after deployment.	Expected containers running and frontend reachable.
Backup	Create, upload and list SQL dump in S3.	Non-empty completed dump is present in private bucket.
Analytics	Compare source and destination record counts.	Reconciliation is recorded for the same extraction window.
Power BI	Refresh report after Snowflake load.	Visuals show refreshed values and correct filters.

Do not mark Maven build, npm production build, automated rollback, backup restore, or security controls as verified unless a recent test has been executed and evidence captured.

22. Security, Risks and Known Limitations

22.1 Security Baseline

- Keep secrets outside source control and rotate exposed credentials.
- Use least-privilege IAM permissions for S3 and deployment.
- Restrict EC2 security group ingress to required ports and trusted source IPs.
- Do not expose MySQL to the public internet.
- Use HTTPS/TLS for public production traffic.
- Implement Spring Security authentication and role-based authorization before public use.
- Treat the APP_USERS user-ID check as a workflow validation, not authentication.
- Keep audit records for approvals and overrides.

22.2 Known Limitations

- A formal end-to-end automated test suite and coverage report are not evidenced in this handover; maintain the release checklist until such tests are added.
- Retry/backoff and dead-letter handling for analytics ETL failures must be confirmed in the current ETL code before being described as implemented.

- A tested database restore procedure is not evidenced merely by a successful backup upload; perform and record a separate restore test.
- Meter data is synthetic rather than supplied by real meters.
- Peak detection is threshold-based; predictive forecasting is not established by the supplied implementation.
- Recommendations and expected reduction are simulated; physical equipment is not controlled.
- The current user-ID check is not a secure authentication mechanism.
- MySQL schema compatibility must be maintained manually through reviewed migrations.
- S3 database backups are documented as manual; do not claim scheduled backup automation.
- Power BI Import mode requires refresh and may show older values between refreshes.
- A single EC2 host is a simplified deployment and is not highly available.
- Distributed locking across multiple backend instances is not established by the current in-process scheduler guard.

Appendix A – Glossary

The following terms are used throughout this implementation and operations guide.

Term	Meaning in this project
API	Application Programming Interface; a defined way for software components to exchange requests and responses.
AWS	Amazon Web Services, the cloud platform hosting the project environment.
CI/CD	Continuous Integration and Continuous Delivery/Deployment; automated build, test and release workflow.
Compose	Docker Compose; a tool for defining and running multi-container applications.
Container	An isolated runtime package containing an application and its required dependencies.
EC2	Amazon Elastic Compute Cloud; a virtual server used to host the deployed application.
ETL	Extract, Transform and Load; the process used to move and prepare operational data for analytics.
IAM	Identity and Access Management; AWS controls for identities, roles and permissions.
Idempotency	A property that makes repeating the same operation safe without creating duplicate effects.
Meter simulator	The Python component that generates synthetic meter readings for the application.
MySQL	The relational database used as the operational source of truth.
Nginx	A web server used by the frontend container to serve the built React application.
Peak event	A recorded event created when valid demand exceeds the configured building threshold.
Peak shaving	Reducing or shifting selected non-critical demand during a high-demand period.
Power BI	The reporting and visualization tool used to present analytical data.
REST API	An HTTP-based interface used by the frontend and Python simulator to communicate with Spring Boot.
S3	Amazon Simple Storage Service; used here for private storage of manual SQL backup files.
Snowflake	The cloud data warehouse used for downstream analytical datasets.
Synthetic data	Generated sample data that imitates readings but is not measured from real building meters.
Vite	The frontend development/build tool used with React.

Appendix B – Development Notes and Problem-Solving

The project was developed in multiple stages, covering application architecture, database design, backend APIs, frontend development, Python data processing, analytics and deployment. During development, AI tools were used at selected stages to explore implementation approaches, understand technical concepts, discuss feature logic, troubleshoot errors and review parts of the code.

Suggestions generated by AI tools were reviewed against the project requirements and existing code structure. Necessary modifications were made, and changes were tested before being incorporated into the application.

The following prompts are selected examples of how AI tools were used during different stages of the project. They have been edited for clarity and are not complete or verbatim chat transcripts.

Prompt 1 – Backend Architecture and REST API Development

Review the backend architecture of my Smart Building Energy Peak-Shaving Controller, developed using Java 21, Spring Boot and MySQL. Explain the responsibilities of entities, repositories, services and REST controllers for buildings, equipment, meter readings, occupancy schedules, peak events and load schedules. Show how requests flow between these layers, where validations should be performed and how business rules should be maintained in the service layer. Include relevant code examples and API interaction flows. The application must remain simulation-based and must not control physical building equipment.

Prompt 2 – Peak-Shaving Algorithm and Scheduling Logic

Work through the implementation of the peak-shaving decision process. Demonstrate how the application detects demand exceeding a configured threshold and identifies suitable equipment for load-reduction recommendations. Consider equipment priority, criticality, controllability, occupancy schedules, comfort restrictions and available load capacity. Include pseudocode or Java examples for equipment eligibility, scheduling decisions and rejection conditions. Explain how the system handles cases where no equipment is eligible. Critical equipment must never be selected, and all generated schedules must remain advisory.

Prompt 3 – React Frontend Development and API Integration

Review the existing React/Vite frontend and its integration with the Spring Boot backend. Focus on the operator's interaction with the application, including the dashboard, buildings, equipment, meter readings, occupancy schedules, peak events, configuration and schedule approval screens. Demonstrate how existing REST endpoints can be integrated into React components. Include examples of API calls, state management, form validation, loading indicators, empty states and error handling. Explain how the frontend should refresh and display updated information after schedule approval or rejection.

Prompt 4 – Python Meter Simulation and Data Processing

Trace the Python-based meter ingestion workflow using Pandas, NumPy and Requests. Show how the simulator retrieves building information and recent readings through Spring Boot APIs, generates synthetic meter readings, validates records, separates accepted and rejected data, and submits valid readings to the backend. Include basic Python code and explain how to handle duplicate timestamps, missing building IDs, stale readings, partial API failures, retries and logging. Explain the purpose of the rejected_readings.csv report. Keep meter simulation separate from the Snowflake analytics ETL process.

Prompt 5 – Snowflake and Power BI Analytics

Review the analytics workflow for GridWise, from operational data stored in MySQL to analytical tables in Snowflake and reporting in Power BI. Discuss database, schema and warehouse configuration, file formats, stages, ETL processing and SQL validation queries. Include examples of checking source and target record counts, handling data inconsistencies and creating Power BI DAX measures for energy consumption, peak demand, estimated reduction and building performance. Clearly distinguish simulated savings from actual measured savings.

Prompt 6 – DevOps Configuration and Technical Troubleshooting

Investigate the Docker, AWS and CI/CD setup for the existing GridWise application. Focus on the React frontend, Spring Boot backend and MySQL containers, Docker Compose configuration, Docker image publishing, AWS EC2 deployment, IAM permissions, security groups and GitHub Actions workflows.

Pay particular attention to the development issues encountered during implementation, including:

- Docker container startup failures, port conflicts and service connectivity.
- Docker Hub authentication and image push failures.
- EC2 website accessibility and security-group configuration.
- IAM role attachment and S3 access-permission errors.
- GitHub Actions workflow failures and self-hosted runner connectivity.
- Deployment verification and application log inspection.
-

For each issue, identify the likely cause, provide the commands or configuration checks required, explain the corrective steps and show how to verify the result. Use the existing project configuration as the starting point. Avoid destructive changes without explaining their impact.

AI Assistance Note

These prompts represent selected examples of AI-assisted discussions across the project lifecycle. AI tools were used as supporting resources for technical discussions, implementation guidance, code review and troubleshooting. Project-specific decisions, code integration, configuration changes and testing were reviewed and carried out as part of the development process.

.

Appendix C – Troubleshooting Quick Reference

Use this table as a first-pass diagnostic guide. Run commands from the stated environment and avoid destructive resets.

Symptom	Likely areas to inspect	Where to check	Initial action
Website does not open	EC2 state, current public IPv4, security group HTTP rule and frontend container	AWS Console; browser; PuTTY	Confirm instance is running; allow TCP 80 from intended audience; use current public IP; check `docker ps` and `curl -I http://localhost`.
Connection times out	Inbound rule, public IP, route or host firewall	AWS Console; PuTTY	Verify HTTP TCP 80 inbound rule and public IPv4. Keep SSH restricted to administrator IP.

Symptom	Likely areas to inspect	Where to check	Initial action
Nginx returns 502	Frontend proxy target or backend service availability	PuTTY	Check frontend and backend logs; confirm Compose service name, internal port and shared network.
Frontend loads but data is missing	Wrong API base URL, CORS mismatch or backend unavailable	Browser DevTools; PuTTY	Inspect Network/Console; verify frontend API URL and backend logs; check configured CORS origin.
Backend container exits/restarts	Database connectivity, schema validation, environment variables or application exception	PuTTY	Run `docker compose -f docker-compose.aws.yml logs --tail=100 backend`; check DB variables, MySQL health and schema compatibility.
MySQL container is unhealthy	Initialization, storage, credentials or database startup	PuTTY	Inspect MySQL logs and volume status. Do not delete the volume as a first response.
Port 80 is already in use	Another process/container owns the host port	PuTTY	Run `sudo ss -tulpn | grep :80` and `sudo docker ps`; identify the owner before changing mappings.
ETL fails or data is missing in Snowflake	Source extraction, watermark, credentials, target schema or load step	Python/Backend logs; Snowflake	Check each extract/transform/load/reconcile stage and compare source/target counts for the same window.
GitHub deployment does not run	Workflow failure, secrets, branch trigger or self-hosted runner offline	GitHub Actions; PuTTY	Inspect the failed job step; verify required repository secrets and runner service status.
S3 backup access denied	EC2 role attachment or policy resource/action scope	PuTTY; AWS IAM	Run `aws sts get-caller-identity`; review the instance role and least-privilege bucket/prefix policy.
Backup file may be incomplete	Dump command failed after output file was created	PuTTY	Check command exit status, file size and dump completion marker before upload; test restore separately.